1.1 ./Link.Foundation.Links.Notation/Binary/ArityRange.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Diagnostics.CodeAnalysis;
using System.Globalization;

namespace Link.Foundation.Links.Notation.Binary;

/// <summary>
/// How many references the links of a <see cref="Section"/> hold:
/// <c>Min..=Max</c>, where a null <see cref="Max"/> means no upper bound.
/// </summary>
/// <param name="Min">Fewest references in a link, at least 1.</param>
/// <param name="Max">Most references in a link, null for no limit.</param>
public readonly record struct ArityRange(ulong Min, ulong? Max)
{
    /// <summary>The largest minimum arity a section shape can hold.</summary>
    public const ulong MaxMin = ulong.MaxValue >> 4;

    /// <summary>Links of exactly two references.</summary>
    public static ArityRange Doublets { get; } = Exactly(2);

    /// <summary>Links of exactly <paramref name="arity"/> references.</summary>
    public static ArityRange Exactly(ulong arity) => new(arity, arity);

    /// <summary>Links of at least <paramref name="min"/> references.</summary>
    public static ArityRange AtLeast(ulong min) => new(min, null);

    /// <summary>Links of <c>min..=max</c> references.</summary>
    public static ArityRange Between(ulong min, ulong max) => new(min, max);

    /// <summary>True when a link of <paramref name="length"/> references fits the
    ↪ range.</summary>
    public bool Contains(ulong length) => length >= Min && (Max is not { } max || length <= max);

    /// <summary>True when every link has the same number of references, so links need no length
    ↪ prefix.</summary>
    public bool IsFixed => Max == Min;

    /// <summary>0 for no maximum, otherwise <c>Max - Min</c>; only for variable
    ↪ arities.</summary>
    internal ulong Extra => Max is { } max ? max - Min : 0;

    /// <summary>Null for a usable range, otherwise why it is not.</summary>
    internal string? Problem()
    {
        if (Min == 0)
        {
            return "arity must be at least 1";
        }
        if (Min > MaxMin)
        {
            return $"arity {Min} is too large";
        }
        if (Max is { } max && max < Min)
        {
            return $"arity range {this} is empty";
        }
        return null;
    }

    /// <summary>The range a section header describes; <paramref name="extra"/> 0 means no
    ↪ maximum.</summary>
    internal static ArityRange FromShape(ulong min, ulong? extra)
    {
        ArityRange range;
        if (extra is not { } added)
        {
            range = Exactly(min);
        }
        else if (added == 0)
        {
            range = AtLeast(min);
        }
        else
        {
```

```
            if (min > ulong.MaxValue - added)
            {
                throw BinaryNotationException.Malformed("arity range overflows 64 bits");
            }
            range = Between(min, min + added);
        }
        if (range.Problem() is { } problem)
        {
            throw BinaryNotationException.Malformed(problem);
        }
        return range;
    }

    /// <summary>Formats the range as <c>n</c>, <c>min..max</c> or <c>min..</c>.</summary>
    public override string ToString() => Max switch
    {
        { } max when max == Min => max.ToString(CultureInfo.InvariantCulture),
        { } max => $"{Min}..{max}",
        null => $"{Min}..",
    };                          -

    /// <summary>Parses <c>n</c>, <c>min..max</c> (inclusive) or <c>min..</c> (no
    ↪ maximum).</summary>
    /// <exception cref="FormatException">The text is not a usable arity range.</exception>
    public static ArityRange Parse(string text) =>
        TryParse(text, out var range, out var error) ? range : throw new FormatException(error);

    /// <summary>Parses <c>n</c>, <c>min..max</c> (inclusive) or <c>min..</c> (no
    ↪ maximum).</summary>
    public static bool TryParse([NotNullWhen(true)] string? text, out ArityRange range) =>
        TryParse(text, out range, out var ignored);

    /// <summary>Parses an arity range, explaining in <paramref name="error"/> why it
    ↪ failed.</summary>
    public static bool TryParse([NotNullWhen(true)] string? text, out ArityRange range,
    ↪ [NotNullWhen(false)] out string? error)
    {
        range = default;
        error = $"invalid arity '{text}': expected n, min..max or min..";
        if (text is null)
        {
            return false;
        }
        var separator = text.IndexOf("..", StringComparison.Ordinal);
        if (separator < 0)
        {
            if (!TryParseNumber(text, out var arity))
            {
                return false;
            }
            range = Exactly(arity);
        }
        else
        {
            var maxText = text[(separator + 2)..];
            if (!TryParseNumber(text[..separator], out var min))
            {
                return false;
            }
            if (maxText.Length == 0)
            {
                range = AtLeast(min);
            }
            else if (TryParseNumber(maxText, out var max))
            {
                range = Between(min, max);
            }
            else
            {
                return false;
            }
        }
        error = range.Problem();
        return error is null;
    }

    // Like Rust's u64::from_str: ASCII digits with an optional leading '+'.
    private static bool TryParseNumber(string text, out ulong value)
```

```
    {
        var digits = text.StartsWith('+') ? text[1..] : text;
        return ulong.TryParse(digits, NumberStyles.None, CultureInfo.InvariantCulture, out
↪   value);
    }
}
```

1.2 ./Link.Foundation.Links.Notation/Binary/BinaryLinoCodec.cs

```
using System.Collections.Generic;
using LinoLink = Link.Foundation.Links.Notation.Link<string>;

namespace Link.Foundation.Links.Notation.Binary;

/// <summary>Convenience codec for complete LiNo packets.</summary>
public sealed record BinaryLinoCodec
{
    public BinaryLinoCodec() { }

    public BinaryLinoCodec(BinaryLinoOptions options) => Options = options;

    /// <summary>Options used when writing.</summary>
    public BinaryLinoOptions Options { get; init; }

    /// <summary>Limits applied when reading and writing.</summary>
    public DecodeLimits Limits { get; init; } = DecodeLimits.Default;

    /// <summary>Encodes the supplied model without canonicalizing its groups.</summary>
    public byte[] Encode(IReadOnlyList<LinoLink> document) =>
↪   LinoMapping.EncodeDocument(document, Options, Limits).ToBytes();

    /// <summary>Decodes exactly one complete packet, rejecting trailing bytes.</summary>
    public IReadOnlyList<LinoLink> Decode(byte[] bytes) =>
        LinoMapping.DecodeDocument(LinksPacket.FromBytes(bytes, Limits), Limits);
}
```

1.3 ./Link.Foundation.Links.Notation/Binary/BinaryNotationException.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
namespace Link.Foundation.Links.Notation.Binary;

/// <summary>What went wrong while encoding, decoding or exchanging a LiNo message.</summary>
public enum BinaryErrorKind
{
    /// <summary>The underlying transport failed.</summary>
    Io,
    /// <summary>The peer sent bytes that are not a valid message.</summary>
    Malformed,
    /// <summary>The message text is not valid LiNo.</summary>
    InvalidLino,
    /// <summary>The message exceeds one of the configured <see cref="DecodeLimits"/>.</summary>
    LimitExceeded,
    /// <summary>The document cannot be represented with the chosen options.</summary>
    Unencodable,
}

/// <summary>Raised by the binary links notation codecs.</summary>
public sealed class BinaryNotationException : Exception
{
    public BinaryNotationException()
        : this(BinaryErrorKind.Malformed, "unknown protocol error")
    {
    }

    public BinaryNotationException(string message)
        : this(BinaryErrorKind.Malformed, message)
    {
    }

    public BinaryNotationException(string message, Exception innerException)
        : this(BinaryErrorKind.Io, message, innerException)
    {
    }

    public BinaryNotationException(BinaryErrorKind kind, string detail, Exception?
↪   innerException = null)
        : base($"{Describe(kind)}: {detail}", innerException)
    {
```

```
        Kind = kind;
        Detail = detail;
    }

    /// <summary>The category of the failure.</summary>
    public BinaryErrorKind Kind { get; }

    /// <summary>The message without the category prefix.</summary>
    public string Detail { get; }

    internal static BinaryNotationException Malformed(string detail) =>
    ↪ new(BinaryErrorKind.Malformed, detail);

    internal static BinaryNotationException Limit(string detail) =>
    ↪ new(BinaryErrorKind.LimitExceeded, detail);

    internal static BinaryNotationException Unencodable(string detail) =>
    ↪ new(BinaryErrorKind.Unencodable, detail);

    private static string Describe(BinaryErrorKind kind) => kind switch
    {
        BinaryErrorKind.Io => "I/O error",
        BinaryErrorKind.Malformed => "malformed message",
        BinaryErrorKind.InvalidLino => "invalid LiNo",
        BinaryErrorKind.LimitExceeded => "limit exceeded",
        BinaryErrorKind.Unencodable => "cannot encode",
          => kind.ToString(),
    };
}
```

1.4 ./Link.Foundation.Links.Notation/Binary/LinksPacket.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Buffers.Binary;

namespace Link.Foundation.Links.Notation.Binary;

/// <summary>One reference inside a <see cref="LinksPacket"/>.</summary>
/// <param name="IsExternal">True for an external value (a number or a code point).</param>
/// <param name="Value">The address inside the packet, or the external value.</param>
public readonly record struct PacketReference(bool IsExternal, ulong Value)
{
    /// <summary>The null reference.</summary>
    public static readonly PacketReference Null = Internal(LinksPacket.NullAddress);

    /// <summary>An address inside the packet (0 null, 1..5 markers, then links).</summary>
    public static PacketReference Internal(ulong address) => new(false, address);

    /// <summary>An external value, e.g. a number or a Unicode code point.</summary>
    public static PacketReference External(ulong value) => new(true, value);

    public override string ToString() => IsExternal ? $"External({Value})" :
    ↪ $"Internal({Value})";
}

/// <summary>Safety limits applied while decoding untrusted input.</summary>
public sealed record DecodeLimits
{
    /// <summary>Maximum number of links in a packet.</summary>
    public ulong MaxLinks { get; init; } = 1UL << 22;

    /// <summary>Maximum number of references in all links of a packet.</summary>
    public ulong MaxReferences { get; init; } = 1UL << 24;

    /// <summary>Maximum number of LiNo nodes a packet may expand to.</summary>
    public long MaxNodes { get; init; } = 1L << 22;

    /// <summary>Maximum total UTF-8 bytes in expanded references, including link ids.</summary>
    public long MaxStringBytes { get; init; } = 64L << 20;

    /// <summary>Maximum LiNo nesting depth.</summary>
    public int MaxDepth { get; init; } = 64;

    internal void Validate()
    {
        if (MaxNodes < 0 || MaxStringBytes < 0 || MaxDepth < 0)
            throw new ArgumentOutOfRangeException(nameof(DecodeLimits), "Limits must be
    ↪ non-negative.");
```

```
    }

    /// <summary>The default limits.</summary>
    public static DecodeLimits Default { get; } = new();

    /// <summary>Limits for trusted input such as a store archive: only the address space bounds the packet.</summary>
    public static DecodeLimits Unlimited { get; } = new()
    {
        MaxLinks = ulong.MaxValue,
        MaxReferences = ulong.MaxValue,
        MaxNodes = long.MaxValue,
        MaxStringBytes = long.MaxValue,
        MaxDepth = int.MaxValue,
    };
}

/// <summary>A run of links at consecutive addresses sharing an arity range and a reference width.</summary>
public sealed class Section
{
    /// <summary>Addresses skipped before the first link of the section.</summary>
    public ulong Gap { get; set; }

    /// <summary>The number of references each link may hold.</summary>
    public ArityRange Arity { get; set; } = ArityRange.Doublets;

    /// <summary>Bytes per reference: 1, 2, 4 or 8.</summary>
    public byte Width { get; set; } = 1;

    /// <summary>The links, in address order.</summary>
    public List<PacketReference[]> Links { get; } = new();

    internal void WriteHeader(List<byte> output)
    {
        var shape = LinksPacket.WidthCode(Width) | (Arity.Min << LinksPacket.ShapeMinArityShift);
        if (Gap != 0)
        {
            shape |= LinksPacket.ShapeHasGap;
        }
        if (!Arity.IsFixed)
        {
            shape |= LinksPacket.ShapeVariableArity;
        }
        LinksPacket.WriteLeb128(output, shape);
        if (Gap != 0)
        {
            LinksPacket.WriteLeb128(output, Gap);
        }
        if (!Arity.IsFixed)
        {
            LinksPacket.WriteLeb128(output, Arity.Extra);
        }
        LinksPacket.WriteLeb128(output, (ulong)Links.Count);
    }

    /// <summary>Reads a section header, returning the still empty section and its link count.</summary>
    internal static (Section Section, ulong Count) ReadHeader(PacketReader reader)
    {
        var shape = LinksPacket.ReadLeb128(reader);
        var width = LinksPacket.WidthFromCode(shape & LinksPacket.ShapeWidthBits);
        var gap = (shape & LinksPacket.ShapeHasGap) != 0 ? LinksPacket.ReadLeb128(reader) : 0;
        ulong? extra = (shape & LinksPacket.ShapeVariableArity) != 0 ? LinksPacket.ReadLeb128(reader) : null;
        var arity = ArityRange.FromShape(shape >> LinksPacket.ShapeMinArityShift, extra);
        var count = LinksPacket.ReadLeb128(reader);
        return (new Section { Gap = gap, Arity = arity, Width = width }, count);
    }

    internal void Validate()
    {
        LinksPacket.WidthCode(Width);
        if (Arity.Problem() is { } problem)
        {
            throw BinaryNotationException.Unencodable(problem);
        }
        foreach (var link in Links)
```

```
        {
            if (!Arity.Contains((ulong)link.Length))
            {
                throw BinaryNotationException.Unencodable(
                    $"a link of {link.Length} references in a section of arity {Arity}");
            }
        }
    }
}

/// <summary>
/// Binary links notation: a self-delimiting packet of links.
/// </summary>
/// <remarks>
/// A packet stores links, each a tuple of one or more references, at
/// implicit consecutive addresses. It knows nothing about LiNo; the LiNo
/// protocol (<see cref="LinoMapping"/>) is one use of it.
/// <code>
/// byte 0     0x10 | flags      high nibble 1 = format version 1
///                              bit 0     external references (Hybrid encoding)
///                              bit 1     explicit layout
///                              bits 2-3  log2 of the width (compact layout only)
///
/// compact layout (bit 1 clear): one section of doublets right after the markers
/// LEB128     N                 number of links, each `source target`
///
/// explicit layout (bit 1 set):
/// LEB128     S                 number of sections, then S section headers:
/// LEB128     shape             bits 0-1  log2 of the reference width in bytes
///                              bit 2     a gap follows
///                              bit 3     variable arity (else every link
///                                        holds exactly min arity references)
///                              bits 4+   min arity, at least 1
/// LEB128     gap               addresses skipped before the section (if bit 2)
/// LEB128     extra arity       0 = no maximum, else max - min (if bit 3)
/// LEB128     count             number of links in the section
///
/// links, section by section; a link in a variable-arity section starts
/// with LEB128 (length - min arity); every reference is `width` bytes,
/// little-endian
/// </code>
/// Address 0 is null. The first section starts at <c>1 + gap</c> and every
/// other section at <c>previous end + gap</c>, so gaps leave holes. The
/// compact layout is exactly one section with gap 5 (the LiNo marker points
/// 1..5), arity 2 and the header width. Each section has its own width: the
/// narrowest of 1, 2, 4 and 8 bytes that holds every reference in it;
/// <see cref="Pack"/> chooses the sections. With external references
/// enabled the top bit of a reference marks it as external, exactly like
/// <c>Platform.Data.Hybrid&amp;lt;T&amp;gt;</c>, which halves every internal range.
/// The byte layout is identical to the Rust <c>links notation::binary::LinksPacket</c>.
/// </remarks>
public sealed class LinksPacket
{
    /// <summary>The high nibble of the header byte; text never starts with 0x10..0x1F.</summary>
    public const byte BinaryVersion1 = 0x10;

    /// <summary>Null link address.</summary>
    public const ulong NullAddress = 0;
    /// <summary>Marker 1: the unary one; 2^k = (2^(k-1) 2^(k-1)).</summary>
    public const ulong One = 1;
    /// <summary>Marker 2: (Number unary) is a non-negative integer.</summary>
    public const ulong Number = 2;
    /// <summary>Marker 3: (String code points...) is a Unicode string.</summary>
    public const ulong String = 3;
    /// <summary>Marker 4: (List elements...) is a list of links.</summary>
    public const ulong List = 4;
    /// <summary>Marker 5: (Identified id values...) is a link with an id.</summary>
    public const ulong Identified = 5;
    /// <summary>Address of the first link after the marker points.</summary>
    public const ulong FirstLinkAddress = 6;

    private const byte FlagExternalReferences = 0b0001;
    private const byte FlagExplicitLayout = 0b0010;
    private const int WidthShift = 2;
    private const byte WidthBits = 0b1100;

    internal const ulong ShapeWidthBits = 0b0011;
    internal const ulong ShapeHasGap = 0b0100;
    internal const ulong ShapeVariableArity = 0b1000;
```

```
    internal const int ShapeMinAritySift = 4;

    /// <summary>The addresses the compact layout skips: the marker points.</summary>
    private const ulong CompactGap = FirstLinkAddress - 1;

    private static readonly byte[] WidthValues = { 1, 2, 4, 8 };

    /// <summary>The reference widths, in bytes, that a packet may use.</summary>
    public static IReadOnlyList<byte> Widths => WidthValues;

    /// <summary>An empty packet.</summary>
    public LinksPacket(bool externalReferences = false) => ExternalReferences =
    externalReferences;

    /// <summary>Header bit 0: references may be external (Hybrid encoding).</summary>
    public bool ExternalReferences { get; set; }

    /// <summary>The links, section by section.</summary>
    public List<Section> Sections { get; } = new();

    /// <summary>Largest internal address that fits in <paramref name="width"/> bytes.</summary>
    public static ulong InternalCapacity(byte width, bool externalReferences)
    {
        var bits = width * 8 - (externalReferences ? 1 : 0);
        return bits >= 64 ? ulong.MaxValue : (1UL << bits) - 1;
    }

    /// <summary>Largest external value that fits in <paramref name="width"/> bytes.</summary>
    public static ulong ExternalCapacity(byte width) => (1UL << (width * 8 - 1)) - 1;

    /// <summary>The narrowest width able to hold the internal address.</summary>
    public static byte AddressTier(ulong address, bool externalReferences)
    {
        foreach (var width in WidthValues)
        {
            if (InternalCapacity(width, externalReferences) >= address)
            {
                return width;
            }
        }
        throw BinaryNotationException.Unencodable($"address {address} exceeds the internal
    range");
    }

    private static ulong WidthMask(byte width) => InternalCapacity(width, false);

    /// <summary>Encodes an external value at <paramref name="width"/> the way Hybrid&lt;T&gt;
    does.</summary>
    public static ulong EncodeExternal(ulong value, byte width) =>
        value == 0 ? 1UL << (width * 8 - 1) : (0UL - value) & WidthMask(width);

    /// <summary>Decodes a raw value; returns true and the value for externals.</summary>
    public static bool TryDecodeExternal(ulong raw, byte width, out ulong value)
    {
        var externalZero = 1UL << (width * 8 - 1);
        value = 0;
        if (raw == externalZero)
        {
            return true;
        }
        if (raw > externalZero)
        {
            value = (0UL - raw) & WidthMask(width);
            return true;
        }
        return false;
    }

    internal static ulong WidthCode(byte width)
    {
        var code = Array.IndexOf(WidthValues, width);
        return code >= 0 ? (ulong)code : throw BinaryNotationException.Unencodable($"invalid
    width {width}");
    }

    /// <summary>The width of a two-bit width code; every code names a width.</summary>
    internal static byte WidthFromCode(ulong code) => WidthValues[code & 0b11];

    /// <summary>The narrowest width able to hold <paramref name="reference"/>.</summary>
```

```
    public static byte ReferenceWidth(PacketReference reference, bool externalReferences)
    {
        if (!reference.IsExternal)
        {
            return AddressTier(reference.Value, externalReferences);
        }
        if (!externalReferences)
        {
            throw BinaryNotationException.Unencodable("external reference in a packet without
↪   external references");
        }
        foreach (var width in WidthValues)
        {
            if (ExternalCapacity(width) >= reference.Value)
            {
                return width;
            }
        }
        throw BinaryNotationException.Unencodable($"external value {reference.Value} exceeds 63
↪   bits");
    }

    /// <summary>
    /// Lays out <paramref name="links"/> - addresses with their references, in
    /// ascending address order - using the version 1 section planner.
    /// </summary>
    /// <remarks>
    /// Holes between addresses start new sections. Without
    /// <paramref name="packedWidths"/> every section uses the width of the
    /// widest reference, so all references have the same size. With it each
    /// section gets the narrowest width its links need and sections split
    /// wherever that saves bytes; the result is never larger than the uniform one.
    /// </remarks>
    public static LinksPacket Pack(
        bool externalReferences,
        IReadOnlyList<(ulong Address, PacketReference[] References)> links,
        bool packedWidths)
    {
        ArgumentNullException.ThrowIfNull(links);
        var planner = new SectionPlanner(externalReferences, links);
        var uniformLayout = planner.Plan(false);
        var uniform = LayOut(externalReferences, links, uniformLayout);
        if (!packedWidths)
        {
            return uniform;
        }
        var packedLayout = planner.Plan(true);
        if (packedLayout.SequenceEqual(uniformLayout))
        {
            return uniform;
        }
        var packed = LayOut(externalReferences, links, packedLayout);
        return packed.ToBytes().Length < uniform.ToBytes().Length ? packed : uniform;
    }

    /// <summary>Splits <paramref name="links"/> into sections of <c>(link count,
↪   width)</c>.</summary>
    private static LinksPacket LayOut(
        bool externalReferences,
        IReadOnlyList<(ulong Address, PacketReference[] References)> links,
        IReadOnlyList<(int Count, byte Width)> layout)
    {
        var packet = new LinksPacket(externalReferences);
        var nextAddress = 1UL;
        var first = 0;
        foreach (var (count, width) in layout)
        {
            var start = links[first].Address;
            var shortest = ulong.MaxValue;
            var longest = 0UL;
            var section = new Section { Gap = start - nextAddress, Width = width };
            for (var index = first; index < first + count; index++)
            {
                var length = (ulong)links[index].References.Length;
                shortest = Math.Min(shortest, length);
                longest = Math.Max(longest, length);
                section.Links.Add(links[index].References);
            }
```

```
            section.Arity = ArityRange.Between(shortest, longest);
            packet.Sections.Add(section);
            nextAddress = start + (ulong)count;
            first += count;
        }
        return packet;
    }

    /// <summary>Every link with its address, in address order.</summary>
    public IEnumerable<(ulong Address, PacketReference[] References)> Links()
    {
        var nextAddress = 1UL;
        foreach (var section in Sections)
        {
            var start = SaturatingAdd(nextAddress, section.Gap);
            nextAddress = SaturatingAdd(start, (ulong)section.Links.Count);
            for (var index = 0; index < section.Links.Count; index++)
            {
                yield return (start + (ulong)index, section.Links[index]);
            }
        }
    }

    private static ulong SaturatingAdd(ulong left, ulong right) =>
        left > ulong.MaxValue - right ? ulong.MaxValue : left + right;

    /// <summary>The number of links in the packet.</summary>
    public ulong LinkCount => Sections.Aggregate(0UL, (total, section) => total +
    ↪ (ulong)section.Links.Count);

    private bool IsCompact => Sections.Count switch
    {
        0 => true,
        1 => Sections[0].Gap == CompactGap && Sections[0].Arity == ArityRange.Doublets &&
    ↪ Sections[0].Links.Count > 0,
          => false,
    };

    internal void Validate()
    {
        var nextAddress = 1UL;
        foreach (var section in Sections)
        {
            section.Validate();
            var count = (ulong)section.Links.Count;
            if (section.Gap > ulong.MaxValue - nextAddress || count > ulong.MaxValue -
    ↪ nextAddress - section.Gap)
            {
                throw BinaryNotationException.Unencodable("addresses overflow 64 bits");
            }
            nextAddress += section.Gap + count;
            foreach (var link in section.Links)
            {
                foreach (var reference in link)
                {
                    RawReference(reference, section.Width);
                }
            }
        }
    }

    /// <summary>Serializes the packet.</summary>
    public byte[] ToBytes()
    {
        Validate();
        var header = BinaryVersion1;
        if (ExternalReferences)
        {
            header |= FlagExternalReferences;
        }
        var output = new List<byte>();
        if (IsCompact)
        {
            var width = Sections.Count > 0 ? Sections[0].Width : (byte)1;
            output.Add((byte)(header | (WidthCode(width) << WidthShift)));
            WriteLeb128(output, LinkCount);
        }
        else
        {
```

```
            output.Add((byte)(header | FlagExplicitLayout));
            WriteLeb128(output, (ulong)Sections.Count);
            foreach (var section in Sections)
            {
                section.WriteHeader(output);
            }
        }
        foreach (var section in Sections)
        {
            foreach (var link in section.Links)
            {
                if (!section.Arity.IsFixed)
                {
                    WriteLeb128(output, (ulong)link.Length - section.Arity.Min);
                }
                foreach (var reference in link)
                {
                    WriteRaw(output, RawReference(reference, section.Width), section.Width);
                }
            }
        }
        return output.ToArray();
    }

    private ulong RawReference(PacketReference reference, byte width)
    {
        if (ReferenceWidth(reference, ExternalReferences) > width)
        {
            throw BinaryNotationException.Unencodable($"{reference} does not fit {width}
↪ byte(s)");
        }
        return reference.IsExternal ? EncodeExternal(reference.Value, width) : reference.Value;
    }

    /// <summary>Parses a complete packet; trailing bytes are an error.</summary>
    public static LinksPacket FromBytes(byte[] bytes, DecodeLimits? limits = null)
    {
        var reader = PacketReader.FromBytes(bytes);
        var packet = ReadFrom(reader, limits) ?? throw BinaryNotationException.Malformed("empty
↪ input");
        if (!reader.AtEnd)
        {
            throw BinaryNotationException.Malformed("trailing bytes after the packet");
        }
        return packet;
    }

    /// <summary>Reads one packet; null on a clean end of stream.</summary>
    public static LinksPacket? ReadFrom(PacketReader reader, DecodeLimits? limits = null)
    {
        ArgumentNullException.ThrowIfNull(reader);
        limits ??= DecodeLimits.Default;
        limits.Validate();
        var first = reader.ReadByte();
        if (first < 0)
        {
            return null;
        }
        var header = (byte)first;
        if ((header & 0xF0) != BinaryVersion1)
        {
            throw BinaryNotationException.Malformed($"unsupported binary header byte
↪ 0x{header:X2}");
        }
        var packet = new LinksPacket((header & FlagExternalReferences) != 0);
        var counts = new List<ulong>();
        if ((header & FlagExplicitLayout) == 0)
        {
            var width = WidthFromCode((ulong)((header & WidthBits) >> WidthShift));
            var count = ReadLeb128(reader);
            if (count > ulong.MaxValue - FirstLinkAddress)
            {
                throw BinaryNotationException.Malformed("addresses overflow 64 bits");
            }
            if (count > 0)
            {
                packet.Sections.Add(new Section { Gap = CompactGap, Arity = ArityRange.Doublets,
↪ Width = width });
```

```
                counts.Add(count);
            }
        }
        else
        {
            if ((header & WidthBits) != 0)
            {
                throw BinaryNotationException.Malformed("the explicit layout keeps the header width bits clear");
            }
            var sectionCount = ReadLeb128(reader);
            if (sectionCount > limits.MaxLinks)
            {
                throw TooManyLinks(limits);
            }
            var nextAddress = 1UL;
            for (ulong index = 0; index < sectionCount; index++)
            {
                var (section, count) = Section.ReadHeader(reader);
                if (section.Gap > ulong.MaxValue - nextAddress || count > ulong.MaxValue - nextAddress - section.Gap)
                {
                    throw BinaryNotationException.Malformed("addresses overflow 64 bits");
                }
                nextAddress += section.Gap + count;
                packet.Sections.Add(section);
                counts.Add(count);
            }
        }
        var total = 0UL;
        foreach (var count in counts)
        {
            if (count > limits.MaxLinks - total)
            {
                throw TooManyLinks(limits);
            }
            total += count;
        }
        var referencesLeft = limits.MaxReferences;
        for (var index = 0; index < packet.Sections.Count; index++)
        {
            var section = packet.Sections[index];
            section.Links.Capacity = (int)Math.Min(counts[index], 4096);
            for (ulong link = 0; link < counts[index]; link++)
            {
                var length = section.Arity.Min;
                if (!section.Arity.IsFixed)
                {
                    var extra = ReadLeb128(reader);
                    if (extra > ulong.MaxValue - length || !section.Arity.Contains(extra + length))
                    {
                        throw BinaryNotationException.Malformed($"link length outside the section arity {section.Arity}");
                    }
                    length += extra;
                }
                if (length > referencesLeft)
                {
                    throw BinaryNotationException.Limit($"references exceed the limit of {limits.MaxReferences}");
                }
                referencesLeft -= length;
                var references = new List<PacketReference>((int)Math.Min(length, 4096));
                for (ulong item = 0; item < length; item++)
                {
                    var raw = ReadRaw(reader, section.Width);
                    references.Add(packet.ExternalReferences && TryDecodeExternal(raw, section.Width, out var value)
                        ? PacketReference.External(value)
                        : PacketReference.Internal(raw));
                }
                section.Links.Add(references.ToArray());
            }
        }
        return packet;
    }

```

```
    private static BinaryNotationException TooManyLinks(DecodeLimits limits) =>
        BinaryNotationException.Limit($"packet declares more than {limits.MaxLinks} links");

    private static void WriteRaw(List<byte> output, ulong value, byte width)
    {
        for (var index = 0; index < width; index++)
        {
            output.Add((byte)(value >> (8 * index)));
        }
    }

    private static ulong ReadRaw(PacketReader reader, byte width)
    {
        Span<byte> bytes = stackalloc byte[8];
        bytes.Clear();
        reader.ReadExactly(bytes[..width]);
        return BinaryPrimitives.ReadUInt64LittleEndian(bytes);
    }

    /// <summary>Appends <paramref name="value"/> as unsigned LEB128.</summary>
    public static void WriteLeb128(List<byte> output, ulong value)
    {
        ArgumentNullException.ThrowIfNull(output);
        while (true)
        {
            var current = (byte)(value & 0x7F);
            value >>= 7;
            if (value == 0)
            {
                output.Add(current);
                return;
            }
            output.Add((byte)(current | 0x80));
        }
    }

    /// <summary>Reads an unsigned LEB128 value of at most 64 bits.</summary>
    public static ulong ReadLeb128(PacketReader reader)
    {
        ArgumentNullException.ThrowIfNull(reader);
        ulong value = 0;
        for (var shift = 0; shift < 64; shift += 7)
        {
            var next = reader.ReadByte();
            if (next < 0)
            {
                throw BinaryNotationException.Malformed("unexpected end of packet");
            }
            var payload = (ulong)(next & 0x7F);
            if (shift == 63 && payload > 1)
            {
                throw BinaryNotationException.Malformed("LEB128 value overflows 64 bits");
            }
            value |= payload << shift;
            if ((next & 0x80) == 0)
            {
                return value;
            }
        }
        throw BinaryNotationException.Malformed("LEB128 value overflows 64 bits");
    }
}
```

1.5 ./Link.Foundation.Links.Notation/Binary/LinoFormat.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using Link.Foundation.Links.Notation;

using LinoLink = Link.Foundation.Links.Notation.Link<string>;

namespace Link.Foundation.Links.Notation.Binary;

/// <summary>
/// Canonical LiNo text for documents, shared by every protocol.
/// </summary>
/// <remarks>
/// A document is the list of top-level links of a message. A reference is a
```

```
/// <see cref="LinoLink"/> whose <c>Values</c> is null. links-notation keeps
/// one extra wrapper around single-reference groups (<c>a</c> parses as
/// <c>(a)</c>); <see cref="ParseDocument"/> removes it, exactly like the Rust
/// port, so documents decoded from the same bytes compare equal in both
/// ports. Formatting preserves the canonical model of parsed documents.
/// Manually constructed ids without values cannot round trip through text.
/// </remarks>
public static class LinoFormat
{

    /// <summary>A reference: a link without values.</summary>
    public static LinoLink Reference(string text) => new(text);

    /// <summary>A link with an optional id and the given values.</summary>
    public static LinoLink Link(string? id, IList<LinoLink> values) => new(id!, values);

    /// <summary>Parses LiNo text into a canonical document. Blank input is the empty document.</summary>
    public static IReadOnlyList<LinoLink> ParseDocument(string text)
    {
        ArgumentNullException.ThrowIfNull(text);
        if (text.All(character => character is ' ' or '\t' or '\r' or '\n'))
        {
            return Array.Empty<LinoLink>();
        }
        IList<LinoLink> parsed;
        try
        {
            parsed = new Parser().Parse(text);
        }
        catch (Exception exception) when (exception is not OutOfMemoryException)
        {
            throw new BinaryNotationException(BinaryErrorKind.InvalidLino, exception.Message, exception);
        }
        return parsed.Select(Canonical).ToList();
    }

    /// <summary>
    /// Converts a link produced by the C# parser into the canonical model:
    /// an unnamed group holding exactly one reference is that reference.
    /// </summary>
    public static LinoLink Canonical(LinoLink link)
    {
        if (link.Values is null)
        {
            return link;
        }
        if (link.Id is null && link.Values.Count == 1 && link.Values[0].Values is null)
        {
            return link.Values[0];
        }
        return Link(link.Id, link.Values.Select(Canonical).ToList());
    }

    /// <summary>
    /// Formats a document as canonical LiNo text, one top-level link per line.
    /// A top-level link without an id and with at least two values is written
    /// without its outer parentheses: <c>() ((1 1))</c>.
    /// </summary>
    public static string FormatDocument(IEnumerable<LinoLink> document)
    {
        ArgumentNullException.ThrowIfNull(document);
        return string.Join("\n", document.Select(FormatTopLevel));
    }

    private static string FormatTopLevel(LinoLink link) =>
        link.Values is { Count: >= 2 } values && link.Id is null ? JoinValues(values) : FormatLink(link);

    /// <summary>Formats one link as it appears nested inside another link.</summary>
    public static string FormatLink(LinoLink link)
    {
        if (link.Values is not { } values)
        {
            return FormatReference(link.Id ?? string.Empty);
        }
        if (link.Id is null)
```

```
        {
            // `(a)` parses back as the reference `a`, so a one-reference
            // link needs a second pair of parentheses.
            return values.Count == 1 && values[0].Values is null
                ? $"(({FormatReference(values[0].Id ?? string.Empty)}))"
                : $"({JoinValues(values)})";
        }
        return values.Count == 0
            ? $"({FormatReference(link.Id)}:)"
            : $"({FormatReference(link.Id)}: {JoinValues(values)})";
    }

    private static string JoinValues(IEnumerable<LinoLink> values) => string.Join(" ",
    ↪ values.Select(FormatLink));

    /// <summary>Quotes a reference when it would not survive parsing as a bare word.</summary>
    /// <remarks>
    /// links-notation opens a quoted reference with a run of N equal quote
    /// characters, closes it with the next run of exactly N, and reads 2N
    /// quotes inside as N literal ones. The opening run is counted greedily,
    /// so the chosen quote must differ from the first character; N is one
    /// more than the longest run of that quote inside, and odd, because an
    /// even delimiter run may be read as an empty reference.
    /// </remarks>
    public static string FormatReference(string reference) => ReferenceLiteral.Format(reference);
}
```

1.6 ./Link.Foundation.Links.Notation/Binary/LinoMapping.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Globalization;
using System.Text;

using LinoLink = Link.Foundation.Links.Notation.Link<string>;

namespace Link.Foundation.Links.Notation.Binary;

/// <summary>
/// Optional features of the binary LiNo protocol. Every feature is off by
/// default; each one can be switched on independently, like stacking a decorator.
/// </summary>
public readonly record struct BinaryLinoOptions
{
    // Doublets are stored as null so that default(BinaryLinoOptions) means doublets.
    private readonly ArityRange?  arity;

    /// <summary>
    /// Send numbers and code points as Hybrid external references instead of
    /// in-band unary links. Halves the internal address range of each width.
    /// </summary>
    public bool ExternalReferences { get; init; }

    /// <summary>
    /// The link lengths the encoder may use. The default, exactly 2, sends
    /// only doublets; <c>2..3</c> adds triplets and <c>1..</c> any length.
    /// The range must include 2.
    /// </summary>
    public ArityRange Arity
    {
        get =>  arity ?? ArityRange.Doublets;
        init =>  arity = value == ArityRange.Doublets ? null : value;
    }

    /// <summary>
    /// Give every section of the packet the narrowest width its links need
    /// instead of one width for the whole packet.
    /// </summary>
    public bool PackedWidths { get; init; }

    /// <summary>Enables or disables external references.</summary>
    public BinaryLinoOptions WithExternalReferences(bool enabled = true) => this with {
    ↪ ExternalReferences = enabled };

    /// <summary>Sets the link lengths the encoder may use.</summary>
    public BinaryLinoOptions WithArity(ArityRange arity) => this with { Arity = arity };

    /// <summary>Enables or disables packed widths.</summary>
```

```
    public BinaryLinoOptions WithPackedWidths(bool enabled = true) => this with { PackedWidths = enabled };

    /// <summary>The options a peer most likely used to write <paramref name="packet"/>, so a reply can be written in the same style.</summary>
    public static BinaryLinoOptions OfPacket(LinksPacket packet)
    {
        ArgumentNullException.ThrowIfNull(packet);
        ulong shortest = 2;
        ulong longest = 2;
        foreach (var ( , link) in packet.Links())
        {
            shortest = Math.Min(shortest, (ulong)link.Length);
            longest = Math.Max(longest, (ulong)link.Length);
        }
        return new BinaryLinoOptions
        {
            ExternalReferences = packet.ExternalReferences,
            Arity = ArityRange.Between(shortest, longest),
            PackedWidths = packet.Sections.Select(section => section.Width).Distinct().Skip(1).Any(),
        };
    }

    /// <summary>Lists every option, for test and error messages.</summary>
    public override string ToString() =>
        $"BinaryLinoOptions {{ ExternalReferences = {ExternalReferences}, Arity = {Arity}, PackedWidths = {PackedWidths} }}";
}

/// <summary>
/// Lossless mapping between LiNo documents and <see cref="LinksPacket"/>s,
/// byte-for-byte compatible with the Rust port.
/// </summary>
/// <remarks>
/// <list type="bullet">
/// <item><c>()</c> is the null link 0.</item>
/// <item>A numeric reference n is <c>(Number unary(n))</c>, or an external reference when enabled.</item>
/// <item>Any other reference is <c>(String code points...)</c>; code points are unary numbers or externals.</item>
/// <item>A link without an id and with exactly two values is a plain doublet.</item>
/// <item>A link without an id and any other number of values is a list.</item>
/// <item>A link with an id is <c>(Identified id values...)</c>.</item>
/// <item>The document is a list of its top-level links, stored last (the root).</item>
/// </list>
/// A list or typed value with any number of values other than two is a
/// single link of that many references when <see cref="BinaryLinoOptions.Arity"/>
/// allows it (<c>[marker, elements...]</c> for typed values, the bare elements
/// for lists), and otherwise the doublet <c>(marker chain)</c>, where chain is
/// the nil-terminated cons list <c>(e1 (e2 (... (en 0))))</c>. Identical
/// sub-links are emitted once and shared. Links are numbered so that every
/// link only refers to earlier ones: doublets of plain doublets first, then
/// the rest in creation order.
/// </remarks>
public static class LinoMapping
{
    /// <summary>Converts a document into a packet.</summary>
    public static LinksPacket EncodeDocument(IReadOnlyList<LinoLink> document, BinaryLinoOptions options = default, DecodeLimits? limits = null)
    {
        ArgumentNullException.ThrowIfNull(document);
        if (options.Arity.Problem() is { } problem)
        {
            throw BinaryNotationException.Unencodable(problem);
        }
        if (!options.Arity.Contains(2))
        {
            throw BinaryNotationException.Unencodable($"arity {options.Arity} does not include doublets (2)");
        }
        // Check the model iteratively before entering the recursive encoder.
        limits ??= DecodeLimits.Default;
        limits.Validate();
        var pending = new Stack<(LinoLink Link, int Depth)>(document.Select(link => (link, 0)));
        var budget = limits.MaxNodes;
        var stringsLeft = limits.MaxStringBytes;
        while (pending.TryPop(out var item))
```

```
        {
            if (item.Depth >= limits.MaxDepth)
            {
                throw BinaryNotationException.Unencodable($"nesting deeper than {limits.MaxDepth}");
            }
            if (--budget < 0)
            {
                throw BinaryNotationException.Unencodable("too many LiNo nodes");
            }
            if (item.Link.Id is { } text)
            {
                if (item.Link.Values is not null && --budget < 0)
                    throw BinaryNotationException.Unencodable("too many LiNo nodes");
                var bytes = Encoding.UTF8.GetByteCount(text);
                if (bytes > stringsLeft) throw BinaryNotationException.Unencodable("too many string bytes");
                stringsLeft -= bytes;
            }
            if (item.Link.Values is { } children)
            {
                foreach (var child in children)
                {
                    pending.Push((child, item.Depth + 1));
                }
            }
        }
        var encoder = new Encoder(options);
        if (document.Count > 0)
        {
            encoder.List(document.Select(encoder.Encode).ToList());
        }
        var packet = encoder.Finish();
        if ((ulong)packet.Sections.Count > limits.MaxLinks || packet.LinkCount > limits.MaxLinks)
            throw BinaryNotationException.Unencodable("too many packet links");
        var referencesLeft = limits.MaxReferences;
        foreach (var ( , link) in packet.Links())
        {
            if ((ulong)link.Length > referencesLeft) throw BinaryNotationException.Unencodable("too many packet references");
            referencesLeft -= (ulong)link.Length;
        }
        return packet;
    }

    /// <summary>Converts a packet back into a document.</summary>
    public static IReadOnlyList<LinoLink> DecodeDocument(LinksPacket packet, DecodeLimits? limits = null)
    {
        ArgumentNullException.ThrowIfNull(packet);
        limits ??= DecodeLimits.Default;
        limits.Validate();
        var decoder = new Decoder(packet, limits);
        if (decoder.LinkCount == 0)
        {
            return Array.Empty<LinoLink>();
        }
        var root = decoder.View(PacketReference.Internal(LinksPacket.FirstLinkAddress + (ulong)decoder.LinkCount - 1));
        IReadOnlyList<PacketReference> items = root switch
        {
            { Kind: ViewKind.Link, Items: [{ IsExternal: false, Value: LinksPacket.List }, var chain] } => decoder.Chain(chain),
            { Kind: ViewKind.Link } when !StartsWithMarker(root.Items!) => root.Items!,
              => throw BinaryNotationException.Malformed("the root link is not a list"),
        };
        var budget = limits.MaxNodes;
        return items.Select(item => decoder.Decode(item, 0, ref budget)).ToList();
    }

    /// <summary>Parses a canonical unsigned decimal number (no sign, no leading zeros).</summary>
    public static bool TryParseCanonicalNumber(string text, out ulong value)
    {
        value = 0;
        var canonical = !string.IsNullOrEmpty(text)
            && text.All(character => character is >= '0' and <= '9')
            && (text == "0" || text[0] != '0');
```

```
192         return canonical && ulong.TryParse(text, NumberStyles.None,
    ↪ CultureInfo.InvariantCulture, out value);
193     }
194 
195     private static bool IsMarker(PacketReference reference) =>
196         !reference.IsExternal && reference.Value is >= LinksPacket.One and <
    ↪ LinksPacket.FirstLinkAddress;
197                 -
198     private static bool StartsWithMarker(IReadOnlyList<PacketReference> items) => items.Count >
    ↪ 0 && IsMarker(items[0]);
199 
200     private enum NodeKind
201     {           -
202         Internal,                                       -
203         External,
204         /// ≤summary>A doublet of plain doublets, markers and externals.</summary>
205         Doublet,
206         /// <summary>Any other link; numbered after every doublet.</summary>
207         Tuple,
208     }
209 
210     private readonly record struct Node(NodeKind Kind, ulong Value)
211     {
212         public static Node Internal(ulong address) => new(NodeKind.Internal, address);
213 
214         public static Node External(ulong value) => new(NodeKind.External, value);
215     }
216 
217     private sealed class NodeListComparer : IEqualityComparer<List<Node>>
218     {
219         public static readonly NodeListComparer Instance = new();
220 
221         public bool Equals(List<Node>? x, List<Node>? y) =>
222             ReferenceEquals(x, y) || (x is not null && y is not null && x.SequenceEqual(y));
223 
224         public int GetHashCode(List<Node> obj)
225         {
226             var hash = new HashCode();
227             foreach (var node in obj)
228             {
229                 hash.Add(node);
230             }
231             return hash.ToHashCode();
232         }
233     }
234 
235     private sealed class Encoder
236     {
237         private readonly BinaryLinoOptions  options;
238         private readonly List<List<Node>>  doublets = new();
239         private readonly List<List<Node>>  tuples = new();
240         private readonly Dictionary<List<Node>, Node>  created = new(NodeListComparer.Instance);
241         private readonly List<Node>  powers = new() { Node.Internal(LinksPacket.One) };
242 
243         public Encoder(BinaryLinoOptions options) =>  options = options;
244 
245         private Node Link(List<Node> items)
246         {
247             if ( created.TryGetValue(items, out var existing))
248             {
249                 return existing;
250             }
251             Node node;
252             if (items.Count == 2 && items.All(item => item.Kind != NodeKind.Tuple))
253             {
254                  doublets.Add(items);
255                 node = new Node(NodeKind.Doublet, (ulong) doublets.Count - 1);
256             }
257             else
258             {
259                  tuples.Add(items);
260                 node = new Node(NodeKind.Tuple, (ulong) tuples.Count - 1);
261             }
262              created[items] = node;
263             return node;
264         }
265 
266         private Node Pair(Node first, Node second) => Link(new List<Node> { first, second });
267 
268         private Node Chain(IReadOnlyList<Node> items)
```

```
        {
            var tail = Node.Internal(LinksPacket.NullAddress);
            for (var index = items.Count - 1; index >= 0; index--)
            {
                tail = Pair(items[index], tail);
            }
            return tail;
        }                                       -

        // One link of `items` when the arity allows it, except that two items
        // are always a doublet.
        private bool FitsOneLink(int items) => items != 2 &&
    ↪   options.Arity.Contains((ulong)items);

        private Node Typed(ulong marker, List<Node> elements)
        {
            if (FitsOneLink(elements.Count + 1))
            {
                elements.Insert(0, Node.Internal(marker));
                return Link(elements);
            }
            return Pair(Node.Internal(marker), Chain(elements));
        }

        public Node List(List<Node> elements) => elements.Count switch
        {
            0 => Node.Internal(LinksPacket.NullAddress),
            2 => Pair(elements[0], elements[1]),
              when FitsOneLink(elements.Count) => Link(elements),
              => Typed(LinksPacket.List, elements),
        };

        private Node Power(int exponent)
        {
            while ( powers.Count <= exponent)
            {
                var previous =  powers[^1];
                 powers.Add(Pair(previous, previous));
            }
            return  powers[exponent];
        }

        private Node Unary(ulong value)
        {
            var powers = new List<Node>();
            for (var bit = 63; bit >= 0; bit--)
            {
                if ((value & (1UL << bit)) != 0)
                {
                    powers.Add(Power(bit));
                }
            }
            if (powers.Count == 0)
            {
                return Node.Internal(LinksPacket.NullAddress);
            }
            var sum = powers[^1];
            for (var index = powers.Count - 2; index >= 0; index--)
            {
                sum = Pair(powers[index], sum);
            }
            return sum;
        }

        private bool IsExternal(ulong value) =>  options.ExternalReferences && value <=
    ↪  LinksPacket.ExternalCapacity(8);

        private Node Scalar(ulong value) => IsExternal(value) ? Node.External(value) :
    ↪  Unary(value);

        private Node Reference(string text)
        {
            var remaining = text.AsSpan();
            while (!remaining.IsEmpty)
            {
                if (Rune.DecodeFromUtf16(remaining, out var rune, out var consumed) !=
    ↪  System.Buffers.OperationStatus.Done)
                {
```

```
                    throw BinaryNotationException.Unencodable("reference contains invalid UTF-16");
                }
                remaining = remaining[consumed..];
            }
            if (TryParseCanonicalNumber(text, out var value))
            {
                return IsExternal(value) ? Node.External(value) : Pair(Node.Internal(LinksPacket.Number), Unary(value));
            }
            var codePoints = text.EnumerateRunes().Select(rune => Scalar((ulong)rune.Value)).ToList();
            return Typed(LinksPacket.String, codePoints);
        }

        public Node Encode(LinoLink link)
        {
            if (link.Values is not { } values)
            {
                return Reference(link.Id ?? string.Empty);
            }
            if (link.Id is { } id)
            {
                var elements = new List<Node> { Reference(id) };
                elements.AddRange(values.Select(Encode));
                return Typed(LinksPacket.Identified, elements);
            }
            return List(values.Select(Encode).ToList());
        }

        public LinksPacket Finish()
        {
            var doubletCount = (ulong) doublets.Count;
            PacketReference Resolve(Node node) => node.Kind switch
            {
                NodeKind.Internal => PacketReference.Internal(node.Value),
                NodeKind.External => PacketReference.External(node.Value),
                NodeKind.Doublet => PacketReference.Internal(LinksPacket.FirstLinkAddress + node.Value),
                  => PacketReference.Internal(LinksPacket.FirstLinkAddress + doubletCount + node.Value),
            };
            var links =  doublets
                .Concat( tuples)
                .Select((items, index) => (LinksPacket.FirstLinkAddress + (ulong)index, items.Select(Resolve).ToArray()))
                .ToList();
            return LinksPacket.Pack( options.ExternalReferences, links,  options.PackedWidths);
        }
    }

    private enum ViewKind
    {
        Null,
        Marker,
        External,
        Link,
    }

    private readonly record struct View(ViewKind Kind, ulong Value = 0, PacketReference[]? Items = null);

    private sealed class Decoder
    {
        private readonly DecodeLimits  limits;
        private long  stringBytesLeft;
        //  links[i] is the link at address FirstLinkAddress + i.
        private readonly List<PacketReference[]>  links = new();
        //  unary[i] is the number link i denotes, if it is a unary number.
        private readonly List<ulong?>  unary = new();

        public Decoder(LinksPacket packet, DecodeLimits limits)
        {
             limits = limits;
             stringBytesLeft = limits.MaxStringBytes;
            if ((ulong)packet.Sections.Count > limits.MaxLinks || packet.LinkCount > limits.MaxLinks)
            {
```

```
                throw BinaryNotationException.Limit($"packet declares more than {limits.MaxLinks} links");
            }
            var referencesLeft = limits.MaxReferences;
            try
            {
                packet.Validate();
            }
            catch (BinaryNotationException exception)
            {
                throw BinaryNotationException.Malformed(exception.Detail);
            }
            foreach (var (address, link) in packet.Links())
            {
                if ((ulong)link.Length > referencesLeft)
                {
                    throw BinaryNotationException.Limit($"references exceed the limit of {limits.MaxReferences}");
                }
                referencesLeft -= (ulong)link.Length;
                var expected = LinksPacket.FirstLinkAddress + (ulong) links.Count;
                if (address != expected)
                {
                    throw BinaryNotationException.Malformed(
                        $"a LiNo packet stores its links contiguously from address {LinksPacket.FirstLinkAddress}, " +
                        $"found link {address} where {expected} belongs");
                }
                foreach (var reference in link)
                {
                    if (!reference.IsExternal && reference.Value >= address)
                    {
                        throw BinaryNotationException.Malformed(
                            $"link {address} refers to {reference}, which is not an earlier link");
                    }
                }
                // Links only refer backwards, so one forward pass evaluates every
                // unary number without recursion.
                ulong? value = null;
                if (link.Length == 2
                    && UnaryValue(link[0]) is { } source
                    && UnaryValue(link[1]) is { } target
                    && source <= ulong.MaxValue - target)
                {
                    value = source + target;
                }
                 unary.Add(value);
                 links.Add(link);
            }
        }

        public int LinkCount =>  links.Count;

        private ulong? UnaryValue(PacketReference reference) => reference switch
        {
            { IsExternal: true } => null,
            { Value: LinksPacket.NullAddress } => 0,
            { Value: LinksPacket.One } => 1,
            { Value: >= LinksPacket.FirstLinkAddress } =>  unary[(int)(reference.Value - LinksPacket.FirstLinkAddress)],
              => null,
        };

        public View View(PacketReference reference) => reference switch
        {
            { IsExternal: true } => new View(ViewKind.External, reference.Value),
            { Value: LinksPacket.NullAddress } => new View(ViewKind.Null),
            { Value: < LinksPacket.FirstLinkAddress } => new View(ViewKind.Marker, reference.Value),
              => new View(ViewKind.Link, Items:  links[(int)(reference.Value - LinksPacket.FirstLinkAddress)]),
        };

        private ulong Number(PacketReference reference) =>
            reference.IsExternal
                ? reference.Value
```

```
                : UnaryValue(reference) ?? throw BinaryNotationException.Malformed("expected a unary number");

        /// <summary>The elements of the cons list <c>(e1 (e2 (... (en 0))))</c>.</summary>
        public List<PacketReference> Chain(PacketReference tail)
        {
            var elements = new List<PacketReference>();
            while (true)
            {
                var view = View(tail);
                switch (view)
                {
                    case { Kind: ViewKind.Null }:
                        return elements;
                    case { Kind: ViewKind.Link, Items: [var head, var next] }:
                        if (elements.Count >=  limits.MaxNodes)
                        {
                            throw BinaryNotationException.Limit("chain too long");
                        }
                        elements.Add(head);
                        tail = next;
                        break;
                    default:
                        throw BinaryNotationException.Malformed("broken element chain");
                }
            }
        }

        private LinoLink Typed(ulong marker, IReadOnlyList<PacketReference> elements, int depth, ref long budget)
        {
            switch (marker)
            {
                case LinksPacket.Number:
                    if (elements.Count != 1)
                    {
                        throw BinaryNotationException.Malformed("a number needs exactly one value");
                    }
                    return ReferenceText(Number(elements[0]).ToString(CultureInfo.InvariantCulture));
                case LinksPacket.String:
                    var text = new StringBuilder((int)Math.Min(elements.Count, stringBytesLeft));
                    long byteCount = 0;
                    foreach (var element in elements)
                    {
                        var codePoint = Number(element);
                        if (codePoint > int.MaxValue || !Rune.IsValid((int)codePoint))
                        {
                            throw BinaryNotationException.Malformed($"invalid code point {codePoint}");
                        }
                        var rune = new Rune((int)codePoint);
                        if (rune.Utf8SequenceLength >  stringBytesLeft - byteCount)
                        {
                            throw BinaryNotationException.Limit("too many expanded string bytes");
                        }
                        byteCount += rune.Utf8SequenceLength;
                        text.Append(rune.ToString());
                    }
                    return ReferenceText(text.ToString());
                case LinksPacket.List:
                    return List(elements, 0, depth, ref budget);
                case LinksPacket.Identified:
                    if (elements.Count == 0)
                    {
                        throw BinaryNotationException.Malformed("an identified link needs an id");
                    }
                    var id = Decode(elements[0], depth, ref budget);
                    if (id.Values is not null)
                    {
                        throw BinaryNotationException.Malformed("a link id must be a reference");
                    }
                    var values = List(elements, 1, depth, ref budget).Values!;
                    return LinoFormat.Link(id.Id ?? string.Empty, values);
```

```
                default:
                    throw BinaryNotationException.Malformed($"marker {marker} cannot start a typed value");
            }
        }

        private LinoLink List(IReadOnlyList<PacketReference> elements, int skip, int depth, ref long budget)
        {
            var values = new List<LinoLink>(Math.Max(0, elements.Count - skip));
            for (var index = skip; index < elements.Count; index++)
            {
                values.Add(Decode(elements[index], depth + 1, ref budget));
            }
            return LinoFormat.Link(null, values);
        }

        private LinoLink ReferenceText(string text)
        {
            var bytes = Encoding.UTF8.GetByteCount(text);
            if (bytes >  stringBytesLeft)
            {
                throw BinaryNotationException.Limit("too many expanded string bytes");
            }
             stringBytesLeft -= bytes;
            return LinoFormat.Reference(text);
        }

        public LinoLink Decode(PacketReference reference, int depth, ref long budget)
        {
            if (depth >=  limits.MaxDepth)
            {
                throw BinaryNotationException.Limit($"nesting deeper than { limits.MaxDepth}");
            }
            if (budget <= 0)
            {
                throw BinaryNotationException.Limit("too many LiNo nodes");
            }
            budget--;
            var view = View(reference);
            switch (view)
            {
                case { Kind: ViewKind.Null }:
                    return LinoFormat.Link(null, new List<LinoLink>());
                case { Kind: ViewKind.External }:
                    return ReferenceText(view.Value.ToString(CultureInfo.InvariantCulture));
                case { Kind: ViewKind.Marker }:
                    throw BinaryNotationException.Malformed($"marker {view.Value} used as a value");
                case { Items: [{ IsExternal: false, Value: LinksPacket.Number }, var value] }:
                    return Typed(LinksPacket.Number, new[] { value }, depth, ref budget);
                case { Items: [var marker, var chain] } when IsMarker(marker):
                    return Typed(marker.Value, Chain(chain), depth, ref budget);
                default:
                    var items = view.Items!;
                    return StartsWithMarker(items)
                        ? Typed(items[0].Value, items[1..], depth, ref budget)
                        : List(items, 0, depth, ref budget);
            }
        }
    }
}
```

1.7 ./Link.Foundation.Links.Notation/Binary/PacketReader.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
namespace Link.Foundation.Links.Notation.Binary;

/// <summary>
/// A buffered reader for consecutive binary packets. Reuse it across reads
/// so bytes buffered from the following packet remain available.
/// </summary>
public sealed class PacketReader
{
    private readonly Stream  stream;
    private readonly byte[]  buffer;
    private int  position;
```

```
    private int  length;
        -
    public PacketReader(Stream stream, int bufferSize = 8192)
    {
        ArgumentNullException.ThrowIfNull(stream);
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(bufferSize);
         stream = stream;
         buffer = new byte[bufferSize];
    }

    /// <summary>Wraps a byte array, e.g. a whole encoded message.</summary>
    public static PacketReader FromBytes(byte[] bytes) => new(new MemoryStream(bytes, writable:
↪   false));

    /// <summary>The next byte without consuming it, or -1 at the end of the stream.</summary>
    public int PeekByte() => Fill() ?  buffer[ position] : -1;

    /// <summary>The next byte, or -1 at the end of the stream.</summary>
    public int ReadByte() => Fill() ?  buffer[ position++] : -1;

    /// <summary>True when every byte of the stream has been consumed.</summary>
    public bool AtEnd => !Fill();

    /// <summary>Fills <paramref name="destination"/>; a short stream is a malformed
↪   message.</summary>
    public void ReadExactly(Span<byte> destination)
    {
        while (!destination.IsEmpty)
        {
            if (!Fill())
            {
                throw BinaryNotationException.Malformed("unexpected end of packet");
            }
            var count = Math.Min(destination.Length,  length -  position);
             buffer.AsSpan( position, count).CopyTo(destination);
             position += count;
            destination = destination[count..];
        }
    }

    private bool Fill()
    {
        if ( position <  length)
        {
            return true;
        }
        try
        {
             length =  stream.Read( buffer, 0,  buffer.Length);
        }
        catch (IOException exception)
        {
            throw new BinaryNotationException(BinaryErrorKind.Io, exception.Message, exception);
        }
        catch (ObjectDisposedException exception)
        {
            throw new BinaryNotationException(BinaryErrorKind.Io, exception.Message, exception);
        }
         position = 0;
        return  length > 0;
    }
}
```

1.8 ./Link.Foundation.Links.Notation/Binary/SectionPlanner.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
namespace Link.Foundation.Links.Notation.Binary;

/// <summary>Splits links into <see cref="Section"/>s with a linear dynamic program.</summary>
/// <remarks>
/// After link k, <c>costs[s]</c> is the fewest estimated bytes for links
/// 0..=k with link k in a section of state s. A link either continues the
/// section of the previous link (same state, no hole between them and, for a
/// fixed arity, the same length) or opens a new section after the cheapest
/// previous state, paying for a section header. The tie-breaking matches the
/// Rust <c>SectionPlanner</c>, so both ports produce identical bytes.
/// </remarks>
```

```
internal sealed class SectionPlanner
{
    /// <summary>Estimated bytes of a section header (shape and count), used to weigh a split.</summary>
    private const ulong SectionHeaderEstimate = 2;
    /// <summary>Estimated extra bytes of a variable-arity section: its extra arity field and each length prefix.</summary>
    private const ulong VariableArityEstimate = 1;
    private const ulong Unreachable = ulong.MaxValue / 4;
    /// <summary>Planner states: a width code (0..4) times fixed (0) or variable (1) arity.</summary>
    private static readonly int States = LinksPacket.Widths.Count * 2;

    private readonly IReadOnlyList<(ulong Address, PacketReference[] References)>  links;
    private readonly byte[]  needs;

    public SectionPlanner(bool externalReferences, IReadOnlyList<(ulong Address, PacketReference[] References)> links)
    {
         links = links;
         needs = new byte[links.Count];
        var previousAddress = 0UL;
        for (var index = 0; index < links.Count; index++)
        {
            var (address, references) = links[index];
            if (address <= previousAddress)
            {
                throw BinaryNotationException.Unencodable(
                    $"link addresses must ascend from 1, got {address} after {previousAddress}");
            }
            if (address == ulong.MaxValue)
            {
                throw BinaryNotationException.Unencodable("addresses overflow 64 bits");
            }
            if (references.Length == 0)
            {
                throw BinaryNotationException.Unencodable($"link {address} has no references");
            }
            previousAddress = address;
            byte need = 1;
            foreach (var reference in references)
            {
                need = Math.Max(need, LinksPacket.ReferenceWidth(reference, externalReferences));
            }
             needs[index] = need;
        }
    }

    private static int FirstCheapest(ulong[] costs)
    {
        var best = 0;
        for (var state = 1; state < costs.Length; state++)
        {
            if (costs[state] < costs[best])
            {
                best = state;
            }
        }
        return best;
    }

    /// <summary>
    /// The sections as <c>(link count, width)</c> pairs; with
    /// <paramref name="packedWidths"/> every width may be used, otherwise only
    /// the widest one needed.
    /// </summary>
    public List<(int Count, byte Width)> Plan(bool packedWidths)
    {
        var sections = new List<(int Count, byte Width)>();
        if ( links.Count == 0)
        {
            return sections;
        }
        var widest =  needs.Max();
        var opensSection = new byte[ links.Count];
        var previousBest = new byte[ links.Count];
        var costs = Enumerable.Repeat(Unreachable, States).ToArray();
        for (var index = 0; index <  links.Count; index++)
```

```
        {
            var (address, link) =  links[index];
            var best = FirstCheapest(costs);
            previousBest[index] = (byte)best;
            var cheapestBefore = index == 0 ? 0 : costs[best];
            var continues = index > 0 &&  links[index - 1].Address + 1 == address;
            var sameLength = index > 0 &&  links[index - 1].References.Length == link.Length;
            var next = Enumerable.Repeat(Unreachable, States).ToArray();
            for (var state = 0; state < States; state++)
            {
                var width = LinksPacket.Widths[state / 2];
                var variable = state % 2 == 1;
                if (!(packedWidths || width == widest) || width <  needs[index])
                {
                    continue;
                }
                var variableEstimate = variable ? VariableArityEstimate : 0;
                var body = (ulong)link.Length * width + variableEstimate;
                var openingCost = cheapestBefore + SectionHeaderEstimate + variableEstimate;
                var continuingCost = continues && (variable || sameLength) ? costs[state] :
    ↪  Unreachable;
                if (continuingCost <= openingCost)
                {
                    next[state] = continuingCost + body;
                }
                else
                {
                    next[state] = openingCost + body;
                    opensSection[index] |= (byte)(1 << state);
                }
            }
            costs = next;
        }
        var current = FirstCheapest(costs);
        var end =  links.Count;
        for (var index =  links.Count - 1; index >= 0; index--)
        {
            if ((opensSection[index] & (1 << current)) != 0)
            {
                sections.Add((end - index, LinksPacket.Widths[current / 2]));
                end = index;
                current = previousBest[index];
            }
        }
        sections.Reverse();
        return sections;
    }
}
```

1.9 ./Link.Foundation.Links.Notation/Comments.cs

```
namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Comments, and the rule that decides where one starts.
    /// </summary>
    /// <remarks>
    /// <c>#</c> starts a comment when it opens a token: at the start of the document or
    /// after a space, a tab or a line break. A <c>#</c> written inside a token
    /// (<c>issue#1047</c>) or inside a delimited reference (<c>"# not a comment"</c>) is
    /// an ordinary character.
    /// </remarks>
    public static class Comments
    {
        /// <summary>
        /// The character that starts a comment.
        /// </summary>
        public const char Comment = '#';

        /// <summary>
        /// What a delimited reference may start after.
        /// </summary>
        private const string BeforeReference = " \t\n\r(:";

        /// <summary>
        /// What a comment may start after.
        /// </summary>
        private const string BeforeComment = " \t\n\r";

        /// <summary>
```

```
        /// Blanks every comment of a document.
        /// </summary>
        /// <param name="document">The document to read.</param>
        /// <returns>
        /// The same document with the characters of every comment replaced by spaces.
        /// </returns>
        /// <remarks>
        /// Every character of a comment is replaced rather than removed, so every later
        /// character keeps the position it had in the document the caller wrote, and
        /// anything the parser reports about a position still points at what the reader
        /// can see.
        /// </remarks>
        public static string StripComments(string document)
        {
            char[]? blanked = null;
            int position = 0;

            while (position < document.Length)
            {
                char character = document[position];

                if (Quotes.IsQuote(character) && Follows(document, position, BeforeReference))
                {
                    int end = Quotes.ReferenceEnd(document, position);
                    position = end < 0 ? position + 1 : end;
                    continue;
                }

                if (character == Comment && Follows(document, position, BeforeComment))
                {
                    blanked ??= document.ToCharArray();
                    while (position < document.Length &&
                           document[position] != '\n' &&
                           document[position] != '\r')
                    {
                        blanked[position] = ' ';
                        position++;
                    }

                    continue;
                }

                position++;
            }

            return blanked == null ? document : new string(blanked);
        }

        /// <summary>
        /// Reports whether the character before the position is one of the allowed ones,
        /// the start of the document counting as allowed.
        /// </summary>
        private static bool Follows(string document, int position, string allowed) =>
            position == 0 || allowed.IndexOf(document[position - 1]) >= 0;
    }
}
```

1.10 ./Link.Foundation.Links.Notation/FormatConfig.cs

```
namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// FormatConfig is an alias for FormatOptions to maintain API consistency
    /// with the Python implementation.
    ///
    /// C# uses 'FormatOptions' internally, but provides 'FormatConfig'
    /// as an alias to match Python's naming convention.
    /// </summary>
    public class FormatConfig : FormatOptions
    {
        // FormatConfig inherits all functionality from FormatOptions
        // No additional implementation needed - this is purely for naming consistency
    }
}
```

1.11 ./Link.Foundation.Links.Notation/FormatOptions.cs

```
using System;

namespace Link.Foundation.Links.Notation
{
```

```
    /// <summary>
    /// FormatOptions for Lino notation formatting.
    /// Provides configuration options for controlling how Link objects are formatted.
    /// </summary>
    public class FormatOptions
    {
        /// <summary>
        /// If true, omit parentheses where safe (default: false)
        /// </summary>
        public bool LessParentheses { get; set; } = false;

        /// <summary>
        /// Maximum line length before auto-indenting (default: 80)
        /// </summary>
        public int MaxLineLength { get; set; } = 80;

        /// <summary>
        /// If true, indent lines exceeding MaxLineLength (default: false)
        /// </summary>
        public bool IndentLongLines { get; set; } = false;

        /// <summary>
        /// Maximum number of references before auto-indenting (default: null = unlimited)
        /// </summary>
        public int? MaxInlineRefs { get; set; } = null;

        /// <summary>
        /// If true, group consecutive links with same ID (default: false)
        /// </summary>
        public bool GroupConsecutive { get; set; } = false;

        /// <summary>
        /// String to use for indentation (default: "  " = two spaces)
        /// </summary>
        public string IndentString { get; set; } = "  ";

        /// <summary>
        /// If true, prefer inline format when under thresholds (default: true)
        /// </summary>
        public bool PreferInline { get; set; } = true;

        /// <summary>
        /// Check if line should be indented based on length
        /// </summary>
        /// <param name="line">The line to check</param>
        /// <returns>True if line should be indented based on length threshold</returns>
        public bool ShouldIndentByLength(string line)
        {
            if (!IndentLongLines)
            {
                return false;
            }
            // Count printable unicode characters
            return line.Length > MaxLineLength;
        }

        /// <summary>
        /// Check if link should be indented based on reference count
        /// </summary>
        /// <param name="refCount">Number of references in the link</param>
        /// <returns>True if link should be indented based on reference count threshold</returns>
        public bool ShouldIndentByRefCount(int refCount)
        {
            if (MaxInlineRefs == null)
            {
                return false;
            }
            return refCount > MaxInlineRefs.Value;
        }
    }
}
```

1.12 ./Link.Foundation.Links.Notation/FurthestFailureTracer.cs

```
using Pegasus.Common;
using Pegasus.Common.Tracing;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
```

```
    /// Remembers the furthest position a parse reached.
    /// </summary>
    /// <remarks>
    /// A parsing expression grammar backtracks, so the cursor the generated parser raises
    /// its error with is the start of the document rather than the place the document
    /// stopped making sense. The furthest position any rule reached is that place, and the
    /// tracer is the only hook the generated parser offers to observe it.
    /// </remarks>
    internal sealed class FurthestFailureTracer : ITracer
    {
        /// <summary>Offset of the furthest position the parse reached.</summary>
        public int Furthest { get; private set; }

        public void TraceRuleEnter(string ruleName, Cursor cursor) => Reach(cursor.Location);

        public void TraceRuleExit<T>(string ruleName, Cursor cursor, IParseResult<T> result)
        {
            if (result != null) Reach(result.EndCursor.Location);
        }

        public void TraceCacheHit<T>(string ruleName, Cursor cursor, CacheKey key,
    ↪   IParseResult<T> result)
        {
            if (result != null) Reach(result.EndCursor.Location);
        }

        public void TraceCacheMiss(string ruleName, Cursor cursor, CacheKey key)
        {
        }

        public void TraceInfo(string ruleName, Cursor cursor, string info)
        {
        }

        private void Reach(int location)
        {
            if (location > Furthest) Furthest = location;
        }
    }
}
```

1.13 ./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs

```
using System.Collections.Generic;
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Provides extension methods for collections of <see cref="LinksGroup{TLinkAddress}"/>
    ↪   instances.
    /// </summary>
    public static class ILinksGroupListExtensions
    {
        /// <summary>
        /// Converts a collection of links groups to a flat list of links by processing each
    ↪   group hierarchically.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This
    ↪   can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
        /// <param name="groups">The collection of links groups to convert.</param>
        /// <returns>A flat list of links representing all the links from the input
    ↪   groups.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static List<Link<TLinkAddress>> ToLinksList<TLinkAddress>(this
    ↪   IList<LinksGroup<TLinkAddress>> groups)
        {
            var list = new List<Link<TLinkAddress>>();
            for (var i = 0; i < groups.Count; i++)
            {
                CollectLinksWithIndentedIdSyntaxSupport(list, groups[i], null);
            }
            return list;
        }

        private static void
    ↪   CollectLinksWithIndentedIdSyntaxSupport<TLinkAddress>(List<Link<TLinkAddress>> list,
    ↪   LinksGroup<TLinkAddress> group, Link<TLinkAddress>? parentDependency)
        {
```

```
            var link = group.Link;
            var groups = group.Groups;

            if (groups != null && groups.Count > 0)
            {
                bool isIndentedIdSyntax = link.Id != null &&
                                          (link.Values == null || link.Values.Count == 0);

                if (isIndentedIdSyntax && !parentDependency.HasValue)
                {
                    var childValues = new List<Link<TLinkAddress>>();
                    for (int i = 0; i < groups.Count; i++)
                    {
                        var transformedLink = TransformIndentedIdLink(groups[i]);
                        childValues.Add(transformedLink);
                    }

                    var linkWithChildren = new Link<TLinkAddress>(link.Id, childValues);
                    list.Add(linkWithChildren);
                }
                else
                {
                    var transformedGroups = new List<LinksGroup<TLinkAddress>>();
                    for (int i = 0; i < groups.Count; i++)
                    {
                        var childGroup = groups[i];
                        var childLink = childGroup.Link;
                        var childGroups = childGroup.Groups;

                        if (childLink.Id != null &&
                            (childLink.Values == null || childLink.Values.Count == 0) &&
                            childGroups != null && childGroups.Count > 0)
                        {
                            var transformedLink = TransformIndentedIdLink(childGroup);
                            transformedGroups.Add(new LinksGroup<TLinkAddress>(transformedLink));
                        }
                        else
                        {
                            transformedGroups.Add(childGroup);
                        }
                    }

                    var currentDependency = parentDependency.HasValue ?
↪ parentDependency.Value.Combine(link) : link;
                    list.Add(currentDependency);

                    for (int i = 0; i < transformedGroups.Count; i++)
                    {
                        CollectLinksWithIndentedIdSyntaxSupport(list, transformedGroups[i],
↪ currentDependency);
                    }
                }
            }
            else
            {
                var currentLink = parentDependency.HasValue ?
↪ parentDependency.Value.Combine(link) : link;
                list.Add(currentLink);
            }
        }

        private static Link<TLinkAddress>
↪ TransformIndentedIdLink<TLinkAddress>(LinksGroup<TLinkAddress> group)
        {
            var link = group.Link;
            var groups = group.Groups;

            if (groups != null && groups.Count > 0 &&
                link.Id != null &&
                (link.Values == null || link.Values.Count == 0))
            {
                var childValues = new List<Link<TLinkAddress>>();
                for (int i = 0; i < groups.Count; i++)
                {
                    var transformedChild = TransformIndentedIdLink(groups[i]);
                    childValues.Add(transformedChild);
                }
                return new Link<TLinkAddress>(link.Id, childValues);
            }
```

```
            else if (groups != null && groups.Count > 0)
            {
                var values = new List<Link<TLinkAddress>>();
                if (link.Values != null)
                {
                    values.AddRange(link.Values);
                }
                for (int i = 0; i < groups.Count; i++)
                {
                    values.Add(TransformIndentedIdLink(groups[i]));
                }
                return new Link<TLinkAddress>(link.Id, values);
            }
            else if (!group.IsParenthesized && link.Id == null && link.Values != null &&
↪   link.Values.Count == 1)
            {
                return link.Values[0];
            }
            else
            {
                return link;
            }
        }
    }
}
```

1.14 ./Link.Foundation.Links.Notation/IListExtensions.cs

```
using Platform.Collections;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Provides extension methods for formatting collections of <see
↪   cref="Link{TLinkAddress}"/> instances.
    /// </summary>
    public static class IListExtensions
    {
        /// <summary>
        /// Formats a collection of links as a multi-line string with each link on a separate
↪   line.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link
↪   addresses/identifiers.</typeparam>
        /// <param name="links">The collection of links to format.</param>
        /// <returns>A multi-line string representation of the links.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links) =>
↪   string.Join(Environment.NewLine, links.Select(l => l.ToString()));

        /// <summary>
        /// Formats a collection of links as a multi-line string with optional parentheses
↪   trimming for cleaner output.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link
↪   addresses/identifiers.</typeparam>
        /// <param name="links">The collection of links to format.</param>
        /// <param name="lessParentheses">True to remove outer parentheses from each link for
↪   cleaner formatting; false to use standard formatting.</param>
        /// <returns>A multi-line string representation of the links.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links, bool
↪   lessParentheses)
        {
            if (lessParentheses == false)
            {
                return links.Format();
            }
            else
            {
                return string.Join(Environment.NewLine, links.Select(l =>
↪   l.ToString().TrimSingle('(').TrimSingle(')')));
            }
        }

```

```
        /// <summary>
        /// Formats a collection of links using FormatOptions configuration.
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link
    ↪ addresses/identifiers.</typeparam>
        /// <param name="links">The collection of links to format.</param>
        /// <param name="options">The FormatOptions to use for formatting.</param>
        /// <returns>A multi-line string representation of the links.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string Format<TLinkAddress>(this IList<Link<TLinkAddress>> links,
    ↪ FormatOptions options)
        {
            if (links == null || links.Count == 0)
            {
                return string.Empty;
            }

            // Apply consecutive link grouping if enabled
            var linksToFormat = options.GroupConsecutive ? GroupConsecutiveLinks(links) : links;

            // Format each link with options
            var formattedLinks = linksToFormat.Select(l => l.FormatWithOptions(options));
            return string.Join(Environment.NewLine, formattedLinks);
        }

        /// <summary>
        /// Group consecutive links with the same ID.
        /// </summary>
        private static List<Link<TLinkAddress>>
    ↪ GroupConsecutiveLinks<TLinkAddress>(IList<Link<TLinkAddress>> links)
        {
            if (links == null || links.Count == 0)
            {
                return new List<Link<TLinkAddress>>();
            }

            var grouped = new List<Link<TLinkAddress>>();
            int i = 0;

            while (i < links.Count)
            {
                var current = links[i];

                // Look ahead for consecutive links with same ID
                if (current.Id != null && current.Values != null && current.Values.Count > 0)
                {
                    // Collect all values with same ID
                    var sameIdValues = new List<Link<TLinkAddress>>(current.Values);
                    int j = i + 1;

                    while (j < links.Count)
                    {
                        var nextLink = links[j];
                        if (EqualityComparer<TLinkAddress>.Default.Equals(nextLink.Id,
    ↪ current.Id) &&
                            nextLink.Values != null && nextLink.Values.Count > 0)
                        {
                            sameIdValues.AddRange(nextLink.Values);
                            j++;
                        }
                        else
                        {
                            break;
                        }
                    }

                    // If we found consecutive links, create grouped link
                    if (j > i + 1)
                    {
                        var groupedLink = new Link<TLinkAddress>(current.Id, sameIdValues);
                        grouped.Add(groupedLink);
                        i = j;
                        continue;
                    }
                }

                grouped.Add(current);
                i++;
            }
```

```

            return grouped;
        }
    }
}
```

1.15 ./Link.Foundation.Links.Notation/Link.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Text;
using Platform.Collections;
using Platform.Collections.Lists;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Represents a link in the Links Notation with an optional identifier and nested values.
    /// Links can represent simple references, complex nested structures, or relationships between references to links.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public struct Link<TLinkAddress> : IEquatable<Link<TLinkAddress>>
    {
        private static readonly EqualityComparer<TLinkAddress> EqualityComparerInstance = EqualityComparer<TLinkAddress>.Default;

        /// <summary>
        /// Gets the identifier or address of this link. Can be null for anonymous links.
        /// </summary>
        public readonly TLinkAddress? Id;

        /// <summary>
        /// Gets the collection of nested link values. Can be null or empty for simple reference links.
        /// </summary>
        public readonly IList<Link<TLinkAddress>>? Values;

        /// <summary>
        /// Initializes a new link with the specified identifier and values.
        /// </summary>
        /// <param name="id">The link identifier or address.</param>
        /// <param name="values">The nested link values.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(TLinkAddress? id, IList<Link<TLinkAddress>>? values) => (Id, Values) = (id, values);

        /// <summary>
        /// Initializes a new anonymous link with the specified values.
        /// </summary>
        /// <param name="values">The nested link values.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(IList<Link<TLinkAddress>> values) : this(default!, values) { }

        /// <summary>
        /// Initializes a new anonymous link with the specified array of values.
        /// </summary>
        /// <param name="values">The nested link values as a parameter array.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(params Link<TLinkAddress>[] values) : this(default!, values) { }

        /// <summary>
        /// Initializes a new simple reference link with the specified identifier.
        /// </summary>
        /// <param name="id">The link identifier or address.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link(TLinkAddress id) : this(id, default!) { }

        /// <summary>
        /// Returns the string representation of this link in Links Notation format.
        /// </summary>
        /// <returns>A string representation of the link with proper escaping and formatting.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override string ToString() => Id == null ?
         $"({GetValuesString()})" :
            (Values == null || Values.Count == 0) ?
                $"({EscapeReference(Id.ToString())})" :
```

```
                $"({EscapeReference(Id.ToString())}: {GetValuesString()})";

        /// <summary>
        /// Gets the string representation of the link's values.
        /// </summary>
        /// <returns>A space-separated string of the values, or empty string if no values
    ↪ exist.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public string GetValuesString()
        {
            if (Values == null || Values.Count == 0)
            {
                return "";
            }
            var sb = new StringBuilder();
            for (int i = 0; i < Values.Count; i++)
            {
                if (i > 0)
                {
                    sb.Append(' ');
                }
                sb.Append(GetValueString(Values[i]));
            }
            return sb.ToString();
        }

        /// <summary>
        /// Simplifies the link structure by unwrapping single-value containers and recursively
    ↪ simplifying nested values.
        /// </summary>
        /// <returns>A simplified version of this link.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link<TLinkAddress> Simplify()
        {
            if (Values == null || Values.Count == 0)
            {
                return this;
            }
            else if (Values.Count == 1)
            {
                return Values[0];
            }
            else
            {
                var newValues = new Link<TLinkAddress>[Values.Count];
                for (int i = 0; i < Values.Count; i++)
                {
                    newValues[i] = Values[i].Simplify();
                }
                return new Link<TLinkAddress>(Id, newValues);
            }
        }

        /// <summary>
        /// Combines this link with another link to create a new link containing both as values.
        /// </summary>
        /// <param name="other">The other link to combine with.</param>
        /// <returns>A new link containing both links as values.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public Link<TLinkAddress> Combine(Link<TLinkAddress> other) => new
    ↪ Link<TLinkAddress>(this, other);

        /// <summary>
        /// Gets the string representation of a link value, choosing the appropriate format.
        /// </summary>
        /// <param name="value">The link value to convert to string.</param>
        /// <returns>The string representation of the link value.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static string GetValueString(Link<TLinkAddress> value) =>
    ↪ value.ToLinkOrIdString();

        /// <summary>
        /// Escapes a reference string for safe use in Links Notation format by adding quotes if
    ↪ necessary.
        /// </summary>
        /// <param name="reference">The reference string to escape.</param>
        /// <returns>The escaped reference string with appropriate quoting.</returns>
        public static string EscapeReference(string? reference)
```

```
        {
            if (reference == null)
            {
                return "";
            }
            return ReferenceLiteral.Format(reference);
        }

        /// <summary>
        /// Converts the link to its string representation, choosing between simple reference format or full link format.
        /// </summary>
        /// <returns>A string representation optimized for the link's content.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public string ToLinkOrIdString() => (Values == null || Values.Count == 0) ? (Id == null ? "" : EscapeReference(Id?.ToString())) : ToString();

        /// <summary>
        /// Implicitly converts a value to a simple reference link.
        /// </summary>
        /// <param name="value">The value to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>(TLinkAddress value) => new Link<TLinkAddress>(value);

        /// <summary>
        /// Implicitly converts a tuple of identifier and values to a link.
        /// </summary>
        /// <param name="value">The tuple containing identifier and values.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((TLinkAddress, IList<Link<TLinkAddress>>) value) => new(value.Item1, value.Item2);

        /// <summary>
        /// Implicitly converts a source-target pair to an anonymous link.
        /// </summary>
        /// <param name="value">The tuple containing source and target links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source, Link<TLinkAddress> target) value) => new(value.source, value.target);

        /// <summary>
        /// Implicitly converts a tuple of identifier, source, and target to a link.
        /// </summary>
        /// <param name="value">The tuple containing id, source and target.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id, Link<TLinkAddress> source, Link<TLinkAddress> target) value) => new(value.id.Id, new[] { value.source, value.target });

        /// <summary>
        /// Implicitly converts a source-linker-target triplet to an anonymous link.
        /// </summary>
        /// <param name="value">The tuple containing source, linker, and target links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((Link<TLinkAddress> source, Link<TLinkAddress> linker, Link<TLinkAddress> target) value) => new(value.source, value.linker, value.target);

        /// <summary>
        /// Implicitly converts a tuple of identifier, source, linker, and target to a link.
        /// </summary>
        /// <param name="value">The tuple containing id, source, linker, and target.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>((id<TLinkAddress> id, Link<TLinkAddress> source, Link<TLinkAddress> linker, Link<TLinkAddress> target) value) => new(value.id.Id, new[] { value.source, value.linker, value.target });

        /// <summary>
        /// Determines whether the specified object is equal to this link.
        /// </summary>
        /// <param name="obj">The object to compare with this link.</param>
        /// <returns>True if the specified object is equal to this link; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override bool Equals(object? obj) => obj is Link<TLinkAddress> link && Equals(link);
```

```

        /// <summary>
        /// Returns the hash code for this link.
        /// </summary>
        /// <returns>A hash code for this link.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override int GetHashCode() => (Id, (Values ??
    ↪  Array.Empty<Link<TLinkAddress>>()).GenerateHashCode()).GetHashCode();

        /// <summary>
        /// Indicates whether the current link is equal to another link.
        /// Two anonymous links (both with null Ids) are considered equal if their values are
    ↪  equal.
        /// </summary>
        /// <param name="other">The link to compare with this link.</param>
        /// <returns>True if the current link is equal to the other parameter; otherwise,
    ↪  false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public bool Equals(Link<TLinkAddress> other)
        {
            // Both have null IDs - compare only values
            if (Id == null && other.Id == null)
            {
                return (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ??
    ↪  Array.Empty<Link<TLinkAddress>>());
            }

            // Only one has null ID - not equal
            if (Id == null || other.Id == null)
            {
                return false;
            }

            // Both have IDs - compare IDs and values
            return EqualityComparerInstance.Equals(Id, other.Id) &&
                   (Values ?? Array.Empty<Link<TLinkAddress>>()).EqualTo(other.Values ??
    ↪  Array.Empty<Link<TLinkAddress>>());
        }

        /// <summary>
        /// Determines whether two link instances are equal.
        /// </summary>
        /// <param name="left">The first link to compare.</param>
        /// <param name="right">The second link to compare.</param>
        /// <returns>True if the links are equal; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator ==(Link<TLinkAddress> left, Link<TLinkAddress> right) =>
    ↪  left.Equals(right);

        /// <summary>
        /// Determines whether two link instances are not equal.
        /// </summary>
        /// <param name="left">The first link to compare.</param>
        /// <param name="right">The second link to compare.</param>
        /// <returns>True if the links are not equal; otherwise, false.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(Link<TLinkAddress> left, Link<TLinkAddress> right) =>
    ↪  !(left == right);
    }
}
```

1.16 ./Link.Foundation.Links.Notation/LinkFormatExtensions.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Provides extension methods for formatting <see cref="Link{TLinkAddress}"/> instances
    ↪  with FormatOptions.
    /// </summary>
    public static class LinkFormatExtensions
    {
        /// <summary>
        /// Format a link using FormatOptions configuration.
```

```
        /// </summary>
        /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers.</typeparam>
        /// <param name="link">The link to format.</param>
        /// <param name="options">The FormatOptions to use.</param>
        /// <returns>Formatted string representation.</returns>
        public static string FormatWithOptions<TLinkAddress>(this Link<TLinkAddress> link, FormatOptions options)
        {
            // Empty link
            if (link.Id == null && (link.Values == null || link.Values.Count == 0))
            {
                return options.LessParentheses ? "" : "()";
            }

            // Link with only ID, no values
            if (link.Values == null || link.Values.Count == 0)
            {
                var escapedId = Link<TLinkAddress>.EscapeReference(link.Id?.ToString());
                return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
                    escapedId :
                    $"({escapedId})";
            }

            // Check if we should use indented format
            bool shouldIndent = false;
            if (options.ShouldIndentByRefCount(link.Values.Count))
            {
                shouldIndent = true;
            }
            else
            {
                // Try inline format first
                var valuesStr = string.Join(" ", link.Values.Select(v => GetValueString(v)));
                string testLine;
                if (link.Id != null)
                {
                    var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
                    testLine = options.LessParentheses ? $"{idStr}: {valuesStr}" : $"({idStr}: {valuesStr})";
                }
                else
                {
                    testLine = options.LessParentheses ? valuesStr : $"({valuesStr})";
                }

                if (options.ShouldIndentByLength(testLine))
                {
                    shouldIndent = true;
                }
            }

            // Format with indentation if needed
            if (shouldIndent && options.PreferInline == false)
            {
                return FormatIndented(link, options);
            }

            // Standard inline formatting
            var values = string.Join(" ", link.Values.Select(v => GetValueString(v)));

            // Link with values only (null id)
            if (link.Id == null)
            {
                if (options.LessParentheses)
                {
                    var allSimple = link.Values.All(v => v.Values == null || v.Values.Count == 0);
                    if (allSimple)
                    {
                        return string.Join(" ", link.Values.Select(v => Link<TLinkAddress>.EscapeReference(v.Id?.ToString())));
                    }
                    return values;
                }
                return $"({values})";
            }

```

```
            // Link with ID and values
            var id = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
            var withColon = $"{id}: {values}";
            return options.LessParentheses && !NeedsParentheses(link.Id?.ToString()) ?
                withColon :
                $"({withColon})";
        }

        /// <summary>
        /// Format a link with indentation.
        /// </summary>
        private static string FormatIndented<TLinkAddress>(Link<TLinkAddress> link,
↪   FormatOptions options)
        {
            // FormatIndented is only reached with a non-empty Values collection,
            // but the nullable annotation on Link<T>.Values does not know that.
            var values = link.Values ?? Array.Empty<Link<TLinkAddress>>();

            if (link.Id == null)
            {
                // Values only - format each on separate line
                var lines = values.Select(v => options.IndentString + GetValueString(v));
                return string.Join(Environment.NewLine, lines);
            }

            // Link with ID - format as id:\n  value1\n  value2
            var idStr = Link<TLinkAddress>.EscapeReference(link.Id.ToString());
            var sb = new StringBuilder();
            sb.Append($"{idStr}:");

            foreach (var v in values)
            {
                sb.Append(Environment.NewLine);
                sb.Append(options.IndentString);
                sb.Append(GetValueString(v));
            }

            return sb.ToString();
        }

        /// <summary>
        /// Get the string representation of a value.
        /// </summary>
        private static string GetValueString<TLinkAddress>(Link<TLinkAddress> value)
        {
            return value.ToLinkOrIdString();
        }

        /// <summary>
        /// Check if a string needs to be wrapped in parentheses.
        /// </summary>
        private static bool NeedsParentheses(string? s)
        {
            return s != null && (s.Contains(" ") || s.Contains(":") || s.Contains("(") ||
↪   s.Contains(")"));
        }
    }
}
```

1.17 ./Link.Foundation.Links.Notation/LinksGroup.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using Platform.Collections;
using Platform.Collections.Lists;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Represents a group of links with hierarchical structure, where each group contains a
↪   primary link and optional nested groups.
    /// This structure supports the indentation-based syntax of the Links Notation.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
↪   any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public struct LinksGroup<TLinkAddress> : IEquatable<LinksGroup<TLinkAddress>>
    {
        /// <summary>
        /// Gets or sets the primary link of this group.
```

```
        /// </summary>
        public Link<TLinkAddress> Link
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            set;
        }

        /// <summary>
        /// Gets or sets the collection of nested groups within this group. Can be null if no nested groups exist.
        /// </summary>
        public IList<LinksGroup<TLinkAddress>>? Groups
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            set;
        }

        /// <summary>
        /// Gets or sets a value indicating whether this group was written as a parenthesized group.
        /// A parenthesized group is a link in its own right, so it keeps its identifier and values
        /// instead of being unwrapped when it is used as a child of the indented identifier syntax.
        /// This is a parse-time hint and therefore takes no part in equality comparisons.
        /// </summary>
        internal bool IsParenthesized
        {
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            get;
            [MethodImpl(MethodImplOptions.AggressiveInlining)]
            set;
        }

        /// <summary>
        /// Initializes a new links group with the specified link and nested groups.
        /// </summary>
        /// <param name="link">The primary link of this group.</param>
        /// <param name="groups">The nested groups within this group.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksGroup(Link<TLinkAddress> link, IList<LinksGroup<TLinkAddress>>? groups)
        {
            Link = link;
            Groups = groups;
        }

        /// <summary>
        /// Initializes a new links group with the specified link and no nested groups.
        /// </summary>
        /// <param name="link">The primary link of this group.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public LinksGroup(Link<TLinkAddress> link) : this(link, null) { }

        /// <summary>
        /// Implicitly converts a links group to a list of links by flattening the hierarchical structure.
        /// </summary>
        /// <param name="value">The links group to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator List<Link<TLinkAddress>>(LinksGroup<TLinkAddress> value) => value.ToLinksList();

        /// <summary>
        /// Converts this links group to a flat list of links, preserving the hierarchical relationships.
        /// </summary>
        /// <returns>A list of links representing the flattened structure of this group.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public List<Link<TLinkAddress>> ToLinksList()
        {
            var list = new List<Link<TLinkAddress>>();
            AppendToLinksList(list);
            return list;
        }
```

```
89
90          /// <summary>
91          /// Appends this links group to the specified list as flattened links.
92          /// </summary>
93          /// <param name="list">The list to append the links to.</param>
94          [MethodImpl(MethodImplOptions.AggressiveInlining)]
95          public void AppendToLinksList(List<Link<TLinkAddress>> list) => AppendToLinksList(list,
    ↪  Link, this);
96
97          /// <summary>
98          /// Recursively appends a links group to a list, combining dependencies with nested
    ↪  groups to create a flattened structure.
99          /// </summary>
100         /// <param name="list">The list to append the links to.</param>
101         /// <param name="dependency">The dependency link to combine with nested groups.</param>
102         /// <param name="group">The links group to process.</param>
103         [MethodImpl(MethodImplOptions.AggressiveInlining)]
104         public static void AppendToLinksList(List<Link<TLinkAddress>> list, Link<TLinkAddress>
    ↪  dependency, LinksGroup<TLinkAddress> group)
105         {
106             list.Add(dependency);
107             var groups = group.Groups;
108             if (groups != null && !groups.IsNullOrEmpty())
109             {
110                 for (int i = 0; i < groups.Count; i++)
111                 {
112                     var innerGroup = groups[i];
113                     AppendToLinksList(list, dependency.Combine(innerGroup.Link), innerGroup);
114                 }
115             }
116         }
117
118         /// <summary>
119         /// Determines whether the specified object is equal to this links group.
120         /// </summary>
121         /// <param name="obj">The object to compare with this links group.</param>
122         /// <returns>True if the specified object is equal to this links group; otherwise,
    ↪  false.</returns>
123         [MethodImpl(MethodImplOptions.AggressiveInlining)]
124         public override bool Equals(object? obj) => obj is LinksGroup<TLinkAddress> linksGroup ?
    ↪  Equals(linksGroup) : false;
125
126         /// <summary>
127         /// Returns the hash code for this links group.
128         /// </summary>
129         /// <returns>A hash code for this links group.</returns>
130         [MethodImpl(MethodImplOptions.AggressiveInlining)]
131         public override int GetHashCode() => (Link, (Groups ??
    ↪  Array.Empty<LinksGroup<TLinkAddress>>()).GenerateHashCode()).GetHashCode();
132
133         /// <summary>
134         /// Indicates whether the current links group is equal to another links group.
135         /// </summary>
136         /// <param name="other">The links group to compare with this links group.</param>
137         /// <returns>True if the current links group is equal to the other parameter; otherwise,
    ↪  false.</returns>
138         [MethodImpl(MethodImplOptions.AggressiveInlining)]
139         public bool Equals(LinksGroup<TLinkAddress> other) => Link.Equals(other.Link) && (Groups
    ↪  ?? Array.Empty<LinksGroup<TLinkAddress>>()).EqualTo(other.Groups ??
    ↪  Array.Empty<LinksGroup<TLinkAddress>>());
140
141         /// <summary>
142         /// Determines whether two links group instances are equal.
143         /// </summary>
144         /// <param name="left">The first links group to compare.</param>
145         /// <param name="right">The second links group to compare.</param>
146         /// <returns>True if the links groups are equal; otherwise, false.</returns>
147         [MethodImpl(MethodImplOptions.AggressiveInlining)]
148         public static bool operator ==(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
    ↪  right) => left.Equals(right);
149
150         /// <summary>
151         /// Determines whether two links group instances are not equal.
152         /// </summary>
153         /// <param name="left">The first links group to compare.</param>
154         /// <param name="right">The second links group to compare.</param>
155         /// <returns>True if the links groups are not equal; otherwise, false.</returns>
```

```
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static bool operator !=(LinksGroup<TLinkAddress> left, LinksGroup<TLinkAddress>
    ↪  right) => !(left == right);

        /// <summary>
        /// Returns a string representation of this links group.
        /// </summary>
        /// <returns>A string representation of this links group.</returns>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public override string ToString() => ToLinksList().Format();
    }
}
```

1.18 ./Link.Foundation.Links.Notation/ParseException.cs

```
using System;
using System.Text;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// The error raised when a document does not parse, carrying the position the parser
    /// stopped at.
    /// </summary>
    /// <remarks>
    /// The position is the furthest one the parser reached, which is the character the
    /// document stops making sense at rather than the point the last alternative gave up
    /// on. It derives from <see cref="FormatException"/>, the exception the generated
    /// parser raised on its own, so code that catches that keeps working.
    /// A document nested deeper than <see cref="Parser.MaxDepth"/> is refused with this
    /// exception too; then <see cref="MaxDepth"/> says how deep the nesting may go, and the
    /// position is the group or the line that is one level too deep.
    /// </remarks>
    public class ParseException : FormatException
    {
        /// <summary>The number of characters a quoted line is cut down to.</summary>
        private const int QuotedLineWidth = 80;

        /// <summary>What a message writes in place of the part of a long line it left
    ↪  out.</summary>
        private const string Ellipsis = "...";

        /// <summary>
        /// Creates an exception describing where <paramref name="subject"/> stopped parsing.
        /// </summary>
        /// <param name="subject">The document that failed to parse.</param>
        /// <param name="offset">Offset of the position the parser stopped at.</param>
        /// <param name="innerException">The error the generated parser raised, if any.</param>
        public ParseException(string subject, int offset, Exception? innerException = null)
            : this(Locate(subject ?? string.Empty, offset), null, innerException)
        {
        }

        /// <summary>
        /// Creates an exception saying that <paramref name="subject"/> nests deeper than
        /// <paramref name="maxDepth"/> at <paramref name="offset"/>.
        /// </summary>
        /// <param name="subject">The document that nests too deeply.</param>
        /// <param name="offset">Offset of the group or the line that is one level too
    ↪  deep.</param>
        /// <param name="maxDepth">The deepest nesting allowed.</param>
        /// <param name="innerException">The error the parser raised, if any.</param>
        public ParseException(string subject, int offset, int maxDepth, Exception?
    ↪  innerException = null)
            : this(Locate(subject ?? string.Empty, offset), maxDepth, innerException)
        {
        }

        private ParseException(Position position, int? maxDepth, Exception? innerException)
            : base(Describe(position, maxDepth), innerException)
        {
            Offset = position.Offset;
            Line = position.Line;
            Column = position.Column;
            Found = position.Found;
            LineText = position.LineText;
            MaxDepth = maxDepth;
        }

        /// <summary>Offset of the offending position from the start of the document.</summary>
```

```
        public int Offset { get; }

        /// <summary>Line the offending position is on, counted from 1.</summary>
        public int Line { get; }

        /// <summary>Column the offending position is at, counted from 1.</summary>
        public int Column { get; }

        /// <summary>The character found instead, or null at the end of the document.</summary>
        public char? Found { get; }

        /// <summary>The offending line, as written, without its line ending.</summary>
        public string LineText { get; }

        /// <summary>
        /// The deepest nesting allowed, when the document is nested deeper than that;
        /// <see langword="null"/> for any other error.
        /// </summary>
        public int? MaxDepth { get; }
            -
        /// <summary>
        /// The one-line summary: where the parser stopped and what stands there, or that the
        /// nesting is too deep there.
        /// </summary>
        public string Summary => Summarize(new Position(Offset, Line, Column, Found, LineText),
    ↪ MaxDepth);

        /// <summary>
        /// The offending line with a caret under the offending column, quoted the way a
        /// compiler quotes source. A long line is shown as a window around the caret, so the
        /// message stays the same size whether the document has ten lines or fifteen hundred.
        /// </summary>
        public string Snippet => Quote(new Position(Offset, Line, Column, Found, LineText));

        /// <summary>Where the parser stopped, and what the document holds there.</summary>
        private readonly record struct Position(
            int Offset,
            int Line,
            int Column,
            char? Found,
            string LineText);

        /// <summary>The message the exception carries: the summary and the quoted
    ↪ line.</summary>
        private static string Describe(Position position, int? maxDepth) =>
            $"{(maxDepth.HasValue ? "Nesting too deep" : "Syntax error")} at
    ↪ {Summarize(position, maxDepth)}\n{Quote(position)}";

        private static string Summarize(Position position, int? maxDepth)
        {
            if (maxDepth.HasValue)
            {
                return $"line {position.Line}, column {position.Column}: nesting depth exceeds
    ↪ the maximum of {maxDepth.Value}";
            }
            var found = position.Found.HasValue
                ? $"\"{Escape(position.Found.Value)}\""
                : "end of input";
            return $"line {position.Line}, column {position.Column}: unexpected {found}";
        }

        private static string Quote(Position position)
        {
            var (quoted, column) = QuoteLine(position.LineText, position.Column);
            var number = position.Line.ToString();
            var gutter = new string(' ', number.Length);
            return $"{number} | {quoted}\n{gutter} | {new string(' ', column - 1)}^";
        }

        private static string Escape(char character) => character switch
        {
            '\n' => "\\n",
            '\r' => "\\r",
            '\t' => "\\t",
            '"' => "\\\"",
            '\\' => "\\\\",
              => character.ToString(),
        };

```

```
        /// <summary>
        /// Turns the position the parser stopped at into a line, a column and the line
        /// itself, so the message can point at the defect instead of quoting the rest of the
        /// document.
        /// </summary>
        private static Position Locate(string document, int offset)
        {
            offset = Math.Max(0, Math.Min(offset, document.Length));

            var line = 1;
            var lineStart = 0;
            for (var index = 0; index < offset; index++)
            {
                if (document[index] == '\n')
                {
                    line++;
                    lineStart = index + 1;
                }
            }

            var lineEnd = document.IndexOf('\n', lineStart);
            if (lineEnd < 0) lineEnd = document.Length;

            var lineText = document.Substring(lineStart, lineEnd - lineStart).TrimEnd('\r');
            var found = offset < document.Length ? document[offset] : (char?)null;
            return new Position(offset, line, offset - lineStart + 1, found, lineText);
        }

        /// <summary>
        /// Cuts <paramref name="line"/> down to a window around <paramref name="column"/>, and
        /// says which column the offending character sits at in that window. Both columns
        /// count from 1.
        /// </summary>
        private static (string Quoted, int Column) QuoteLine(string line, int column)
        {
            if (line.Length <= QuotedLineWidth) return (line, column);

            var target = column - 1;
            var lastStart = line.Length - QuotedLineWidth;
            var start = Math.Min(Math.Max(target - (QuotedLineWidth / 2), 0), lastStart);
            var end = start + QuotedLineWidth;

            var quoted = new StringBuilder();
            if (start > 0) quoted.Append(Ellipsis);
            quoted.Append(line, start, QuotedLineWidth);
            if (end < line.Length) quoted.Append(Ellipsis);

            var shift = start > 0 ? Ellipsis.Length : 0;
            return (quoted.ToString(), target - start + shift + 1);
        }
    }
}
```

1.19 ./Link.Foundation.Links.Notation/Parser.cs

```
using System;
using System.Collections.Generic;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Parses Links Notation documents into links.
    /// </summary>
    /// <remarks>
    /// The rules live in <c>Parser.peg</c> and are compiled into
    /// <see cref="GeneratedParser"/>; this class runs them and, when a document does not
    /// parse, reports the position the parser stopped at instead of the bare
    /// <see cref="FormatException"/> the generated parser raises.
    /// </remarks>
    public class Parser
    {
        /// <summary>
        /// How deep links may nest when nothing says otherwise.
        /// </summary>
        /// <remarks>
        /// The parser recurses once per level of nesting, and running out of stack ends the
        /// process where no caller can catch it. The limit is set well below what fits in a
        /// 1 MiB thread stack, and it is the same in every Links Notation implementation, so
        /// a document one of them reads is not too deep for another.
        /// </remarks>
```

```
        public const int DefaultMaxDepth = 64;

        /// <summary>
        /// Creates a parser.
        /// </summary>
        /// <param name="comments">
        /// When <see langword="false"/>, <c>#</c> is read as an ordinary character
        /// instead of the start of a comment.
        /// </param>
        public Parser(bool comments = true) : this(comments, DefaultMaxDepth)
        {
        }

        /// <summary>
        /// Creates a parser that refuses links nested deeper than <paramref name="maxDepth"/>.
        /// </summary>
        /// <param name="comments">
        /// When <see langword="false"/>, <c>#</c> is read as an ordinary character
        /// instead of the start of a comment.
        /// </param>
        /// <param name="maxDepth">How deep links may nest; see <see cref="MaxDepth"/>.</param>
        /// <exception cref="ArgumentOutOfRangeException"><paramref name="maxDepth"/> is
    ↪ negative.</exception>
        public Parser(bool comments, int maxDepth)
        {
            Comments = comments;
            MaxDepth = maxDepth;
        }

        /// <summary>
        /// Whether <c>#</c> starts a comment that runs to the end of its line.
        /// </summary>
        public bool Comments { get; }

        /// <summary>
        /// How deep links may nest. Every parenthesized group and every indentation level
        /// is one level, and the lines of a document are at level 0, so with a limit of 1
        /// <c>(a)</c> parses while <c>((a))</c>, <c>(a (b))</c> and a group on an indented
        /// line do not. A document nested deeper is refused with a
        /// <see cref="ParseException"/> whose <see cref="ParseException.MaxDepth"/> is set,
        /// rather than recursed into until the stack runs out.
        /// </summary>
        /// <exception cref="ArgumentOutOfRangeException">The value is negative.</exception>
        public int MaxDepth
        {
            get =>  maxDepth;
            init =>  maxDepth = value >= 0
                ? value
                : throw new ArgumentOutOfRangeException(nameof(MaxDepth), value, "Maximum
    ↪ nesting depth cannot be negative.");
        }

        private readonly int  maxDepth;

        /// <summary>
        /// Parses a Links Notation document.
        /// </summary>
        /// <param name="subject">The document to parse.</param>
        /// <param name="fileName">The optional file name to use in error messages.</param>
        /// <returns>The links the document holds.</returns>
        /// <exception cref="ParseException">
        /// Thrown when the document does not parse, or nests deeper than
        /// <see cref="MaxDepth"/>. The exception carries the line, the column and the
        /// offending line of the position the parser stopped at.
        /// </exception>
        public IList<Link<string>> Parse(string subject, string? fileName = null)
        {
            // Comments are blanked rather than removed, so a position reported for the
            // prepared document is the same position in the document the caller wrote.
            var prepared = Comments ? Notation.Comments.StripComments(subject) : subject;
            var tracer = new FurthestFailureTracer();
            var parser = new GeneratedParser { Tracer = tracer, MaxDepth = MaxDepth };
            try
            {
                return parser.Parse(prepared, fileName);
            }
            catch (GeneratedParser.NestingTooDeepException error)
            {
                throw new ParseException(subject, error.Offset, error.MaxDepth, error);
```

```
            }
            catch (FormatException error) when (error is not ParseException)
            {
                throw new ParseException(subject, tracer.Furthest, error);
            }
        }
    }
}
```

1.20 ./Link.Foundation.Links.Notation/Quotes.cs

```
using System.Text;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// Reads a delimited reference, the shape shared by the grammar and by
    /// <see cref="Comments"/>.
    /// </summary>
    /// <remarks>
    /// Any number N of delimiters opens and closes a reference, and 2N delimiters stand
    /// for N written as content. The grammar reads references through this class, so
    /// blanking comments reaches a reference exactly as far as the parser does.
    /// </remarks>
    internal static class Quotes
    {
        /// <summary>
        /// Reports whether the character is one of the delimiters a reference can be
        /// written between.
        /// </summary>
        /// <param name="character">The character to test.</param>
        /// <returns><see langword="true"/> when the character opens a reference.</returns>
        internal static bool IsQuote(char character) =>
            character == '"' || character == '\'' || character == '`';

        /// <summary>
        /// A body written between an even run of delimiters is substantive when it
        /// holds at least one visible character and does not straddle a parenthesis.
        /// An even run can always be read as delimiter pairs enclosing nothing, so the
        /// n-quote reading is only taken when it carries something the pairs cannot.
        /// </summary>
        /// <param name="content">The body to weigh.</param>
        /// <returns><see langword="true"/> when the body carries something a pair
    ↪  cannot.</returns>
        internal static bool IsSubstantiveBody(string content)
        {
            int depth = 0;
            bool hasVisible = false;

            foreach (var c in content)
            {
                if (c == '(')
                {
                    depth++;
                }
                else if (c == ')')
                {
                    depth--;
                    if (depth < 0) return false;
                }

                if (!char.IsWhiteSpace(c)) hasVisible = true;
            }

            return hasVisible && depth == 0;
        }

        /// <summary>
        /// Reads the reference starting at the given position.
        /// </summary>
        /// <param name="subject">The whole document being read.</param>
        /// <param name="start">The position of the opening delimiter.</param>
        /// <param name="value">The content of the reference, without its delimiters.</param>
        /// <param name="length">How many characters the reference spans.</param>
        /// <returns><see langword="true"/> when a reference was read at that position.</returns>
        internal static bool TryParse(string? subject, int start, out string? value, out int
    ↪  length)
        {
            value = null;
            length = 0;
```

```
            if (subject == null || start >= subject.Length || !IsQuote(subject[start])) return false;

            char quoteChar = subject[start];

            // Count opening quotes
            int quoteCount = 0;
            int pos = start;
            while (pos < subject.Length && subject[pos] == quoteChar)
            {
                quoteCount++;
                pos++;
            }

            bool isEvenRun = quoteCount % 2 == 0;

            string openClose = new string(quoteChar, quoteCount);
            string escapeSeq = new string(quoteChar, quoteCount * 2);

            var content = new StringBuilder();
            while (pos < subject.Length)
            {
                // Check for escape sequence (2*N quotes)
                if (pos + escapeSeq.Length <= subject.Length &&
                    string.CompareOrdinal(subject, pos, escapeSeq, 0, escapeSeq.Length) == 0)
                {
                    content.Append(openClose); // 2*N quotes become N quotes
                    pos += escapeSeq.Length;
                    continue;
                }

                // Check for closing quotes (exactly N quotes, not more)
                if (pos + quoteCount <= subject.Length &&
                    string.CompareOrdinal(subject, pos, openClose, 0, quoteCount) == 0)
                {
                    int afterClose = pos + quoteCount;
                    if (afterClose >= subject.Length || subject[afterClose] != quoteChar)
                    {
                        // Found valid closing
                        if (isEvenRun && !IsSubstantiveBody(content.ToString()))
                        {
                            return SetEmptyReference(isEvenRun, quoteCount, out value, out length);
                        }

                        value = content.ToString();
                        length = afterClose - start;
                        return true;
                    }
                }

                // Take next character
                content.Append(subject[pos]);
                pos++;
            }

            // No valid closing found
            return SetEmptyReference(isEvenRun, quoteCount, out value, out length);
        }

        /// <summary>
        /// Finds where the reference starting at the given position ends.
        /// </summary>
        /// <param name="subject">The whole document being read.</param>
        /// <param name="start">The position of the opening delimiter.</param>
        /// <returns>
        /// The position right after the closing delimiters, or -1 when no reference
        /// starts there.
        /// </returns>
        internal static int ReferenceEnd(string subject, int start) =>
            TryParse(subject, start, out var ignored, out var length) ? start + length : -1;

        /// <summary>
        /// A run of an even number of delimiters that does not open a reference with a
        /// substantive body is the empty reference: the shortest reading, a bare
        /// delimiter pair enclosing nothing, wins over a longer n-quote delimiter.
        /// </summary>
        private static bool SetEmptyReference(bool isEvenRun, int quoteCount, out string? value, out int length)
```

```
        {
            if (!isEvenRun)
            {
                value = null;
                length = 0;
                return false;
            }

            value = string.Empty;
            length = quoteCount;
            return true;
        }
    }
}
```

1.21 ./Link.Foundation.Links.Notation/ReferenceLiteral.cs

```
using System;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;

namespace Link.Foundation.Links.Notation;

/// <summary>Version 1 lossless Unicode reference literals.</summary>
public static class ReferenceLiteral
{
    private static readonly UTF8Encoding Utf8 = new(false, true);
    private static readonly Regex Literal = new("\\A~([0-9]+)\\{([0-9a-fA-F]*)\\}\\z");

    /// <summary>Encodes exact Unicode as a version 1 UTF-8 hexadecimal literal.</summary>
    public static string Encode(string text) =>
    $"~1{{{Convert.ToHexString(Utf8.GetBytes(text)).ToLowerInvariant()}}}";

    /// <summary>Decodes a complete literal; malformed UTF-8 and versions are refused.</summary>
    public static string Decode(string literal)
    {
        var match = Literal.Match(literal);
        if (!match.Success || match.Groups[1].Value != "1" || match.Groups[2].Length % 2 != 0)
        {
            throw new FormatException("Invalid or unsupported reference literal (expected
    ~1{UTF-8 hex})");
        }
        try { return Utf8.GetString(Convert.FromHexString(match.Groups[2].Value)); }
        catch (DecoderFallbackException error) { throw new FormatException("Reference literal
    must contain valid UTF-8", error); }
    }

    internal static bool HasPrefix(string subject, int start)
    {
        if (start >= subject.Length || subject[start] != '~') return false;
        var position = start + 1;
        while (position < subject.Length && subject[position] is >= '0' and <= '9') position++;
        return position > start + 1 && position < subject.Length && subject[position] == '{';
    }

    internal static bool TryRead(string subject, int start, out string value, out int length)
    {
        value = "";
        length = 0;
        if (!HasPrefix(subject, start)) return false;
        var opening = subject.IndexOf('{', start);
        var closing = subject.IndexOf('}', opening + 1);
        if (closing < 0) throw new FormatException("Unclosed reference literal");
        length = closing + 1 - start;
        value = Decode(subject.Substring(start, length));
        return true;
    }

    internal static string Format(string text)
    {
        Utf8.GetByteCount(text); // Refuse unpaired surrogates before formatting.
        if (text.Length == 0 || text.Any(c => c < 32 || c == 127)) return Encode(text);
        var needsQuotes = HasPrefix(text, 0) || text.StartsWith('#') || text.Any(c =>
    char.IsWhiteSpace(c) || c is '\ufeff' or '(' or ')' or ':' or '\'' or '"' or '`');
        if (!needsQuotes) return text;
        var (quote, count) = new[] { '\'', '"', '`' }
            .Where(quote => text[0] != quote)
            .Select(quote => (Quote: quote, Count: (LongestRun(text, quote) + 1) | 1))
            .MinBy(candidate => candidate.Count);
```

```
        var delimiter = new string(quote, count);
        return $"{delimiter}{text}{delimiter}";
    }

    private static int LongestRun(string text, char quote)
    {
        var (longest, current) = (0, 0);
        foreach (var character in text)
        {
            current = character == quote ? current + 1 : 0;
            longest = Math.Max(longest, current);
        }
        return longest;
    }
}
```

1.22 ./Link.Foundation.Links.Notation/StreamParseException.cs

```
using System;

namespace Link.Foundation.Links.Notation
{
    /// <summary>A parser error located relative to the complete input stream.</summary>
    public sealed class StreamParseException : FormatException
    {
        internal StreamParseException(ParseException error, int offset, int line, int column)
            : base($"Stream parse error at line {line}, column {column}: {error.Message}", error)
        {
            Offset = offset;
            Line = line;
            Column = column;
            ParseError = error;
        }

        /// <summary>The canonical parser error raised for the buffered record.</summary>
        public ParseException ParseError { get; }

        /// <summary>Absolute character offset from the start of the stream.</summary>
        public int Offset { get; }

        /// <summary>One-based line number in the complete stream.</summary>
        public int Line { get; }

        /// <summary>One-based column number in the complete stream.</summary>
        public int Column { get; }
    }
}
```

1.23 ./Link.Foundation.Links.Notation/StreamParser.cs

```
using System;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace Link.Foundation.Links.Notation
{
    /// <summary>Incrementally emits complete top-level Links Notation records.</summary>
    public sealed class StreamParser
    {
        private const int DefaultMaxBufferSize = 10 * 1024 * 1024;
        private const string BeforeReference = " \t\n\r(:";
        private const string BeforeComment = " \t\n\r";

        private readonly Parser  parser;
        private readonly StringBuilder  buffer = new();
        private readonly StringBuilder  currentLine = new();
        private readonly List<Link<string>>  links = new();
        private int?  baseIndentation;
        private bool  lineClassified;
        private int  offset;
        private int  line;
        private int  column;
        private int  segmentOffset;
        private int  segmentLine;
        private bool  ended;
        private int  maxBufferSize = DefaultMaxBufferSize;

        /// <summary>Creates a stream backed by a default canonical parser.</summary>
        public StreamParser() : this(new Parser()) { }

```

```
        /// <summary>Creates a stream backed by <paramref name="parser"/>.</summary>
        public StreamParser(Parser parser)
        {
             parser = parser ?? throw new ArgumentNullException(nameof(parser));
            Reset();
        }

        /// <summary>Raised once for every completed link.</summary>
        public event Action<Link<string>>? LinkParsed;

        /// <summary>Whether emitted links are retained for <see cref="Finish()"/> and <see cref="Drain"/>.</summary>
        public bool Collect { get; set; } = true;

        /// <summary>The largest unresolved record accepted by the stream.</summary>
        public int MaxBufferSize
        {
            get =>  maxBufferSize;
            set =>  maxBufferSize = value > 0
                ? value
                : throw new ArgumentOutOfRangeException(nameof(value), "Maximum buffer size must be positive.");
        }

        /// <summary>The point immediately after the last character written.</summary>
        public StreamPosition Position => new(
             offset,
             line,
             column,
             buffer.Length +  currentLine.Length);

        /// <summary>Consumes a chunk and returns links made complete by it.</summary>
        public IReadOnlyList<Link<string>> Write(string chunk)
        {
            ArgumentNullException.ThrowIfNull(chunk);
            if ( ended) throw new InvalidOperationException("Cannot write after Finish().");

            var emitted = new List<Link<string>>();
            foreach (var character in chunk)
            {
                 currentLine.Append(character);
                 offset++;

                if (character == '\n')
                {
                     buffer.Append( currentLine);
                     currentLine.Clear();
                     lineClassified = false;
                     line++;
                     column = 1;
                }
                else
                {
                    if (! lineClassified && character != ' ' && character != '\t' && character != '\r')
                    {
                         lineClassified = true;
                        if (!( parser.Comments && character == Comments.Comment))
                        {
                            StartContentLine(LeadingSpaces( currentLine), emitted);
                        }
                    }
                     column++;
                }

                if ( buffer.Length +  currentLine.Length > MaxBufferSize)
                {
                    throw new ArgumentException(
                        $"Buffered record exceeds maximum size of {MaxBufferSize} characters.",
                        nameof(chunk));
                }
            }
            return emitted;
        }

        /// <summary>Finishes the stream and returns all undrained links.</summary>
        public IReadOnlyList<Link<string>> Finish() => Finish(string.Empty);

        /// <summary>Consumes a final chunk and finishes the stream.</summary>
        public IReadOnlyList<Link<string>> Finish(string chunk)
```

```
            -
            -
        {   -
            if ( ended)
            {
                if (chunk.Length > 0) throw new InvalidOperationException("Cannot write after
    ↪  Finish().");
                return Collect ?  links.ToArray() : Array.Empty<Link<string>>();
            }

            var emitted = chunk.Length == 0
                ? new List<Link<string>>()
                : new List<Link<string>>(Write(chunk));
            var document =  buffer.ToString() +  currentLine;
            if (document.Length > 0)
            {
                IList<Link<string>> parsed;
                try
                {
                    parsed =  parser.Parse(document);
                }
                catch (ParseException error)
                {
                    throw new StreamParseException(
                        error,
                         segmentOffset + error.Offset,
                         segmentLine + error.Line - 1,
                        error.Column);
                }
                Publish(parsed, emitted);
                AdvanceSegment(document);
            }

             buffer.Clear();
             currentLine.Clear();
             baseIndentation = null;
             ended = true;
            return Collect ?  links.ToArray() : emitted;
        }

        /// <summary>Returns and forgets links retained since the previous call.</summary>
        public IReadOnlyList<Link<string>> Drain()
        {
            var links =  links.ToArray();
             links.Clear();
            return links;
        }

        /// <summary>Reuses the parser while preserving its options and event handlers.</summary>
        public StreamParser Reset()
        {
             buffer.Clear();
             currentLine.Clear();
             baseIndentation = null;
             lineClassified = false;
             links.Clear();
             offset = 0;
             line = 1;
             column = 1;
             segmentOffset = 0;
             segmentLine = 1;
             ended = false;
            return this;
        }

        /// <summary>Lazily parses a sequence of chunks as a native enumerable.</summary>
        public static IEnumerable<Link<string>> ParseChunks(
            IEnumerable<string> chunks,
            Parser? parser = null)
        {
            var stream = new StreamParser(parser ?? new Parser()) { Collect = false };
            foreach (var chunk in chunks)
            {
                foreach (var link in stream.Write(chunk)) yield return link;
            }
            foreach (var link in stream.Finish()) yield return link;
        }

        /// <summary>Lazily parses an asynchronous sequence of chunks.</summary>
        public static async IAsyncEnumerable<Link<string>> ParseChunksAsync(
            IAsyncEnumerable<string> chunks,
            Parser? parser = null,
```

```
            [EnumeratorCancellation] CancellationToken cancellationToken = default)
        {
            var stream = new StreamParser(parser ?? new Parser()) { Collect = false };
            await foreach (var chunk in chunks.WithCancellation(cancellationToken))
            {
                foreach (var link in stream.Write(chunk)) yield return link;
            }
            foreach (var link in stream.Finish()) yield return link;
        }

        private void StartContentLine(int indentation, List<Link<string>> emitted)
        {
            if ( buffer.Length > 0 &&
                 baseIndentation.HasValue &&
                indentation <=  baseIndentation.Value &&
                StructurallyComplete( buffer.ToString(),  parser.Comments))
            {
                try
                {
                    var document =  buffer.ToString();
                    var parsed =  parser.Parse(document);
                    Publish(parsed, emitted);
                    AdvanceSegment(document);
                     buffer.Clear();
                     baseIndentation = null;
                }
                catch (ParseException)
                {
                    // A rejected prefix can become valid when more input arrives.
                }
            }

             baseIndentation ??= indentation;
        }

        private void Publish(IEnumerable<Link<string>> links, List<Link<string>> emitted)
        {
            foreach (var link in links)
            {
                emitted.Add(link);
                if (Collect)  links.Add(link);
                LinkParsed?.Invoke(link);
            }
        }

        private void AdvanceSegment(string document)
        {
             segmentOffset += document.Length;
            foreach (var character in document)
            {
                if (character == '\n')  segmentLine++;
            }
        }

        private static int LeadingSpaces(StringBuilder line)
        {
            var indentation = 0;
            while (indentation < line.Length && line[indentation] == ' ') indentation++;
            return indentation;
        }

        private static bool StructurallyComplete(string document, bool comments)
        {
            var depth = 0;
            for (var position = 0; position < document.Length; position++)
            {
                var character = document[position];
                if (Quotes.IsQuote(character) && Follows(document, position, BeforeReference))
                {
                    var end = Quotes.ReferenceEnd(document, position);
                    if (end < 0) return false;
                    position = end - 1;
                    continue;
                }
                if (comments && character == Comments.Comment && Follows(document, position,
    ↪  BeforeComment))
                {
                    var newline = document.IndexOf('\n', position);
                    if (newline < 0) break;
```

```
                                        -
                    position = newline;
                    continue;
                }
                if (character == '(') depth++;
                else if (character == ')') depth--;
            }
            return depth == 0;          -
        }

        private static bool Follows(string document, int position, string allowed) =>
            position == 0 || allowed.IndexOf(document[position - 1]) >= 0;
    }

    /// <summary>An absolute stream position and the unresolved record size.</summary>
    public readonly record struct StreamPosition(int Offset, int Line, int Column, int Buffered);
}
```

1.24 ./Link.Foundation.Links.Notation/ .cs

```
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// A utility struct that provides convenient implicit conversions for creating links from various tuple combinations.
    /// The underscore name follows functional programming conventions for placeholder/utility types.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public struct  <TLinkAddress>
    {
        /// <summary>
        /// Gets the underlying link represented by this utility struct.
        /// </summary>
        public readonly Link<TLinkAddress> Link;

        /// <summary>
        /// Initializes a new instance with the specified link.
        /// </summary>
        /// <param name="id">The link to wrap.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public  (Link<TLinkAddress> id) => Link = id;

        /// <summary>
        /// Implicitly converts a link to this utility struct.
        /// </summary>
        /// <param name="value">The link to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>(Link<TLinkAddress> value) => new(value);

        /// <summary>
        /// Implicitly converts a link address to this utility struct.
        /// </summary>
        /// <param name="id">The link address to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>(TLinkAddress id) => new(id);

        /// <summary>
        /// Implicitly converts a tuple of address and link to this utility struct.
        /// </summary>
        /// <param name="value">The tuple containing address and link.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>((TLinkAddress, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2 });

        /// <summary>
        /// Implicitly converts a tuple of address and two links to this utility struct.
        /// </summary>
        /// <param name="value">The tuple containing address and two links.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>((TLinkAddress, Link<TLinkAddress>, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[] { value.Item2, value.Item3 });

        /// <summary>
        /// Implicitly converts a tuple of address and three links to this utility struct.
        /// </summary>
        /// <param name="value">The tuple containing address and three links.</param>
```

```
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator  <TLinkAddress>((TLinkAddress, Link<TLinkAddress>,
    ↪  Link<TLinkAddress>, Link<TLinkAddress>) value) => new Link<TLinkAddress>(value.Item1, new[]
    ↪  { value.Item2, value.Item3, value.Item4 });

        /// <summary>                                -
        /// Implicitly converts this utility struct back to a link.
        /// </summary>
        /// <param name="value">The utility struct to convert.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static implicit operator Link<TLinkAddress>( <TLinkAddress> value) => value.Link;
    }
}
```

1.25 ./Link.Foundation.Links.Notation/id.cs

```
using System.Runtime.CompilerServices;

namespace Link.Foundation.Links.Notation
{
    /// <summary>
    /// A readonly struct that explicitly represents a link identifier/address.
    /// This type is used to distinguish between regular values and explicit identifiers in link
    ↪  construction.
    /// </summary>
    /// <typeparam name="TLinkAddress">The type used for link addresses/identifiers. This can be
    ↪  any type that uniquely identifies a link, such as string, int, or Guid.</typeparam>
    public readonly struct id<TLinkAddress>
    {
        /// <summary>
        /// Gets the identifier value.
        /// </summary>
        public readonly TLinkAddress Id;

        /// <summary>
        /// Initializes a new identifier with the specified value.
        /// </summary>
        /// <param name="id">The identifier value.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public id(TLinkAddress id) => Id = id;

        /// <summary>
        /// Explicitly converts a value to an identifier struct.
        /// </summary>
        /// <param name="id">The value to convert to an identifier.</param>
        [MethodImpl(MethodImplOptions.AggressiveInlining)]
        public static explicit operator id<TLinkAddress>(TLinkAddress id) => new(id);
    }
}
```

1.26 ./Link.Foundation.Links.Notation.Tests/ApiTests.cs

```
using System;
using System.Collections.Generic;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class ApiTests
    {
        [Fact]
        public static void IsRefTest()
        {
            // C# doesn't have separate Ref/Link types, but we can test simple link behavior
            var simpleLink = new Link<string>("some value", null);
            Assert.Equal("some value", simpleLink.Id);
            Assert.Null(simpleLink.Values);
        }

        [Fact]
        public static void IsLinkTest()
        {
            // Test link with values
            var values = new List<Link<string>> { new Link<string>("child", null) };
            var link = new Link<string>("id", values);
            Assert.Equal("id", link.Id);
            Assert.Single(link.Values!);
            Assert.Equal("child", link.Values?[0].Id);
        }

        [Fact]
```

```
        public static void IsRefEquivalentTest()
        {
            // C# doesn't have separate Ref/Link types, but we can test simple link behavior
            var simpleLink = new Link<string>("some value", null);
            Assert.Equal("some value", simpleLink.Id);
            Assert.Null(simpleLink.Values);
        }

        [Fact]
        public static void IsLinkEquivalentTest()
        {
            // Test link with values
            var values = new List<Link<string>> { new Link<string>("child", null) };
            var link = new Link<string>("id", values);
            Assert.Equal("id", link.Id);
            Assert.Single(link.Values!);
            Assert.Equal("child", link.Values?[0].Id);
        }

        [Fact]
        public static void EmptyLinkTest()
        {
            var link = new Link<string>(null, new List<Link<string>>());
            var output = link.ToString();
            Assert.Equal("()", output);
        }

        [Fact]
        public static void SimpleLinkTest()
        {
            var input = "(1: 1 1)";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Contains("1:", output);
            Assert.Contains("1", output);
        }

        [Fact]
        public static void LinkWithSourceTargetTest()
        {
            var input = "(index: source target)";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void LinkWithSourceTypeTargetTest()
        {
            var input = "(index: source type target)";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void SingleLineFormatTest()
        {
            var input = "id: value1 value2";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // The parser should handle single-line format
            var output = parsed.Format();
            Assert.Contains("id", output);
            Assert.Contains("value1", output);
            Assert.Contains("value2", output);
        }

        [Fact]
```

```
109         public static void QuotedReferencesTest()
110         {
111             var input = @(""quoted id"": ""value with spaces"")";
112             var parser = new Parser();
113             var parsed = parser.Parse(input);
114 
115             var output = parsed.Format();
116             Assert.Contains("quoted id", output);
117             Assert.Contains("value with spaces", output);
118         }
119 
120         [Fact]
121         public static void QuotedReferencesParsingTest()
122         {
123             // Test that quoted references are parsed correctly
124             var input = @(""quoted id"": ""value with spaces"")";
125             var parser = new Parser();
126             var parsed = parser.Parse(input);
127 
128             // Verify parsing worked correctly
129             var output = parsed.Format();
130             Assert.Contains("quoted id", output);
131             Assert.Contains("value with spaces", output);
132         }
133 
134         [Fact]
135         public static void IndentedIdSyntaxParsingTest()
136         {
137             // Test that indented ID syntax is parsed correctly
138             var indented = "id:\n  value1\n  value2";
139             var inline = "(id: value1 value2)";
140 
141             var parser = new Parser();
142             var indentedParsed = parser.Parse(indented);
143             var inlineParsed = parser.Parse(inline);
144 
145             // Both should produce equivalent structures
146             var indentedOutput = indentedParsed.Format();
147             var inlineOutput = inlineParsed.Format();
148             Assert.Equal(indentedOutput, inlineOutput);
149             Assert.Equal("(id: value1 value2)", indentedOutput);
150         }
151 
152         [Fact]
153         public static void IndentedIdSyntaxRoundtripTest()
154         {
155             var input = "id:\n  value1\n  value2";
156             var parser = new Parser();
157             var parsed = parser.Parse(input);
158 
159             // Validate that we can format with indented syntax using FormatOptions
160             var options = new FormatOptions
161             {
162                 MaxInlineRefs = 1,  // Force indentation with more than 1 ref
163                 PreferInline = false
164             };
165             var output = parsed.Format(options);
166             Assert.Equal(input, output);
167         }
168 
169         [Fact]
170         public static void MultipleIndentedIdSyntaxParsingTest()
171         {
172             // Test that multiple indented ID links are parsed correctly
173             var indented = "id1:\n  a\n  b\nid2:\n  c\n  d";
174             var inline = "(id1: a b)\n(id2: c d)";
175 
176             var parser = new Parser();
177             var indentedParsed = parser.Parse(indented);
178             var inlineParsed = parser.Parse(inline);
179 
180             // Both should produce equivalent structures
181             var indentedOutput = indentedParsed.Format();
182             var inlineOutput = inlineParsed.Format();
183             Assert.Equal(indentedOutput, inlineOutput);
184             Assert.Equal("(id1: a b)\n(id2: c d)", indentedOutput);
185         }
186 
187         [Fact]
```

```
        -

        public static void MultipleIndentedIdSyntaxRoundtripTest()
        {
            var input = "id1:\n  a\n  b\nid2:\n  c\n  d";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate that we can format with indented syntax using FormatOptions
            var options = new FormatOptions
            {
                MaxInlineRefs = 1,  // Force indentation with more than 1 ref
                PreferInline = false
            };
            var output = parsed.Format(options);
            Assert.Equal(input, output);
        }
    }
}
```

1.27 ./Link.Foundation.Links.Notation.Tests/BinaryLinksNotationTests.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using Xunit;
using Link.Foundation.Links.Notation.Binary;

using LinoLink = Link.Foundation.Links.Notation.Link<string>;

namespace Link.Foundation.Links.Notation.Tests.Binary;

/// <summary>
/// The binary links notation: packets, sections, arities, widths and the
/// LiNo mapping on top of them. Mirrors <c>rust/tests/protocol packet tests.rs</c>.
/// </summary>
public sealed class BinaryLinksNotationTests
{
    internal static string Hex(byte[] bytes) => string.Join(" ", bytes.Select(value =>
    ↪ value.ToString("x2")));

    internal static byte[] Unhex(string text) =>
        text.Split(' ', StringSplitOptions.RemoveEmptyEntries).Select(value =>
    ↪ Convert.ToByte(value, 16)).ToArray();

    private static IReadOnlyList<LinoLink> Parse(string text) => LinoFormat.ParseDocument(text);

    private static bool ReferencesOption(string field) => field switch
    {
        "plain" => false,
        "external" => true,
          => throw new ArgumentException($"unknown references option {field}"),
    };

    private static bool WidthsOption(string field) => field switch
    {
        "uniform" => false,
        "packed" => true,
          => throw new ArgumentException($"unknown widths option {field}"),
    };

    /// <summary><c>address:reference reference...</c> separated by <c>;</c>, <c>#v</c> being
    ↪ external.</summary>
    private static List<(ulong Address, PacketReference[] References)> ParseLinks(string text) =>
        text.Split(';').Select(link =>
        {
            var parts = link.Split(':', 2);
            var references = parts[1].Split(' ').Select(reference => reference.StartsWith('#')
                ? PacketReference.External(ulong.Parse(reference[1..]))
                : PacketReference.Internal(ulong.Parse(reference))).ToArray();
            return (ulong.Parse(parts[0]), references);
        }).ToList();

    /// <summary>The golden vectors shared with the Rust test suite: both ports must write and
    ↪ read exactly these bytes.</summary>
    private static IEnumerable<string[]> GoldenVectors() =>
        File.ReadAllLines(Path.Combine(AppContext.BaseDirectory, "Protocol",
    ↪ "binary-links-notation-vectors.txt"))
            .Where(line => line.Length > 0 && !line.StartsWith('#'))
            .Select(line => line.Split('\t'));

```

```
    private static string Describe(IEnumerable<(ulong Address, PacketReference[] References)>
↪ links) =>
        string.Join(";", links.Select(link => $"{link.Address}:{string.Join(" ",
↪ link.References)}"));

    private static string Describe(LinksPacket packet) =>
        $"{packet.ExternalReferences} " + string.Join(" | ", packet.Sections.Select(section =>
            $"gap {section.Gap} arity {section.Arity} width {section.Width}: "
            + string.Join(";", section.Links.Select(link => string.Join(" ", link)))));

    [Fact]
    public void GoldenDocumentsAreStable()
    {
        var checkedVectors = 0;
        foreach (var fields in GoldenVectors().Where(fields => fields[0] == "document"))
        {
            Assert.Equal(6, fields.Length);
            var text = fields[1].Replace("\\n", "\n");
            var options = new BinaryLinoOptions
            {
                ExternalReferences = ReferencesOption(fields[2]),
                Arity = ArityRange.Parse(fields[3]),
                PackedWidths = WidthsOption(fields[4]),
            };
            var document = Parse(text);
            var binary = new BinaryLinoCodec(options);
            Assert.True(fields[5] == Hex(binary.Encode(document)), $"{text} with {options}");
            Assert.Equal(document, binary.Decode(Unhex(fields[5])));
            checkedVectors++;
        }
        Assert.True(checkedVectors >= 80, $"only {checkedVectors} document vectors");
    }

    [Fact]
    public void GoldenPacketsAreStable()
    {
        var checkedVectors = 0;
        foreach (var fields in GoldenVectors().Where(fields => fields[0] == "links"))
        {
            Assert.Equal(5, fields.Length);
            var links = ParseLinks(fields[1]);
            var packet = LinksPacket.Pack(ReferencesOption(fields[2]), links,
↪ WidthsOption(fields[3]));
            Assert.True(fields[4] == Hex(packet.ToBytes()), fields[1]);
            var read = LinksPacket.FromBytes(Unhex(fields[4]));
            Assert.Equal(Describe(packet), Describe(read));
            Assert.Equal(Describe(links), Describe(read.Links()));
            checkedVectors++;
        }
        Assert.True(checkedVectors >= 10, $"only {checkedVectors} links vectors");
    }

    [Fact]
    public void WidthTiersFollowTheNumberOfLinks()
    {
        Assert.Equal(1, LinksPacket.AddressTier(0, false));
        Assert.Equal(1, LinksPacket.AddressTier(255, false));
        Assert.Equal(2, LinksPacket.AddressTier(256, false));
        Assert.Equal(2, LinksPacket.AddressTier(65 535, false));
        Assert.Equal(4, LinksPacket.AddressTier(65 536, false));
        Assert.Equal(4, LinksPacket.AddressTier(uint.MaxValue, false));
        Assert.Equal(8, LinksPacket.AddressTier((ulong)uint.MaxValue + 1, false));
        // External references take the top bit, halving every range.
        Assert.Equal(1, LinksPacket.AddressTier(127, true));
        Assert.Equal(2, LinksPacket.AddressTier(128, true));
        Assert.Equal(2, LinksPacket.AddressTier(32 767, true));
        Assert.Equal(4, LinksPacket.AddressTier(32 768, true));
        Assert.Equal(8, LinksPacket.AddressTier(ulong.MaxValue, false));
        // No width holds an internal address in the external half.
        Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
↪ LinksPacket.AddressTier(1UL << 63, true)).Kind);
        Assert.Equal(ulong.MaxValue, LinksPacket.InternalCapacity(8, false));
        Assert.Equal((ulong)long.MaxValue, LinksPacket.InternalCapacity(8, true));
        Assert.Equal(127UL, LinksPacket.ExternalCapacity(1));
    }

    [Fact]
    public void ExternalReferencesMatchPlatformDataHybrid()
```

```
130     {
131         foreach (var width in LinksPacket.Widths)
132         {
133             foreach (var value in new ulong[] { 0, 1, 2, 100,
    ↪  LinksPacket.ExternalCapacity(width) })
134             {
135                 var raw = LinksPacket.EncodeExternal(value, width);
136                 Assert.True(LinksPacket.TryDecodeExternal(raw, width, out var decoded));
137                 Assert.Equal(value, decoded);
138             }
139             Assert.False(LinksPacket.TryDecodeExternal(LinksPacket.InternalCapacity(width,
    ↪  true), width, out var ignored));
140         }
141         Assert.Equal(0xFFUL, LinksPacket.EncodeExternal(1, 1));
142         Assert.Equal(0x80UL, LinksPacket.EncodeExternal(0, 1));
143 
144     }
145 
146     private static string ManyLinks(int count) =>
147         string.Join("\n", Enumerable.Range(0, count).Select(index => $"(n{index} m{index})"));
148 
149     private static byte[] SectionWidths(LinksPacket packet) => packet.Sections.Select(section =>
    ↪  section.Width).ToArray();
150 
151     [Fact]
152     public void UniformWidthsGrowPast256AddressesAndPackedWidthsStaySmall()
153     {
154         Assert.Equal(new byte[] { 1 },
    ↪  SectionWidths(LinoMapping.EncodeDocument(Parse(ManyLinks(3)))));
155 
156         var document = Parse(ManyLinks(400));
157         var uniform = LinoMapping.EncodeDocument(document);
158         Assert.Equal(new byte[] { 2 }, SectionWidths(uniform));
159         var uniformBytes = uniform.ToBytes();
160         const int header = 1 + 2; // header byte + two-byte LEB128 count
161         Assert.Equal(header + (long)uniform.LinkCount * 2 * 2, uniformBytes.Length);
162 
163         var packed = LinoMapping.EncodeDocument(document, new
    ↪  BinaryLinoOptions().WithPackedWidths());
164         Assert.Equal(1, SectionWidths(packed)[0]);
165         Assert.Contains((byte)2, SectionWidths(packed));
166         var packedBytes = packed.ToBytes();
167         Assert.True(packedBytes.Length < uniformBytes.Length);
168         foreach (var bytes in new[] { uniformBytes, packedBytes })
169         {
170             Assert.Equal(document, LinoMapping.DecodeDocument(LinksPacket.FromBytes(bytes)));
171         }
172     }
173 
174     [Fact]
175     public void PackedWidthsNeverTakeMoreBytesThanUniformOnes()
176     {
177         var corpus = BinaryLinoCodecTests.Corpus.Append(ManyLinks(200)).Append("(1000 1)");
178         foreach (var text in corpus)
179         {
180             var document = Parse(text);
181             foreach (var options in BinaryLinoCodecTests.AllOptions().Where(options =>
    ↪  options.PackedWidths))
182             {
183                 var packed = LinoMapping.EncodeDocument(document, options).ToBytes();
184                 var uniform = LinoMapping.EncodeDocument(document,
    ↪  options.WithPackedWidths(false)).ToBytes();
185                 Assert.True(packed.Length <= uniform.Length, $"{text} with {options}");
186             }
187         }
188     }
189 
190     [Fact]
191     public void LargeExternalValuesWidenOnlyTheirSection()
192     {
193         var options = new BinaryLinoOptions().WithExternalReferences().WithPackedWidths();
194         Assert.Equal(new byte[] { 1 }, SectionWidths(LinoMapping.EncodeDocument(Parse("(100
    ↪  1)"), options)));
195         Assert.Equal(new byte[] { 4, 1 }, SectionWidths(LinoMapping.EncodeDocument(Parse("(70000
    ↪  1)"), options)));
196         // Beyond 63 bits the number falls back to in-band unary links.
197         Assert.True(LinoMapping.EncodeDocument(Parse("18446744073709551615"), options).LinkCount
    ↪  > 60);
```

```
    }

    private static int[] LinkLengths(IReadOnlyList<LinoLink> document, BinaryLinoOptions options)
    {
        var packet = LinoMapping.EncodeDocument(document, options);
        Assert.Equal(document, new BinaryLinoCodec().Decode(packet.ToBytes()));
        return packet.Links().Select(link => link.References.Length).ToArray();
    }

    [Fact]
    public void ArityOptionsChooseDoubletsTripletsOrAnyLength()
    {
        var document = Parse("(a b c)\n(d e)");
        var external = new BinaryLinoOptions().WithExternalReferences();
        Assert.All(LinkLengths(document, external), length => Assert.Equal(2, length));
        // `(a b c)` is one triplet; the strings `(String code point)` stay doublets.
        var triplets = LinkLengths(document, external.WithArity(ArityRange.Between(2, 3)));
        Assert.Single(triplets, length => length == 3);
        Assert.All(triplets, length => Assert.InRange(length, 2, 3));
        Assert.Equal(triplets, LinkLengths(document, external.WithArity(ArityRange.AtLeast(1))));
        // Only any length turns a one-item list, here the root, into one link.
        Assert.Equal(new[] { 2, 2, 2, 2 }, LinkLengths(Parse("a"), external));
        Assert.Equal(new[] { 2, 2, 1 }, LinkLengths(Parse("a"),
↪   external.WithArity(ArityRange.AtLeast(1))));
        // Long lists only become single links when the range allows their length.
        var longList = Parse("(a b c d e f g h)");
        foreach (var (arity, singleLink) in new[]
        {
            (ArityRange.Between(2, 3), false),
            (ArityRange.Between(2, 8), true),
            (ArityRange.AtLeast(2), true),
        })
        {
            Assert.True(LinkLengths(longList, external.WithArity(arity)).Max() == 8 ==
↪   singleLink, arity.ToString());
        }
    }

    [Fact]
    public void AritiesWithoutDoubletsAreUnencodable()
    {
        foreach (var arity in new[] { ArityRange.Exactly(3), ArityRange.Exactly(1),
↪   ArityRange.AtLeast(3) })
        {
            var error = Assert.Throws<BinaryNotationException>(
                () => LinoMapping.EncodeDocument(Parse("(a b)"), new
↪   BinaryLinoOptions().WithArity(arity)));
            Assert.Equal(BinaryErrorKind.Unencodable, error.Kind);
        }
    }

    [Fact]
    public void ArityRangesParseAndDisplay()
    {
        foreach (var (text, range) in new[]
        {
            ("2", ArityRange.Doublets),
            ("2..3", ArityRange.Between(2, 3)),
            ("1..", ArityRange.AtLeast(1)),
            ("3..3", ArityRange.Exactly(3)),
            ("+2", ArityRange.Doublets),
        })
        {
            Assert.Equal(range, ArityRange.Parse(text));
        }
        Assert.Equal("3", ArityRange.Between(3, 3).ToString());
        Assert.Equal("2..3", ArityRange.Between(2, 3).ToString());
        Assert.Equal("1..", ArityRange.AtLeast(1).ToString());
        Assert.Equal(ArityRange.Doublets, new BinaryLinoOptions().Arity);
        Assert.Equal(default, new BinaryLinoOptions().WithArity(ArityRange.Doublets));
        foreach (var invalid in new[] { "", "0", "0..2", "3..2", "a", "1..b", "..3", "-1", " 2",
↪   "2 ", "++2", null })
        {
            Assert.False(ArityRange.TryParse(invalid, out var ignored), invalid);
        }
        Assert.False(ArityRange.TryParse("3..2", out var invalidRange, out var error));
        Assert.Equal("arity range 3..2 is empty", error);
```

```
        Assert.False(ArityRange.TryParse("x", out invalidRange, out error));
        Assert.Equal("invalid arity 'x': expected n, min..max or min..", error);
        Assert.Equal("arity must be at least 1", Assert.Throws<FormatException>(() =>
↪   ArityRange.Parse("0")).Message);
        Assert.True(ArityRange.AtLeast(2).Contains(1 000));
        Assert.False(ArityRange.Between(2, 3).Contains(4));
        Assert.False(ArityRange.Between(2, 3).Contains(1));
        Assert.True(ArityRange.Exactly(2).IsFixed && !ArityRange.AtLeast(2).IsFixed);
    }

    [Fact]
    public void ReplyOptionsFollowTheReceivedPacket()
    {
        var document = Parse("(a b c) 70000");
        var options = new BinaryLinoOptions()
            .WithExternalReferences()
            .WithArity(ArityRange.Between(2, 3))
            .WithPackedWidths();
        Assert.Equal(options, BinaryLinoOptions.OfPacket(LinoMapping.EncodeDocument(document,
↪   options)));
        Assert.Equal(default, BinaryLinoOptions.OfPacket(LinoMapping.EncodeDocument(document)));
        Assert.Equal(default, BinaryLinoOptions.OfPacket(new LinksPacket()));
        Assert.Equal("BinaryLinoOptions { ExternalReferences = True, Arity = 2..3, PackedWidths
↪   = True }", options.ToString());
    }

    private static Section DoubletSection(ulong gap, ArityRange arity, byte width, params (ulong
↪   Source, ulong Target)[] links)
    {
        var section = new Section { Gap = gap, Arity = arity, Width = width };
        section.Links.AddRange(links.Select(link =>
            new[] { PacketReference.Internal(link.Source), PacketReference.Internal(link.Target)
↪   }));
        return section;
    }

    [Fact]
    public void HandBuiltPacketsDecode()
    {
        // One doublet (One One) is 2^1 in unary, so `(Number 7)` is the number 2.
        var links = new (ulong, ulong)[] { (1, 1), (2, 6), (7, 0), (4, 8) };
        var compact = new LinksPacket();
        compact.Sections.Add(DoubletSection(5, ArityRange.Doublets, 1, links));
        var bytes = compact.ToBytes();
        Assert.Equal("10 04 01 01 02 06 07 00 04 08", Hex(bytes));
        Assert.Equal("2", LinoFormat.FormatDocument(new BinaryLinoCodec().Decode(bytes)));

        // The same links as a variable section of width 2 use the explicit layout.
        var explicitLayout = new LinksPacket();
        explicitLayout.Sections.Add(DoubletSection(5, ArityRange.AtLeast(1), 2, links));
        bytes = explicitLayout.ToBytes();
        Assert.Equal("12 01 1d 05 00 04 01 01 00 01 00 01 02 00 06 00 01 07 00 00 00 01 04 00 08
↪   00", Hex(bytes));
        Assert.Equal("2", LinoFormat.FormatDocument(new BinaryLinoCodec().Decode(bytes)));
        Assert.Equal("10 00", Hex(new LinksPacket().ToBytes()));
    }

    private static PacketReference[] Internals(params ulong[] addresses) =>
        addresses.Select(PacketReference.Internal).ToArray();

    [Fact]
    public void PackingRejectsLinksItCannotLayOut()
    {
        void Unencodable(bool externalReferences, params (ulong, PacketReference[])[] links)
        {
            var error = Assert.Throws<BinaryNotationException>(() =>
↪   LinksPacket.Pack(externalReferences, links, true));
            Assert.Equal(BinaryErrorKind.Unencodable, error.Kind);
        }
        Unencodable(false, (0, Internals(1))); // address 0 is null
        Unencodable(false, (2, Internals(1)), (1, Internals(1))); // descending
        Unencodable(false, (2, Internals(1)), (2, Internals(1))); // repeated
        Unencodable(false, (1, Array.Empty<PacketReference>())); // no references
        Unencodable(false, (1, new[] { PacketReference.External(1) }));
        Unencodable(true, (1, new[] { PacketReference.External(1UL << 63) }));
        Unencodable(true, (1, Internals(1UL << 63))); // the top bit marks externals
        Assert.NotNull(LinksPacket.Pack(false, new[] { (1UL, Internals(ulong.MaxValue)) },
↪   true));
```

```
    }

    [Fact]
    public void WritingRejectsSectionsThatDoNotHoldTheirLinks()
    {
        static void Unencodable(Section section)
        {
            var packet = new LinksPacket();
            packet.Sections.Add(section);
            var error = Assert.Throws<BinaryNotationException>(() => packet.ToBytes());
            Assert.Equal(BinaryErrorKind.Unencodable, error.Kind);
        }
        Unencodable(DoubletSection(0, ArityRange.Exactly(3), 1, (1, 1)));
        Unencodable(DoubletSection(0, ArityRange.Exactly(0), 1));
        Unencodable(DoubletSection(0, ArityRange.Between(3, 2), 1));
        Unencodable(DoubletSection(0, ArityRange.Exactly(1UL << 62), 1));
        Unencodable(DoubletSection(0, ArityRange.Doublets, 3, (1, 1)));
        Unencodable(DoubletSection(0, ArityRange.Doublets, 1, (256, 0)));
        Unencodable(DoubletSection(ulong.MaxValue, ArityRange.Doublets, 1, (1, 1)));
    }

    private static BinaryNotationException DecodeError(byte[] bytes, DecodeLimits? limits = null)
    {
        var protocol = new BinaryLinoCodec { Limits = limits ?? DecodeLimits.Default };
        return Assert.Throws<BinaryNotationException>(() => protocol.Decode(bytes));
    }

    /// <summary>Each packet breaks exactly one rule, the one its detail names.</summary>
    [Theory]
    [InlineData("", "empty input")]
    [InlineData("20 00", "unsupported binary header byte 0x20")]
    [InlineData("10 01 06 00", "link 6 refers to Internal(6), which is not an earlier link")]
    [InlineData("10 01 07 00", "link 6 refers to Internal(7), which is not an earlier link")]
    [InlineData("10 02 00", "unexpected end of packet")]
    [InlineData("10 00 00", "trailing bytes after the packet")]
    [InlineData("10 ff ff ff ff ff ff ff ff ff 7f", "LEB128 value overflows 64 bits")] // a
    ↪ tenth byte above 1
    [InlineData("10 80 80 80 80 80 80 80 80 80 81", "LEB128 value overflows 64 bits")] // an
    ↪ eleventh byte
    [InlineData("10 01 01 01", "the root link is not a list")]
    [InlineData("10 02 00 02 04 06", "broken element chain")] // the chain ends in marker 2, not
    ↪ null
    [InlineData("10 01 04 02", "broken element chain")]
    [InlineData("10 02 02 00 04 06", "marker 2 used as a value")]
    [InlineData("10 03 01 00 06 00 04 07", "marker 1 cannot start a typed value")]
    [InlineData("12 02 14 05 01 20 02 02 06 00 04 07", "a number needs exactly one value")] //
    ↪ (Number) alone
    [InlineData("10 03 05 00 06 00 04 07", "an identified link needs an id")]
    [InlineData("10 03 02 03 06 00 04 07", "expected a unary number")] // (Number 3): marker 3
    ↪ is no number
    [InlineData("10 05 05 05 06 00 02 07 08 00 04 09", "expected a unary number")] // (Number 6)
    ↪ where link 6 is (5 5)
    [InlineData("10 04 00 00 05 06 07 00 04 08", "a link id must be a reference")] // the id is
    ↪ ()
    [InlineData("16 00", "the explicit layout keeps the header width bits clear")]
    [InlineData("12 01 04 05 00", "arity must be at least 1")]
    [InlineData("12 01 18 01 01 05 00", "link length outside the section arity 1..2")] // a link
    ↪ of 6
    [InlineData("12 01 f8 ff ff ff ff ff ff ff ff 01 ff ff ff ff ff ff ff ff ff 01 00", "arity
    ↪ range overflows 64 bits")]
    [InlineData("12 01 24 ff ff ff ff ff ff ff ff ff 01 01", "addresses overflow 64 bits")] //
    ↪ the gap
    [InlineData("12 01 20 ff ff ff ff ff ff ff ff ff 01", "addresses overflow 64 bits")] // the
    ↪ count
    [InlineData("12 01 24 06 01 01 01", "a LiNo packet stores its links contiguously from
    ↪ address 6, found link 7 where 6 belongs")]
    [InlineData("12 02 24 05 01 24 01 01 01 01 04 06", "a LiNo packet stores its links
    ↪ contiguously from address 6, found link 8 where 7 belongs")] // a hole
    public void MalformedPacketsAreRejected(string hex, string detail)
    {
        var error = DecodeError(Unhex(hex));
        Assert.Equal(BinaryErrorKind.Malformed, error.Kind);
        Assert.Equal(detail, error.Detail);
    }

    private static List<(ulong Address, PacketReference[] References)> FromAddress6(params
    ↪ PacketReference[][] links) =>
```

```
403         links.Select((references, index) => (6UL + (ulong)index, references)).ToList();
404
405     [Fact]
406     public void InvalidCodePointsAreRejected()
407     {
408         // (String (0x110000)) is not a valid code point.
409         var links = FromAddress6(
410             new[] { PacketReference.External(0x11 0000), PacketReference.Null },
411             Internals(3, 6),
412             new[] { PacketReference.Internal(7), PacketReference.Null },
413             Internals(4, 8));
414         Assert.Equal("invalid code point 1114112", DecodeError(LinksPacket.Pack(true, links,
    ↪  false).ToBytes()).Detail);
415     }
416
417     [Fact]
418     public void HostilePacketsHitLimits()
419     {
420         void Limited(string hex, DecodeLimits limits) =>
421             Assert.True(BinaryErrorKind.LimitExceeded == DecodeError(Unhex(hex), limits).Kind,
    ↪  hex);
422         var fewLinks = new DecodeLimits { MaxLinks = 8 };
423         Limited("10 09", fewLinks);
424         Limited("12 09", fewLinks); // more sections than links allowed
425         Limited("12 02 20 05 20 05", fewLinks); // 10 in all
426         var fewReferences = new DecodeLimits { MaxReferences = 5 };
427         Limited("10 03 01 01 01 01 01 01", fewReferences);
428         Limited("12 01 18 00 01 05", fewReferences); // one link of 6
429
430         // A doubling chain `d(k) = (d(k-1) d(k-1))` expands to 2^k nodes.
431         var chain = new List<PacketReference[]> { new[] { PacketReference.Null,
    ↪  PacketReference.Null } };
432         for (ulong address = 6; address < 60; address++)
433         {
434             chain.Add(Internals(address, address));
435         }
436         var last = 5 + (ulong)chain.Count;
437         chain.Add(new[] { PacketReference.Internal(last), PacketReference.Null });
438         chain.Add(Internals(4, last + 1));
439         var bomb = LinksPacket.Pack(false, FromAddress6(chain.ToArray()), false);
440         Assert.Equal("limit exceeded: too many LiNo nodes", DecodeError(bomb.ToBytes(), new
    ↪  DecodeLimits { MaxNodes = 1 << 12 }).Message);
441
442         var deep = string.Concat(Enumerable.Repeat("(y ", 40)) + "x" + new string(')', 40);
443         var bytes = new BinaryLinoCodec().Encode(Parse(deep));
444         Assert.Equal("limit exceeded: nesting deeper than 10", DecodeError(bytes, new
    ↪  DecodeLimits { MaxDepth = 10 }).Message);
445
446         var threeLinks = new BinaryLinoCodec().Encode(Parse("a b c"));
447         Assert.Equal("limit exceeded: chain too long", DecodeError(threeLinks, new DecodeLimits
    ↪  { MaxNodes = 2 }).Message);
448         Assert.Equal(ulong.MaxValue, DecodeLimits.Unlimited.MaxLinks);
449     }
450 }
```

1.28 ./Link.Foundation.Links.Notation.Tests/BinaryLinoCodecTests.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.IO;
4  using System.Linq;
5  using Xunit;
6  using Link.Foundation.Links.Notation.Binary;
7
8  using LinoLink = Link.Foundation.Links.Notation.Link<string>;
9
10 namespace Link.Foundation.Links.Notation.Tests.Binary;
11
12 /// <summary>Binary and text LiNo protocol codecs (issue #105).</summary>
13 public sealed class BinaryLinoCodecTests
14 {
15     [Fact]
16     public void EncoderUsesConfiguredLimits()
17     {
18         var document = new[] { new LinoLink("abcdef", new[] { new LinoLink("value") }) };
19         foreach (var limits in new[]
20         {
21             new DecodeLimits { MaxNodes = 1 },
22             new DecodeLimits { MaxStringBytes = 2 },
23             new DecodeLimits { MaxDepth = 1 },
```

```
            new DecodeLimits { MaxLinks = 1 },
            new DecodeLimits { MaxReferences = 1 },
        })
        {
            Assert.Throws<BinaryNotationException>(() => new BinaryLinoCodec { Limits = limits }.Encode(document));
        }
        var deep = new LinoLink("a");
        for (var index = 0; index < 70; index++) deep = new LinoLink(null, new[] { deep });
        var model = new[] { deep };
        var codec = new BinaryLinoCodec { Limits = new DecodeLimits { MaxDepth = 80 } };
        Assert.Equal(model, codec.Decode(codec.Encode(model)));
    }

    internal static readonly string[] Corpus =
    {
        "() ((1 1))",
        "((1: 1 1)) ((1: 1 2))",
        "((1 1)) ()",
        "((1: 1 1)) ()",
        "(($i: $s $t)) (($i: $s $t))",
        "((($index: $source $target)) (($index: $target $source)))",
        "(a b c d)",
        "(name: 'with space' \"it's\")",
        "((a))",
        "(((a)))",
        "(a (b c) ((d)))",
        "1\n2\n3",
        "hello",
        "\U0001F600 привет \u4E16\u754C",
        "0",
        "007",
        "9223372036854775807",
        "9223372036854775808",
        "18446744073709551615",
        "18446744073709551616",
        "'multi\nline'",
        ".dot",
        "'a''b\"c`d'",
        "(1: (2: 3 4) 5)",
        "() ()",
        "(* *)",
        "(type: type type)",
    };

    internal static IEnumerable<BinaryLinoOptions> AllOptions()
    {
        foreach (var externalReferences in new[] { false, true })
        {
            foreach (var arity in new[] { ArityRange.Doublets, ArityRange.Between(2, 3), ArityRange.AtLeast(1) })
            {
                foreach (var packedWidths in new[] { false, true })
                {
                    yield return new BinaryLinoOptions
                    {
                        ExternalReferences = externalReferences,
                        Arity = arity,
                        PackedWidths = packedWidths,
                    };
                }
            }
        }
    }

    private static IReadOnlyList<LinoLink> Parse(string text) => LinoFormat.ParseDocument(text);

    [Fact]
    public void EveryOptionSetRoundTripsTheCorpus()
    {
        foreach (var text in Corpus)
        {
            var document = Parse(text);
            foreach (var options in AllOptions())
            {
                var binary = new BinaryLinoCodec(options);
                var bytes = binary.Encode(document);
                Assert.InRange(bytes[0], 0x10, 0x1F);
```

```
                Assert.True(document.SequenceEqual(binary.Decode(bytes)), $"{text} with {options}");
            }
        }
    }

    [Fact]
    public void CanonicalTextRoundTripsTheCorpus()
    {
        foreach (var text in Corpus)
        {
            var document = Parse(text);
            var canonical = LinoFormat.FormatDocument(document);
            Assert.True(document.SequenceEqual(Parse(canonical)), $"{text} -> {canonical}");
        }
        Assert.Equal("() ((1 1))", LinoFormat.FormatDocument(Parse("(()((1 1)))")));
        Assert.Equal("((1: 1 1)) ((1: 1 2))", LinoFormat.FormatDocument(Parse("((1: 1 1)) ((1: 1 2))")));
        // Only a binary message carries an id without values; links-notation reads its text back as the reference.
        Assert.Equal("(a:)", LinoFormat.FormatDocument(new[] { LinoFormat.Link("a", new List<LinoLink>()) }));
        Assert.True(Parse("(a:)").SequenceEqual(new[] { LinoFormat.Reference("a") }));
    }

    [Fact]
    public void ParsedDocumentsMatchTheRustModel()
    {
        // The C# parser wraps single references in an extra group; the
        // canonical model drops it so both ports decode to equal documents.
        // These structures match the canonical model used by the shared vectors.
        var a = LinoFormat.Reference("a");
        LinoLink Group(LinoLink value) => LinoFormat.Link(null, new List<LinoLink> { value });
        Assert.Equal(new[] { a }, Parse("a"));
        Assert.Equal(new[] { a }, Parse("(a)"));
        Assert.Equal(new[] { Group(a) }, Parse("((a))"));
        Assert.Equal(new[] { Group(Group(a)) }, Parse("(((a)))"));
        Assert.Equal(new[] { LinoFormat.Link("a", new List<LinoLink> { LinoFormat.Reference("b") }) }, Parse("(a: (b))"));
        Assert.Equal(new[] { LinoFormat.Reference("1"), LinoFormat.Reference("2"), LinoFormat.Reference("3") }, Parse("1\n2\n3"));
    }

    [Fact]
    public void ReferencesAreQuotedOnlyWhenNeeded()
    {
        Assert.Equal("plain", LinoFormat.FormatReference("plain"));
        Assert.Equal("$x", LinoFormat.FormatReference("$x"));
        Assert.Equal("~1{}", LinoFormat.FormatReference(""));
        Assert.Equal("'a b'", LinoFormat.FormatReference("a b"));
        Assert.Equal("\"it's\"", LinoFormat.FormatReference("it's"));
        Assert.Equal("`'\"`", LinoFormat.FormatReference("'\""));
        Assert.Equal("\"\"\"'\"`\"\"\"", LinoFormat.FormatReference("'\"`"));
        foreach (var reference in new[]
        {
            "", "a b", "a:b", "(x)", "it's", "'\"", "'\"`", "tab\there", "a\nb", "'x", "x'", "'", "''", "'''",
            "\"'`x`'\"", "a''''b\"\"`", "`'\"\"\"'''``", "(a) ''",
        })
        {
            var formatted = LinoFormat.FormatReference(reference);
            var document = Parse(formatted);
            Assert.Equal(formatted, LinoFormat.FormatDocument(document));
            Assert.Equal(new[] { LinoFormat.Reference(reference) }, document);
        }
    }

    [Fact]
    public void EveryShortReferenceOverDelimitersRoundTrips()
    {
        var alphabet = new[] { '\'', '"', '`', 'a', ' ', '(', ')', ':' };
        var stack = new Stack<string>();
        stack.Push(string.Empty);
        while (stack.TryPop(out var reference))
        {
            if (reference.Length > 0)
            {
```

```
                var formatted = LinoFormat.FormatReference(reference);
                var document = Parse(formatted);
                Assert.True(
                    document.Count == 1 && document[0].Equals(LinoFormat.Reference(reference)),
                    $"{reference} as {formatted}");
            }
            if (reference.Length < 5)
            {
                foreach (var character in alphabet)
                {
                    stack.Push(reference + character);
                }
            }
        }
    }

    [Fact]
    public void CommentReferencesSurviveBinaryAndTextRoundTrips()
    {
        foreach (var reference in new[] { "#", "#tag", "# with space", "issue#1047" })
        {
            var document = new[] { LinoFormat.Reference(reference) };
            var codec = new BinaryLinoCodec();
            var decoded = codec.Decode(codec.Encode(document));
            Assert.Equal(document, decoded);
            Assert.Equal(document, Parse(LinoFormat.FormatDocument(decoded)));
        }
    }

    [Fact]
    public void TextHelpersPreserveUnicodeWhitespaceReferences()
    {
        foreach (var reference in new[] { "\u00a0", "\u2003", "\u2028" })
        {
            var document = new[] { LinoFormat.Reference(reference) };
            Assert.Equal(document, Parse(reference));
            Assert.Equal(document, Parse(LinoFormat.FormatDocument(document)));
        }
    }
    [Fact]
    public void NativeParserGroupsRoundTripWithoutCanonicalization()
    {
        foreach (var text in Corpus)
        {
            var document = new Parser().Parse(text).ToList();
            foreach (var options in AllOptions())
            {
                var codec = new BinaryLinoCodec(options);
                Assert.Equal(document, codec.Decode(codec.Encode(document)));
            }
        }
    }

    [Fact]
    public void InvalidEncoderModelsAreErrors()
    {
        var link = LinoFormat.Reference("a");
        for (var depth = 0; depth < DecodeLimits.Default.MaxDepth; depth++)
        {
            link = LinoFormat.Link(null, new List<LinoLink> { link });
        }
        Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
    ↪ new BinaryLinoCodec().Encode(new[] { link })).Kind);
        var invalid = new BinaryLinoOptions().WithArity(ArityRange.Between(0, 2));
        Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
    ↪ new BinaryLinoCodec(invalid).Encode(Array.Empty<LinoLink>())).Kind);
        Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
    ↪ new BinaryLinoCodec().Encode(new[] { LinoFormat.Reference("\ud800") })).Kind);
    }

    [Fact]
    public void IdsDoNotAddAnExtraModelNestingLevel()
    {
        var link = LinoFormat.Link("id", new List<LinoLink>());
        for (var depth = 1; depth < DecodeLimits.Default.MaxDepth; depth++)
        {
            link = LinoFormat.Link(null, new List<LinoLink> { link });
        }
```

```
245         var document = new[] { link };
246         foreach (var options in AllOptions())
247         {
248             var codec = new BinaryLinoCodec(options);
249             Assert.Equal(document, codec.Decode(codec.Encode(document)));
250         }
251     }
252 
253     [Fact]
254     public void InMemoryPacketsObeyLimits()
255     {
256         var packet = LinoMapping.EncodeDocument(Parse("(a b)"));
257         foreach (var limits in new[] { new DecodeLimits { MaxLinks = 0 }, new DecodeLimits {
    ↪ MaxReferences = 0 } })
258         {
259             Assert.Equal(BinaryErrorKind.LimitExceeded,
    ↪ Assert.Throws<BinaryNotationException>(() => LinoMapping.DecodeDocument(packet,
    ↪ limits)).Kind);
260         }
261         var links = new List<(ulong, PacketReference[])> { (6, new[] { PacketReference.Null,
    ↪ PacketReference.Null }) };
262         for (ulong address = 7; address < 80; address++)
263         {
264             links.Add((address, new[] { PacketReference.Internal(address - 1),
    ↪ PacketReference.Null }));
265         }
266         var deep = LinksPacket.Pack(false, links, false);
267         Assert.Equal(BinaryErrorKind.LimitExceeded, Assert.Throws<BinaryNotationException>(() =>
    ↪ LinoMapping.DecodeDocument(deep)).Kind);
268         Assert.Equal(BinaryErrorKind.Unencodable, Assert.Throws<BinaryNotationException>(() =>
    ↪ LinksPacket.Pack(false, new[] { (ulong.MaxValue, new[] { PacketReference.Null }) },
    ↪ false)).Kind);
269     }
270 
271     [Fact]
272     public void InMemoryPacketsRejectInvalidWireShapes()
273     {
274         foreach (var kind in new[] { "external", "width", "arity" })
275         {
276             var packet = LinksPacket.Pack(true, new[] { (6UL, new[] {
    ↪ PacketReference.External(3) }) }, false);
277             if (kind == "external") packet.ExternalReferences = false;
278             if (kind == "width") packet.Sections[0].Width = 3;
279             if (kind == "arity") packet.Sections[0].Arity = ArityRange.Doublets;
280             Assert.Equal(BinaryErrorKind.Malformed, Assert.Throws<BinaryNotationException>(() =>
    ↪ LinoMapping.DecodeDocument(packet)).Kind);
281         }
282     }
283 
284     [Fact]
285     public void CompactAddressOverflowIsRejectedBeforeReadingLinks()
286     {
287         var bytes = new List<byte> { 0x10 };
288         LinksPacket.WriteLeb128(bytes, ulong.MaxValue);
289         var exception = Assert.Throws<BinaryNotationException>(() =>
    ↪ LinksPacket.FromBytes(bytes.ToArray(), DecodeLimits.Unlimited));
290         Assert.Equal(BinaryErrorKind.Malformed, exception.Kind);
291         Assert.Contains("addresses overflow", exception.Detail);
292     }
293 
294     [Fact]
295     public void RepeatedReferencesObeyATotalUtf8ExpansionBudget()
296     {
297         foreach (var reference in new[] { "\U0001F600", "12" })
298         {
299             var document = new[] { LinoFormat.Reference(reference),
    ↪ LinoFormat.Reference(reference) };
300             foreach (var options in AllOptions())
301             {
302                 var packet = LinoMapping.EncodeDocument(document, options);
303                 var bytes = System.Text.Encoding.UTF8.GetByteCount(reference) * 2;
304                 Assert.Equal(document, LinoMapping.DecodeDocument(packet, new DecodeLimits {
    ↪ MaxStringBytes = bytes }));
305                 Assert.Equal(BinaryErrorKind.LimitExceeded,
    ↪ Assert.Throws<BinaryNotationException>(() => LinoMapping.DecodeDocument(packet, new
    ↪ DecodeLimits { MaxStringBytes = bytes - 1 })).Kind);
306             }
```

```
        }
    }

    [Fact]
    public void StringBudgetStopsBeforeLaterScalars()
    {
        var bytes = Convert.FromHexString("13024605011001030000009fffffff9effffff0028fff06");
        var codec = new BinaryLinoCodec { Limits = new DecodeLimits { MaxStringBytes = 1 } };
        Assert.Equal(BinaryErrorKind.LimitExceeded, Assert.Throws<BinaryNotationException>(() =>
↪   codec.Decode(bytes)).Kind);
    }

    [Fact]
    public void StreamPacketsAreReadOneAtATimeAndTruncationIsRejected()
    {
        var bytes = new BinaryLinoCodec().Encode(Parse("(a b)"));
        for (var end = 0; end < bytes.Length; end++)
        {
            Assert.Throws<BinaryNotationException>(() => LinksPacket.FromBytes(bytes[..end]));
        }
        var reader = PacketReader.FromBytes(bytes.Concat(bytes).ToArray());
        Assert.NotNull(LinksPacket.ReadFrom(reader));
        Assert.NotNull(LinksPacket.ReadFrom(reader));
        Assert.Null(LinksPacket.ReadFrom(reader));
    }
}
```

1.29 ./Link.Foundation.Links.Notation.Tests/CommentsTests.cs

```
using System.Linq;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    /// <summary>
    /// Conformance tests for line comments.
    ///
    /// https://github.com/link-foundation/links-notation/issues/301
    ///
    /// <c>#</c> starts a comment that runs to the end of the line, unless it sits
    /// inside a token or inside a delimited reference. Parsers read comments by
    /// default and can be told to treat <c>#</c> as an ordinary character again.
    /// The table below is shared with the Rust, JavaScript, Python, Go, Java and
    /// PHP suites, so a document written by one implementation reads the same in
    /// all of them.
    /// </summary>
    public static class CommentsTests
    {
        /// <summary>
        /// Renders a parsed node unambiguously: every reference is wrapped in
        /// angle brackets.
        /// </summary>
        private static string Render(Link<string> node)
        {
            if (node.Values == null || node.Values.Count == 0)
            {
                return "<" + (node.Id ?? "") + ">";
            }
            var head = node.Id == null ? "" : "<" + node.Id + ">: ";
            return "(" + head + string.Join(" ", node.Values.Select(Render)) + ")";
        }

        private static string RenderedWith(Parser parser, string source) =>
            string.Join("\n", parser.Parse(source).Select(Render));

        private static string Rendered(string source) => RenderedWith(new Parser(), source);

        private static void AssertParsesAs(string source, string expected)
        {
            Assert.Equal(expected, Rendered(source));
        }

        [Fact]
        public static void ALineThatStartsWithAHashIsAComment()
        {
            AssertParsesAs("# a b\n", "");
        }

        [Fact]
        public static void ACommentMayHoldAColon()
```

```
        {
            // The document from #301: prose with a colon used to be rejected.
            AssertParsesAs("# a: b\n", "");
        }

        [Fact]
        public static void ACommentMayHoldAnythingAtAll()
        {
            AssertParsesAs("# ) : ( \" ' ` #\n", "");
        }

        [Fact]
        public static void ACommentEndsAtTheEndOfItsLine()
        {
            AssertParsesAs("# note\na: b\n", "(<a>: <b>)");
        }

        [Fact]
        public static void ACommentMayFollowALink()
        {
            AssertParsesAs("a: b # why\n", "(<a>: <b>)");
        }

        [Fact]
        public static void ACommentMayFollowAGroup()
        {
            AssertParsesAs("(a b) # why\n", "(<a> <b>)");
        }

        [Fact]
        public static void ACommentNeedsNoClosingNewline()
        {
            AssertParsesAs("a: b # why", "(<a>: <b>)");
        }

        [Fact]
        public static void ACommentLineInsideAnIndentedBlockIsSkipped()
        {
            Assert.Equal(
                Rendered("parent\n  child\n"),
                Rendered("parent\n  # what the child is for\n  child\n"));
        }

        [Fact]
        public static void ACommentLineInsideAGroupIsSkipped()
        {
            Assert.Equal(
                Rendered("(\n  a\n  b\n)\n"),
                Rendered("(\n  a\n  # why\n  b\n)\n"));
        }

        [Fact]
        public static void ALineOfSpacesSeparatesLinksTheWayAnEmptyLineDoes()
        {
            // Blanking a comment leaves a line of spaces behind, so such a line
            // has to read the way an empty line does.
            Assert.Equal(Rendered("a\n\nb\n"), Rendered("a\n   \nb\n"));
        }

        [Fact]
        public static void ADocumentOfCommentsAloneHoldsNoLinks()
        {
            AssertParsesAs("# one\n# two\n", "");
        }

        [Fact]
        public static void AHashInsideATokenIsAnOrdinaryCharacter()
        {
            AssertParsesAs("issue#1047\n", "(<issue#1047>)");
        }

        [Fact]
        public static void AHashThatOpensATokenIsAnOrdinaryCharacter()
        {
            AssertParsesAs("a: b#c\n", "(<a>: <b#c>)");
        }

        [Fact]
        public static void AHashInsideADelimitedReferenceIsContent()
```

```
        {
            AssertParsesAs("\"# not a comment\" a\n", "(<# not a comment> <a>)");
        }

        [Fact]
        public static void ACommentMayFollowADelimitedReference()
        {
            AssertParsesAs("\"a\" # why\n", "(<a>)");
        }

        [Fact]
        public static void AHashInsideAMultiLineDelimitedReferenceIsContent()
        {
            AssertParsesAs("\"a # b\nc\" d\n", "(<a # b\nc> <d>)");
        }

        [Fact]
        public static void CommentsAreOnByDefault()
        {
            Assert.True(new Parser().Comments);
            AssertParsesAs("# a b\n", "");
        }

        [Fact]
        public static void AParserWithoutCommentsKeepsTheHash()
        {
            Assert.Equal("(<#> <a> <b>)", RenderedWith(new Parser(comments: false), "# a b\n"));
        }

        [Fact]
        public static void BlankingACommentKeepsTheLengthOfTheDocument()
        {
            Assert.Equal("a: b      \n", Comments.StripComments("a: b # why\n"));
            Assert.Equal("\"# kept\"\n", Comments.StripComments("\"# kept\"\n"));
            Assert.Equal("issue#1047\n", Comments.StripComments("issue#1047\n"));
            Assert.Equal("     \n", Comments.StripComments("# why\n"));
        }

        [Fact]
        public static void ACommentDoesNotMoveThePositionALaterErrorIsReportedAt()
        {
            var error = Assert.Throws<ParseException>(
                () => new Parser().Parse("# a comment\nstage: rust: nextest\n"));

            Assert.Equal(2, error.Line);
            Assert.Equal(12, error.Column);
        }

        [Fact]
        public static void AReferenceThatBeginsWithAHashIsWrittenQuoted()
        {
            // Without the quotes the document would read as `a` followed by a comment.
            var document = new Link<string>(new Link<string>("a"), new
↪   Link<string>("#tag")).ToString();

            Assert.Equal("(a '#tag')", document);
            Assert.Equal("(<a> <#tag>)", Rendered(document));
        }

        [Fact]
        public static void AHashThatCannotOpenACommentIsLeftUnquoted()
        {
            Assert.Equal("issue#1047", Link<string>.EscapeReference("issue#1047"));
            Assert.Equal("'#'", Link<string>.EscapeReference("#"));
            Assert.Equal("'#ff0000'", Link<string>.EscapeReference("#ff0000"));
        }
    }
}
```

1.30 ./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class EdgeCaseParserTests
    {
        [Fact]
        public static void EmptyLinkTest()
```

```
        {
            var source = @":";
            var parser = new Parser();
            // Standalone ':' is now forbidden and should throw an exception
            Assert.Throws<ParseException>(() => parser.Parse(source));
        }

        [Fact]
        public static void EmptyLinkWithParenthesesTest()
        {
            var source = @"()";
            var target = @"()";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void EmptyLinkWithEmptySelfReferenceTest()
        {
            var source = @"(:)";
            var parser = new Parser();
            // '(:)' is now forbidden and should throw an exception
            Assert.Throws<ParseException>(() => parser.Parse(source));
        }

        [Fact]
        public static void AllFeaturesTest()
        {
            // Test single-line link with id
            var input = "id: value1 value2";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test multi-line link with id
            input = "(id: value1 value2)";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test link without id (single-line) - now forbidden
            input = ": value1 value2";
            Assert.Throws<ParseException>(() => new Parser().Parse(input));

            // Test link without id (multi-line) - now forbidden
            input = "(: value1 value2)";
            Assert.Throws<ParseException>(() => new Parser().Parse(input));

            // Test singlet link
            input = "(singlet)";
            result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values!);
            Assert.Equal("singlet", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);

            // Test value link
            input = "(value1 value2 value3)";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test quoted references
            input = @"(""id with spaces"": ""value with spaces"")";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test single-quoted references
            input = "('id': 'value')";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // Test nested links
            input = "(outer: (inner: value))";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }
```

```
89          [Fact]
90          public static void TestSingletLinks()
91          {
92              // Test singlet (1)
93              var input = "(1)";
94              var result = new Parser().Parse(input);
95              Assert.Single(result);
96              Assert.Null(result[0].Id);
97              Assert.NotNull(result[0].Values);
98              Assert.Single(result[0].Values!);
99              Assert.Equal("1", result[0].Values?[0].Id);
100             Assert.Null(result[0].Values?[0].Values);
101
102             // Test (1 2)
103             input = "(1 2)";
104             result = new Parser().Parse(input);
105             Assert.Single(result);
106             Assert.Null(result[0].Id);
107             Assert.NotNull(result[0].Values);
108             Assert.Equal(2, result[0].Values?.Count);
109             Assert.Equal("1", result[0].Values?[0].Id);
110             Assert.Null(result[0].Values?[0].Values);
111             Assert.Equal("2", result[0].Values?[1].Id);
112             Assert.Null(result[0].Values?[1].Values);
113
114             // Test (1 2 3)
115             input = "(1 2 3)";
116             result = new Parser().Parse(input);
117             Assert.Single(result);
118             Assert.Null(result[0].Id);
119             Assert.NotNull(result[0].Values);
120             Assert.Equal(3, result[0].Values?.Count);
121             Assert.Equal("1", result[0].Values?[0].Id);
122             Assert.Null(result[0].Values?[0].Values);
123             Assert.Equal("2", result[0].Values?[1].Id);
124             Assert.Null(result[0].Values?[1].Values);
125             Assert.Equal("3", result[0].Values?[2].Id);
126             Assert.Null(result[0].Values?[2].Values);
127
128             // Test (1 2 3 4)
129             input = "(1 2 3 4)";
130             result = new Parser().Parse(input);
131             Assert.Single(result);
132             Assert.Null(result[0].Id);
133             Assert.NotNull(result[0].Values);
134             Assert.Equal(4, result[0].Values?.Count);
135             Assert.Equal("1", result[0].Values?[0].Id);
136             Assert.Null(result[0].Values?[0].Values);
137             Assert.Equal("2", result[0].Values?[1].Id);
138             Assert.Null(result[0].Values?[1].Values);
139             Assert.Equal("3", result[0].Values?[2].Id);
140             Assert.Null(result[0].Values?[2].Values);
141             Assert.Equal("4", result[0].Values?[3].Id);
142             Assert.Null(result[0].Values?[3].Values);
143         }
144
145         [Fact]
146         public static void EmptyDocumentTest()
147         {
148             var input = "";
149             // Empty document should return empty list
150             var result = new Parser().Parse(input);
151             Assert.Empty(result);
152         }
153
154         [Fact]
155         public static void WhitespaceOnlyTest()
156         {
157             var input = "   \n   \n   ";
158             // Whitespace-only document should return empty list (similar to empty document)
159             var result = new Parser().Parse(input);
160             Assert.Empty(result);
161         }
162
163         [Fact]
164         public static void EmptyLinksTest()
165         {
166             var input = "()";
```

```
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);

            // '(:)' is now forbidden
            input = "(:)";
            Assert.Throws<ParseException>(() => new Parser().Parse(input));

            input = "(id:)";
            result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void InvalidInputTest()
        {
            var input = "(invalid";
            // Unclosed parentheses should throw an exception
            Assert.Throws<ParseException>(() => new Parser().Parse(input));
        }
    }
}
```

1.31 ./Link.Foundation.Links.Notation.Tests/EmptyReferenceTests.cs

```
using System.Collections.Generic;
using System.Linq;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    /// <summary>
    /// Conformance tests for the empty reference.
    ///
    /// https://github.com/link-foundation/links-notation/issues/288
    ///
    /// A bare delimiter pair is the empty reference. The three delimiters
    /// <c>""</c>, <c>''</c> and <c>``</c> behave identically, and every longer
    /// n-quote run keeps the meaning it already had. The table below is shared
    /// with the Rust, JavaScript, Python, Go, Java and PHP suites, so a document
    /// written by one implementation reads the same in all of them.
    /// </summary>
    public static class EmptyReferenceTests
    {
        /// <summary>
        /// Renders a parsed node unambiguously: every reference is wrapped in
        /// angle brackets so an empty one is visible as &lt;&gt;.
        /// </summary>
        private static string Render(Link<string> node)
        {
            if (node.Values == null || node.Values.Count == 0)
            {
                return "<" + (node.Id ?? "") + ">";
            }
            var head = node.Id == null ? "" : "<" + node.Id + ">: ";
            return "(" + head + string.Join(" ", node.Values.Select(Render)) + ")";
        }

        private static string Rendered(string source)
        {
            var links = new Parser().Parse(source);
            return string.Join("\n", links.Select(Render));
        }

        private static void AssertParsesAs(string source, string expected)
        {
            Assert.Equal(expected, Rendered(source));
        }

        [Fact]
        public static void BareDelimiterPairIsTheEmptyReference()
        {
            AssertParsesAs("(a \"\" b)", "(<a> <> <b>)");
        }

        [Fact]
        public static void EveryDelimiterStyleYieldsTheSameEmptyReference()
        {
            AssertParsesAs("(a \"\" b)", "(<a> <> <b>)");
            AssertParsesAs("(a '' b)", "(<a> <> <b>)");
            AssertParsesAs("(a `` b)", "(<a> <> <b>)");
```

```
        }

        [Fact]
        public static void AdjacentEmptyReferencesStaySeparate()
        {
            AssertParsesAs("(a \"\" \"\" b)", "(<a> <> <> <b>)");
            AssertParsesAs("(a '' '' b)", "(<a> <> <> <b>)");
            AssertParsesAs("(a `` `` b)", "(<a> <> <> <b>)");
            AssertParsesAs("(a \"\"  \"\" b)", "(<a> <> <> <b>)");
        }

        [Fact]
        public static void NestedEmptyReferencesParse()
        {
            AssertParsesAs("(\"\" (\"\" 1))", "(<> (<> <1>))");
            AssertParsesAs("(\"\" ('' 1))", "(<> (<> <1>))");
            AssertParsesAs("(\"x\" (\"\" 1))", "(<x> (<> <1>))");
            AssertParsesAs("(\"\" (\"x\" 1))", "(<> (<x> <1>))");
            AssertParsesAs("(\"\" x (\"\" 1))", "(<> <x> (<> <1>))");
            AssertParsesAs("(\"\" 1 (\"\" 1))", "(<> <1> (<> <1>))");
        }

        [Fact]
        public static void EmptyReferenceIsValidAsAnId()
        {
            AssertParsesAs("(\"\": 1)", "(<>: <1>)");
            AssertParsesAs("(o: (\"\" (o: (\"\" 1))))", "(<o>: (<> (<o>: (<> <1>))))");
        }

        [Fact]
        public static void NQuoteDelimitedBodiesAreUnchanged()
        {
            // A run that encloses a substantive body keeps its n-quote meaning.
            AssertParsesAs("(a \"\"x\"\" b)", "(<a> <x> <b>)");
            AssertParsesAs("(x \"\" \" \"\")", "(<x> < \" >)");
            AssertParsesAs("(x ' \" ')", "(<x> < \" >)");
            // An n-quote-delimited empty is still empty.
            AssertParsesAs("(a \"\"\"\" b)", "(<a> <> <b>)");
        }

        [Fact]
        public static void ASingleSpaceStillReadsAsASpace()
        {
            AssertParsesAs("(a \" \" b)", "(<a> < > <b>)");
        }

        [Fact]
        public static void EmptyReferenceSurvivesARoundTrip()
        {
            var sources = new[]
            {
                "(a \"\" b)",
                "(a \"\" \"\" b)",
                "(\"\" (\"\" 1))",
                "(\"\": 1)",
                "(o: (\"\" (o: (\"\" 1))))",
            };
            foreach (var source in sources)
            {
                var formatted = ((IList<Link<string>>)new Parser().Parse(source)).Format();
                var reparsed = ((IList<Link<string>>)new Parser().Parse(formatted)).Format();
                Assert.Equal(formatted, reparsed);
            }
        }

        [Fact]
        public static void EmptyReferenceIsWrittenAsAVersionedLiteral()
        {
            var links = (IList<Link<string>>)new Parser().Parse("(a \"\" b)");
            Assert.Equal("(a ~1{} b)", links.Format());
        }
    }
}
```

1.32 ./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs

```
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
```

```
5       public class FormatConfigTests
6       {
7           [Fact]
8           public void FormatConfigBasicTest()
9           {
10              var config = new FormatConfig();
11              Assert.False(config.LessParentheses);
12              Assert.Equal(80, config.MaxLineLength);
13              Assert.False(config.IndentLongLines);
14          }
15
16          [Fact]
17          public void FormatWithLineLengthLimitTest()
18          {
19              // Create a config with line length limit
20              // The line would be 32+ chars, so use threshold of 30
21              var config = new FormatConfig
22              {
23                  IndentLongLines = true,
24                  MaxLineLength = 30,
25                  PreferInline = false
26              };
27
28              // Verify config is set correctly
29              Assert.True(config.IndentLongLines);
30              Assert.Equal(30, config.MaxLineLength);
31              // Note: Full formatting integration would test actual output here
32          }
33
34          [Fact]
35          public void FormatWithMaxInlineRefsTest()
36          {
37              // Create a config with MaxInlineRefs=3 (should trigger indentation with 4 refs)
38              var config = new FormatConfig
39              {
40                  MaxInlineRefs = 3,
41                  PreferInline = false
42              };
43
44              // Verify config is set correctly
45              Assert.Equal(3, config.MaxInlineRefs);
46              Assert.True(config.ShouldIndentByRefCount(4));
47              // Note: Full formatting integration would test actual output here
48          }
49
50          [Fact]
51          public void FormatWithConsecutiveGroupingTest()
52          {
53              // Create a config with consecutive grouping enabled
54              var config = new FormatConfig
55              {
56                  GroupConsecutive = true
57              };
58
59              // Verify config is set correctly
60              Assert.True(config.GroupConsecutive);
61              // Note: Full formatting integration would test grouping behavior here
62          }
63
64          [Fact]
65          public void FormatConfigCustomIndentTest()
66          {
67              var config = new FormatConfig
68              {
69                  MaxInlineRefs = 3,
70                  PreferInline = false,
71                  IndentString = "    " // 4 spaces
72              };
73
74              Assert.Equal("    ", config.IndentString);
75          }
76
77          [Fact]
78          public void FormatConfigLessParenthesesTest()
79          {
80              var config = new FormatConfig
81              {
82                  LessParentheses = true
83              };
84
```

```
            Assert.True(config.LessParentheses);
        }

        [Fact]
        public void RoundtripWithLineLengthFormattingTest()
        {
            //This test would require full format integration
            // For now, just verify config creation
            var config = new FormatConfig
            {
                MaxInlineRefs = 2,
                PreferInline = false
            };

            Assert.Equal(2, config.MaxInlineRefs);
        }

        [Fact]
        public void ShouldIndentByLengthTest()
        {
            var config = new FormatConfig
            {
                IndentLongLines = true,
                MaxLineLength = 80
            };

            Assert.False(config.ShouldIndentByLength("short"));
            Assert.True(config.ShouldIndentByLength(new string('a', 100)));
        }

        [Fact]
        public void ShouldIndentByRefCountTest()
        {
            var config = new FormatConfig
            {
                MaxInlineRefs = 3
            };

            Assert.False(config.ShouldIndentByRefCount(2));
            Assert.False(config.ShouldIndentByRefCount(3));
            Assert.True(config.ShouldIndentByRefCount(4));
        }
    }
}
```

1.33 ./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs

```
using Xunit;
using System.Collections.Generic;

namespace Link.Foundation.Links.Notation.Tests
{
    public class IndentationConsistencyTests
    {
        [Fact]
        public void LeadingSpacesVsNoLeadingSpaces()
        {
            // Example with 2 leading spaces (from issue #135)
            var withLeading = @"  TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
  TELEGRAM ALLOWED CHATS:
    -1002975819706
    -1002861722681
  TELEGRAM HIVE OVERRIDES:
    --all-issues
    --once
  TELEGRAM BOT VERBOSE: true";

            // Example without leading spaces (from issue #135)
            var withoutLeading = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
TELEGRAM ALLOWED CHATS:
  -1002975819706
  -1002861722681
TELEGRAM HIVE OVERRIDES:
  --all-issues
  --once
TELEGRAM BOT VERBOSE: true";

            var resultWith = new Parser().Parse(withLeading);
            var resultWithout = new Parser().Parse(withoutLeading);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultWithout.Format(), resultWith.Format());
```

```
        }

        [Fact]
        public void TwoSpacesVsFourSpacesIndentation()
        {
            // Example with 2 spaces per level
            var twoSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
TELEGRAM ALLOWED CHATS:
  -1002975819706
  -1002861722681
TELEGRAM HIVE OVERRIDES:
  --all-issues
  --once
  --auto-fork
  --skip-issues-with-prs
  --attach-logs
  --verbose
  --no-tool-check
TELEGRAM SOLVE OVERRIDES:
  --auto-fork
  --auto-continue
  --attach-logs
  --verbose
  --no-tool-check
TELEGRAM BOT VERBOSE: true";

            // Example with 4 spaces per level
            var fourSpaces = @"TELEGRAM BOT TOKEN: '849...355:AAG...rgk YZk...aPU'
TELEGRAM ALLOWED CHATS:
    -1002975819706
    -1002861722681
TELEGRAM HIVE OVERRIDES:
    --all-issues
    --once
    --auto-fork
    --skip-issues-with-prs
    --attach-logs
    --verbose
    --no-tool-check
TELEGRAM SOLVE OVERRIDES:
    --auto-fork
    --auto-continue
    --attach-logs
    --verbose
    --no-tool-check
TELEGRAM BOT VERBOSE: true";

            var resultTwo = new Parser().Parse(twoSpaces);
            var resultFour = new Parser().Parse(fourSpaces);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultFour.Format(), resultTwo.Format());
        }

        [Fact]
        public void SimpleTwoVsFourSpacesIndentation()
        {
            // Simple example with 2 spaces
            var twoSpaces = @"parent:
  child1
  child2";

            // Simple example with 4 spaces
            var fourSpaces = @"parent:
    child1
    child2";

            var resultTwo = new Parser().Parse(twoSpaces);
            var resultFour = new Parser().Parse(fourSpaces);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultFour.Format(), resultTwo.Format());
        }

        [Fact]
        public void ThreeLevelNestingWithDifferentIndentation()
        {
            // Three levels with 2 spaces
            var twoSpaces = @"level1:
  level2:
    level3a
    level3b
  level2b";
```

```

            // Three levels with 4 spaces
            var fourSpaces = @"level1:
    level2:
        level3a
        level3b
    level2b";

            var resultTwo = new Parser().Parse(twoSpaces);
            var resultFour = new Parser().Parse(fourSpaces);

            // Compare the entire formatted output (complete round trip test)
            Assert.Equal(resultFour.Format(), resultTwo.Format());
        }
    }
}
```

1.34 ./Link.Foundation.Links.Notation.Tests/IndentedIdNestedValuesTests.cs

```
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public class IndentedIdNestedValuesTests
    {
        [Fact]
        public void IndentedIdRetainsNestedValues()
        {
            var cases = new[]
            {
                new[] { "outer:\n  inner:\n    value1\n    value2\n  value3", "(outer: (inner: value1 value2) value3)" },
                new[] { "statement:\n  subject:\n    I\n  verb:\n    love\n  object:\n    you:\n      very\n      much", "(statement: (subject: I) (verb: love) (object: (you: very much)))" },
                new[] { "document:\n  (metadata: title author date)\n  content:\n    paragraph1\n    (paragraph2: text (with: nested structure))\n    paragraph3", "(document: (metadata: title author date) (content: paragraph1 (paragraph2: text (with: nested structure)) paragraph3))" },
                new[] { "image:\n    file: .gitpod.Dockerfile\n    context: ./docker-content", "(image: (file: .gitpod.Dockerfile) (context: ./docker-content))" },
                new[] { "level1:\n  level2:\n    level3a\n    level3b\n  level2b", "(level1: (level2: level3a level3b) level2b)" },
                new[] { "root:\n  child1\n  child2\n    grandchild", "(root: child1 (child2 grandchild))" },
            };

            var parser = new Parser();
            foreach (var testCase in cases)
            {
                Assert.Equal(testCase[1], parser.Parse(testCase[0]).Format());
            }
        }
    }
}
```

1.35 ./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class IndentedIdSyntaxTests
    {
        [Fact]
        public static void BasicIndentedIdSyntaxTest()
        {
            var indentedSyntax = @"3:
  papa
  loves
  mama";
            var inlineSyntax = "(3: papa loves mama)";

            var parser = new Parser();
            var indentedResult = parser.Parse(indentedSyntax);
            var inlineResult = parser.Parse(inlineSyntax);

            var indentedFormatted = indentedResult.Format();
            var inlineFormatted = inlineResult.Format();

```

```
            Assert.Equal(inlineFormatted, indentedFormatted);
            Assert.Equal("(3: papa loves mama)", indentedFormatted);
        }

        [Fact]
        public static void IndentedIdSyntaxWithSingleValueTest()
        {
            var input = @"greeting:
  hello";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("(greeting: hello)", formatted);
            Assert.Single(result);
            Assert.Equal("greeting", result[0].Id);
            Assert.Single(result[0].Values!);
            Assert.Equal("hello", result[0].Values![0].Id);
        }

        [Fact]
        public static void IndentedIdSyntaxWithMultipleValuesTest()
        {
            var input = @"action:
  run
  fast
  now";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("(action: run fast now)", formatted);
            Assert.Single(result);
            Assert.Equal("action", result[0].Id);
            Assert.Equal(3, result[0].Values!.Count);
        }

        [Fact]
        public static void IndentedIdSyntaxWithNumericIdTest()
        {
            var input = @"42:
  answer
  to
  everything";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("(42: answer to everything)", formatted);
        }

        [Fact]
        public static void IndentedIdSyntaxWithQuotedIdTest()
        {
            var input = @"""complex id"":
  value1
  value2";

            var parser = new Parser();
            var result = parser.Parse(input);
            var formatted = result.Format();

            Assert.Equal("('complex id': value1 value2)", formatted);
        }

        [Fact]
        public static void MultipleIndentedIdLinksTest()
        {
            var input = @"first:
  a
  b
second:
  c
  d";

            var parser = new Parser();
            var result = parser.Parse(input);
```

```
            var formatted = result.Format();

            Assert.Equal(2, result.Count);
            Assert.Contains("(first: a b)", formatted);
            Assert.Contains("(second: c d)", formatted);
        }

        [Fact]
        public static void MixedIndentedAndRegularSyntaxTest()
        {
            var input = @"first:
  a
  b
(second: c d)
third value";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Equal(3, result.Count);

            var formatted = result.Format();
            Assert.Contains("(first: a b)", formatted);
            Assert.Contains("(second: c d)", formatted);
            Assert.Contains("third value", formatted);
        }

        [Fact]
        public static void UnsupportedColonOnlySyntaxShouldFailTest()
        {
            var input = @":
  papa
  loves
  mama";

            var parser = new Parser();
            Assert.Throws<ParseException>(() => parser.Parse(input));
        }

        [Fact]
        public static void EmptyIndentedIdShouldWorkTest()
        {
            var input = "empty:";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("empty", result[0].Id);
            var emptyValues = result[0].Values;
            Assert.True(emptyValues == null || emptyValues.Count == 0);

            var formatted = result.Format();
            Assert.Equal("(empty)", formatted);
        }

        [Fact]
        public static void EquivalenceTestComprehensiveTest()
        {
            var testCases = new[]
            {
                new { Indented = "test:\n  one", Inline = "(test: one)" },
                new { Indented = "x:\n  a\n  b\n  c", Inline = "(x: a b c)" },
                new { Indented = "\"quoted\":\n  value", Inline = "(\"quoted\": value)" }
            };

            var parser = new Parser();

            foreach (var testCase in testCases)
            {
                var indentedResult = parser.Parse(testCase.Indented);
                var inlineResult = parser.Parse(testCase.Inline);

                var indentedFormatted = indentedResult.Format();
                var inlineFormatted = inlineResult.Format();

                Assert.Equal(inlineFormatted, indentedFormatted);
            }
        }

```

```
        [Fact]
        public static void IndentedIdWithDeeperNestingTest()
        {
            var input = @"root:
  child1
  child2
    grandchild";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);

            var rootLink = result[0];
            Assert.Equal("root", rootLink.Id);
            Assert.Equal(2, rootLink.Values!.Count);
            Assert.Equal("(root: child1 (child2 grandchild))", result.Format());
        }
    }
}
```

1.36 ./Link.Foundation.Links.Notation.Tests/LinkTests.cs

```
using System;
using Xunit;
using System.Collections.Generic;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class LinkTests
    {
        [Fact]
        public static void LinkConstructorWithIdOnlyTest()
        {
            var link = new Link<string>("test", null);
            Assert.Equal("test", link.Id);
            Assert.Null(link.Values);
        }

        [Fact]
        public static void LinkConstructorWithIdAndValuesTest()
        {
            var values = new List<Link<string>> { new Link<string>("value1"), new
↪   Link<string>("value2") };
            var link = new Link<string>("parent", values);
            Assert.Equal("parent", link.Id);
            Assert.Equal(2, link.Values?.Count);
        }

        [Fact]
        public static void LinkToStringWithIdOnlyTest()
        {
            var link = new Link<string>("test");
            Assert.Equal("(test)", link.ToString());
        }

        [Fact]
        public static void LinkToStringWithValuesOnlyTest()
        {
            var values = new List<Link<string>> { new Link<string>("value1"), new
↪   Link<string>("value2") };
            var link = new Link<string>(null, values);
            Assert.Equal("(value1 value2)", link.ToString());
        }

        [Fact]
        public static void LinkToStringWithIdAndValuesTest()
        {
            var values = new List<Link<string>> { new Link<string>("child1"), new
↪   Link<string>("child2") };
            var link = new Link<string>("parent", values);
            Assert.Equal("(parent: child1 child2)", link.ToString());
        }

        [Fact]
        public static void LinkEqualsTest()
        {
            var link1 = new Link<string>("test");
            var link2 = new Link<string>("test");
            var link3 = new Link<string>("other");
```

```
55
56              Assert.Equal(link1, link2);
57              Assert.NotEqual(link1, link3);
58          }
59
60          [Fact]
61          public static void LinkCombineTest()
62          {
63              var link1 = new Link<string>("first");
64              var link2 = new Link<string>("second");
65              var combined = link1.Combine(link2);
66
67              Assert.Null(combined.Id);
68              Assert.Equal(2, combined.Values?.Count);
69              Assert.Equal("first", combined.Values?[0].Id);
70              Assert.Equal("second", combined.Values?[1].Id);
71          }
72
73          [Fact]
74          public static void LinkEscapeReferenceSimpleTest()
75          {
76              Assert.Equal("test", Link<string>.EscapeReference("test"));
77              Assert.Equal("test123", Link<string>.EscapeReference("test123"));
78          }
79
80          [Fact]
81          public static void LinkEscapeReferenceWithSpecialCharactersTest()
82          {
83              Assert.Equal("'has space'", Link<string>.EscapeReference("has space"));
84              Assert.Equal("'has:colon'", Link<string>.EscapeReference("has:colon"));
85              Assert.Equal("'has(paren)'", Link<string>.EscapeReference("has(paren)"));
86              Assert.Equal("'has)paren'", Link<string>.EscapeReference("has)paren"));
87              Assert.Equal("\"has'quote\"", Link<string>.EscapeReference("has'quote"));
88              Assert.Equal("'has\"quote'", Link<string>.EscapeReference("has\"quote"));
89          }
90
91          [Fact]
92          public static void LinkSimplifyTest()
93          {
94              // Test simplify with empty values
95              var link1 = new Link<string>("test", null);
96              var simplified1 = link1.Simplify();
97              Assert.Equal(link1, simplified1);
98
99              // Test simplify with single value
100             var link2 = new Link<string>(null, new List<Link<string>> { new
    ↪  Link<string>("single", null) });
101             var simplified2 = link2.Simplify();
102             Assert.Equal("single", simplified2.Id);
103             Assert.Null(simplified2.Values);
104         }
105     }
106 }
```

1.37 ./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs

```
1  using System;
2  using System.Collections.Generic;
3  using Xunit;
4
5  namespace Link.Foundation.Links.Notation.Tests
6  {
7      public static class LinksGroupTests
8      {
9          [Fact]
10         public static void LinksGroupConstructorTest()
11         {
12             var element = new Link<string>("root", null);
13             var groups = new List<LinksGroup<string>>
14             {
15                 new LinksGroup<string>(new Link<string>("child1", null)),
16                 new LinksGroup<string>(new Link<string>("child2", null))
17             };
18             var group = new LinksGroup<string>(element, groups);
19
20             Assert.Equal(element, group.Link);
21             Assert.Equal(groups, group.Groups);
22         }
23
24         [Fact]
```

```
25          public static void LinksGroupToListFlattensStructureTest()
26          {
27              var root = new Link<string>("root", null);
28              var child1 = new Link<string>("child1", null);
29              var child2 = new Link<string>("child2", null);
30              var grandchild = new Link<string>("grandchild", null);
31
32              var childGroup = new LinksGroup<string>(child2, new List<LinksGroup<string>>
33              {
34                  new LinksGroup<string>(grandchild)
35              });
36
37              var group = new LinksGroup<string>(root, new List<LinksGroup<string>>
38              {
39                  new LinksGroup<string>(child1),
40                  childGroup
41              });
42
43              var list = group.ToLinksList();
44              // The C# implementation creates a hierarchical structure
45              // root, root.Combine(child1), root.Combine(child2),
    ↪  root.Combine(child2).Combine(grandchild)
46              Assert.Equal(4, list.Count);
47              Assert.Equal(root, list[0]);
48          }
49
50          [Fact]
51          public static void LinksGroupAppendToLinksListTest()
52          {
53              var element = new Link<string>("root", null);
54              var children = new List<LinksGroup<string>>
55              {
56                  new LinksGroup<string>(new Link<string>("child1", null)),
57                  new LinksGroup<string>(new Link<string>("child2", null))
58              };
59              var group = new LinksGroup<string>(element, children);
60
61              var list = new List<Link<string>>();
62              group.AppendToLinksList(list);
63
64              Assert.Equal(3, list.Count);
65              Assert.Equal(element, list[0]);
66          }
67
68          [Fact]
69          public static void LinksGroupToStringTest()
70          {
71              var element = new Link<string>("root", null);
72              var children = new List<LinksGroup<string>>
73              {
74                  new LinksGroup<string>(new Link<string>("child1", null)),
75                  new LinksGroup<string>(new Link<string>("child2", null))
76              };
77              var group = new LinksGroup<string>(element, children);
78
79              var str = group.ToString();
80              Assert.Contains("root", str);
81              Assert.Contains("child1", str);
82              Assert.Contains("child2", str);
83          }
84
85          [Fact]
86          public static void LinksGroupConstructorEquivalentTest()
87          {
88              // Test creating a LinksGroup structure in an equivalent way
89              var root = new Link<string>("root", null);
90              var children = new List<Link<string>>
91              {
92                  new Link<string>("child1", null),
93                  new Link<string>("child2", null)
94              };
95
96              // Create a group with an id
97              var groupElement = new Link<string>("group", null);
98              var childGroups = new List<LinksGroup<string>>
99              {
100                 new LinksGroup<string>(root),
101                 new LinksGroup<string>(children[0]),
102                 new LinksGroup<string>(children[1])
```

```
            };
            var group = new LinksGroup<string>(groupElement, childGroups);

            Assert.Equal("group", group.Link.Id);
            Assert.Equal(3, group.Groups!.Count);
            Assert.Equal(root, group.Groups[0].Link);
        }
    }
}
```

1.38 ./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MixedIndentationModesTests
    {
        [Fact]
        public static void HeroExampleMixedModesTest()
        {
            var input = @"empInfo
  employees:
    (
      name (James Kirk)
      age 40
    )
    (
      name (Jean-Luc Picard)
      age 45
    )
    (
      name (Wesley Crusher)
      age 27
    )";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("empInfo", formatted);
            Assert.Contains("employees", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
            Assert.Contains("Wesley Crusher", formatted);
        }

        [Fact]
        public static void HeroExampleAlternativeFormatTest()
        {
            var input = @"empInfo
  (
    employees:
      (
        name (James Kirk)
        age 40
      )
      (
        name (Jean-Luc Picard)
        age 45
      )
      (
        name (Wesley Crusher)
        age 27
      )
  )";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("empInfo", formatted);
            Assert.Contains("employees:", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
            Assert.Contains("Wesley Crusher", formatted);
        }

```

```
        [Fact]
        public static void HeroExampleEquivalenceTest()
        {
            var version1 = @"empInfo
employees:
  (
    name (James Kirk)
    age 40
  )
  (
    name (Jean-Luc Picard)
    age 45
  )
  (
    name (Wesley Crusher)
    age 27
  )";

            var version2 = @"empInfo
(
  employees:
    (
      name (James Kirk)
      age 40
    )
    (
      name (Jean-Luc Picard)
      age 45
    )
    (
      name (Wesley Crusher)
      age 27
    )
)";

            var parser = new Parser();
            var result1 = parser.Parse(version1);
            var result2 = parser.Parse(version2);

            Assert.NotEmpty(result1);
            Assert.NotEmpty(result2);

            var formatted1 = result1.Format();
            var formatted2 = result2.Format();

            Assert.Equal(formatted1, formatted2);
        }

        [Fact]
        public static void SetContextWithoutColonTest()
        {
            var input = @"empInfo
employees";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("empInfo", formatted);
            Assert.Contains("employees", formatted);
        }

        [Fact]
        public static void SequenceContextWithColonTest()
        {
            var input = @"employees:
James Kirk
Jean-Luc Picard
Wesley Crusher";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);
            var formatted = result.Format();
            Assert.Contains("employees:", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
```

```
            Assert.Contains("Wesley Crusher", formatted);
        }

        [Fact]
        public static void SequenceContextWithComplexValuesTest()
        {
            var input = @"employees:
  (
    name (James Kirk)
    age 40
  )
  (
    name (Jean-Luc Picard)
    age 45
  )";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);
            var formatted = result.Format();
            Assert.Contains("employees:", formatted);
            Assert.Contains("James Kirk", formatted);
            Assert.Contains("Jean-Luc Picard", formatted);
        }

        [Fact]
        public static void NestedSetAndSequenceContextsTest()
        {
            var input = @"company
  departments:
    engineering
    sales
  employees:
    (name John)
    (name Jane)";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("company", formatted);
            Assert.Contains("departments:", formatted);
            Assert.Contains("employees:", formatted);
        }

        [Fact]
        public static void DeeplyNestedMixedModesTest()
        {
            var input = @"root
  level1
    level2:
      value1
      value2
    level2b
      level3";

            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            var formatted = result.Format();
            Assert.Contains("root", formatted);
            Assert.Contains("level2:", formatted);
        }
    }
}
```

1.39 ./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MultiQuoteParserTests
    {
        // Helper to extract single reference ID
```

```
        private static string? GetSingleRefId(System.Collections.Generic.IList<Link<string>>
    ↪ result)
        {
            if (result.Count == 1 && result[0].Id == null && result[0].Values?.Count == 1)
            {
                return result[0].Values![0].Id;
            }
            return result.Count == 1 ? result[0].Id : null;
        }

        // ================================================================================
        // Backtick Quote Tests (Single Backtick)
        // ================================================================================

        [Fact]
        public static void TestBacktickQuotedReference()
        {
            var parser = new Parser();
            var result = parser.Parse("`backtick quoted`");
            Assert.Equal("backtick quoted", GetSingleRefId(result));
        }

        [Fact]
        public static void TestBacktickQuotedWithSpaces()
        {
            var parser = new Parser();
            var result = parser.Parse("`text with spaces`");
            Assert.Equal("text with spaces", GetSingleRefId(result));
        }

        [Fact]
        public static void TestBacktickQuotedMultiline()
        {
            var parser = new Parser();
            var result = parser.Parse("(`line1\nline2`)");
            Assert.Single(result);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values!);
            Assert.Equal("line1\nline2", result[0].Values![0].Id);
        }

        [Fact]
        public static void TestBacktickQuotedWithEscapedBacktick()
        {
            var parser = new Parser();
            var result = parser.Parse("`text with `` escaped backtick`");
            Assert.Equal("text with ` escaped backtick", GetSingleRefId(result));
        }

        // ================================================================================
        // Single Quote Tests (with escaping)
        // ================================================================================

        [Fact]
        public static void TestSingleQuoteWithEscapedSingleQuote()
        {
            var parser = new Parser();
            var result = parser.Parse("'text with '' escaped quote'");
            Assert.Equal("text with ' escaped quote", GetSingleRefId(result));
        }

        // ================================================================================
        // Double Quote Tests (with escaping)
        // ================================================================================

        [Fact]
        public static void TestDoubleQuoteWithEscapedDoubleQuote()
        {
            var parser = new Parser();
            var result = parser.Parse("\"text with \"\" escaped quote\"");
            Assert.Equal("text with \" escaped quote", GetSingleRefId(result));
        }

        // ================================================================================
        // Double Quotes (2 quote chars) Tests
        // ================================================================================

        [Fact]
```

```
    public static void TestDoubleDoubleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"double double quotes\"\"");
        Assert.Equal("double double quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleDoubleQuotesWithSingleQuoteInside()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"text with \" inside\"\"");
        Assert.Equal("text with \" inside", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleDoubleQuotesWithEscape()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"text with \"\"\"\" escaped double\"\"");
        Assert.Equal("text with \"\" escaped double", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleSingleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("''double single quotes''");
        Assert.Equal("double single quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleSingleQuotesWithSingleQuoteInside()
    {
        var parser = new Parser();
        var result = parser.Parse("''text with ' inside''");
        Assert.Equal("text with ' inside", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleSingleQuotesWithEscape()
    {
        var parser = new Parser();
        var result = parser.Parse("''text with '''' escaped single''");
        Assert.Equal("text with '' escaped single", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleBacktickQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("``double backtick quotes``");
        Assert.Equal("double backtick quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleBacktickQuotesWithBacktickInside()
    {
        var parser = new Parser();
        var result = parser.Parse("``text with ` inside``");
        Assert.Equal("text with ` inside", GetSingleRefId(result));
    }

    [Fact]
    public static void TestDoubleBacktickQuotesWithEscape()
    {
        var parser = new Parser();
        var result = parser.Parse("``text with ```` escaped backtick``");
        Assert.Equal("text with `` escaped backtick", GetSingleRefId(result));
    }

    // ========================================================================
    // Triple Quotes (3 quote chars) Tests
    // ========================================================================

    [Fact]
    public static void TestTripleDoubleQuotes()
    {
```

```
        var parser = new Parser();
        var result = parser.Parse("\"\"\"triple double quotes\"\"\"");
        Assert.Equal("triple double quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestTripleDoubleQuotesWithDoubleQuoteInside()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"\"text with \"\" inside\"\"\"");
        Assert.Equal("text with \"\" inside", GetSingleRefId(result));
    }

    [Fact]
    public static void TestTripleDoubleQuotesWithEscape()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"\"text with \"\"\"\"\"\" escaped triple\"\"\"");
        Assert.Equal("text with \"\"\" escaped triple", GetSingleRefId(result));
    }

    [Fact]
    public static void TestTripleSingleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("'''triple single quotes'''");
        Assert.Equal("triple single quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestTripleBacktickQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("```triple backtick quotes```");
        Assert.Equal("triple backtick quotes", GetSingleRefId(result));
    }

    // ========================================================================
    // Quadruple Quotes (4 quote chars) Tests
    // ========================================================================

    [Fact]
    public static void TestQuadrupleDoubleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"\"\"quadruple double quotes\"\"\"\"");
        Assert.Equal("quadruple double quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuadrupleSingleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("''''quadruple single quotes''''");
        Assert.Equal("quadruple single quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuadrupleBacktickQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("````quadruple backtick quotes````");
        Assert.Equal("quadruple backtick quotes", GetSingleRefId(result));
    }

    // ========================================================================
    // Quintuple Quotes (5 quote chars) Tests
    // ========================================================================

    [Fact]
    public static void TestQuintupleDoubleQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"\"\"\"quintuple double quotes\"\"\"\"\"");
        Assert.Equal("quintuple double quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuintupleSingleQuotes()
```

```
    {
        var parser = new Parser();
        var result = parser.Parse("'''''quintuple single quotes'''''");
        Assert.Equal("quintuple single quotes", GetSingleRefId(result));
    }

    [Fact]
    public static void TestQuintupleBacktickQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("`````quintuple backtick quotes`````");
        Assert.Equal("quintuple backtick quotes", GetSingleRefId(result));
    }

    // ================================================================================
    // Complex Scenarios Tests
    // ================================================================================

    [Fact]
    public static void TestMixedQuotesInLink()
    {
        var parser = new Parser();
        var result = parser.Parse("(\"double\" 'single' `backtick`)");
        Assert.Single(result);
        Assert.NotNull(result[0].Values);
        Assert.Equal(3, result[0].Values!.Count);
        Assert.Equal("double", result[0].Values![0].Id);
        Assert.Equal("single", result[0].Values![1].Id);
        Assert.Equal("backtick", result[0].Values![2].Id);
    }

    [Fact]
    public static void TestBacktickAsIdInLink()
    {
        var parser = new Parser();
        var result = parser.Parse("(`myId`: value1 value2)");
        Assert.Single(result);
        Assert.Equal("myId", result[0].Id);
        Assert.NotNull(result[0].Values);
        Assert.Equal(2, result[0].Values!.Count);
    }

    [Fact]
    public static void TestCodeBlockLikeContent()
    {
        var parser = new Parser();
        var result = parser.Parse("```const x = 1;```");
        Assert.Equal("const x = 1;", GetSingleRefId(result));
    }

    [Fact]
    public static void TestNestedQuotesInMarkdown()
    {
        var parser = new Parser();
        var result = parser.Parse("``Use `code` in markdown``");
        Assert.Equal("Use `code` in markdown", GetSingleRefId(result));
    }

    [Fact]
    public static void TestJsonStringWithQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"\"{ \"key\": \"value\"}\"\"");
        Assert.Equal("{ \"key\": \"value\"}", GetSingleRefId(result));
    }

    // ================================================================================
    // Edge Cases
    // ================================================================================

    [Fact]
    public static void TestWhitespacePreservedInQuotes()
    {
        var parser = new Parser();
        var result = parser.Parse("\"  spaces  \"");
        Assert.Equal("  spaces  ", GetSingleRefId(result));
    }

```

```
321         [Fact]
322         public static void TestMultilineInDoubleDoubleQuotes()
323         {
324             var parser = new Parser();
325             var result = parser.Parse("(\"\"line1\nline2\"\")");
326             Assert.Single(result);
327             Assert.NotNull(result[0].Values);
328             Assert.Single(result[0].Values!);
329             Assert.Equal("line1\nline2", result[0].Values![0].Id);
330         }
331 
332         // ================================================================================
333         // Unlimited Quotes (6+ quote chars) Tests
334         // ================================================================================
335 
336         [Fact]
337         public static void TestUnlimitedQuotes6()
338         {
339             // Test 6-quote strings
340             var parser = new Parser();
341             var result = parser.Parse("\"\"\"\"\"\"hello\"\"\"\"\"\"");
342             Assert.Equal("hello", GetSingleRefId(result));
343         }
344 
345         [Fact]
346         public static void TestUnlimitedQuotes10()
347         {
348             // Test 10-quote strings
349             var parser = new Parser();
350             var result = parser.Parse("\"\"\"\"\"\"\"\"\"\"very deeply
    ↪  quoted\"\"\"\"\"\"\"\"\"\"");
351             Assert.Equal("very deeply quoted", GetSingleRefId(result));
352         }
353 
354         [Fact]
355         public static void TestUnlimitedQuotes6WithInnerQuotes()
356         {
357             // Test 6-quote strings with inner 5-quote sequences
358             var parser = new Parser();
359             var result = parser.Parse("\"\"\"\"\"\"hello with \"\"\"\"\" five quotes
    ↪  inside\"\"\"\"\"\"");
360             Assert.Equal("hello with \"\"\"\" five quotes inside", GetSingleRefId(result));
361         }
362 
363         [Fact]
364         public static void TestUnlimitedSingleQuotes7()
365         {
366             // Test 7-quote single quote strings
367             var parser = new Parser();
368             var result = parser.Parse("'''''''seven single quotes'''''''");
369             Assert.Equal("seven single quotes", GetSingleRefId(result));
370         }
371 
372         [Fact]
373         public static void TestUnlimitedBackticks8()
374         {
375             // Test 8-quote backtick strings
376             var parser = new Parser();
377             var result = parser.Parse("````````eight backticks````````");
378             Assert.Equal("eight backticks", GetSingleRefId(result));
379         }
380     }
381 }
```

1.40 ./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs

```
1  using System;
2  using Xunit;
3 
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      public static class MultilineParserTests
7      {
8          [Fact]
9          public static void TwoLinksTest()
10         {
11             var source = @"(first: x y)
12 (second: a b)";
13             var parser = new Parser();
```

```
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void ParseAndStringifyTest()
        {
            var source = @"(papa (lovesMama: loves mama))
(son lovesMama)
(daughter lovesMama)
(all (love mama))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void ParseAndStringifyTest2()
        {
            var source = @"father (lovesMom: loves mom)
son lovesMom
daughter lovesMom
all (love mom)";
            var links = new Parser().Parse(source);
            var target = links.Format(lessParentheses: true);
            Assert.Equal(source, target);
        }

        [Fact]
        public static void ParseAndStringifyWithLessParenthesesTest()
        {
            var source = @"lovesMama: loves mama
papa lovesMama
son lovesMama
daughter lovesMama
all (love mama)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format(lessParentheses: true);
            Assert.Equal(source, target);
        }

        [Fact]
        public static void DuplicateIdentifiersTest()
        {
            var source = @"(a: a b)
(a: b c)";
            var target = @"(a: a b)
(a: b c)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void ComplexStructureTest()
        {
            var input = @"(Type: Type Type)
  Number
  String
  Array
  Value
    (property: name type)
    (method: name params return)";

            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MixedFormatsTest()
        {
            // Mix of single-line and multi-line formats
            var input = @"id1: value1
(id2: value2 value3)
simple ref
```

```
(complex:
  nested1
  nested2
)";

            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultilineWithIdTest()
        {
            // Test multi-line link with id
            var input = "(id: value1 value2)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultipleTopLevelElementsTest()
        {
            // Test multiple top-level elements
            var input = "(elem1: val1)\n(elem2: val2)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void MultilineSimpleLinksTest()
        {
            var input = "(1: 1 1)\n(2: 2 2)";
            var parser = new Parser();
            var parsed = parser.Parse(input);
            Assert.NotEmpty(parsed);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.Contains("(1: 1 1)", output);
            Assert.Contains("(2: 2 2)", output);

            // Validate alternate formatting matches input
            var outputAlternate = parsed.Format();
            Assert.Equal(input, outputAlternate);
        }

        [Fact]
        public static void IndentedChildrenTest()
        {
            var input = "parent\n  child1\n  child2";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // The parsed structure should have parent with children
            Assert.NotEmpty(parsed);
        }
    }
}
```

1.41 ./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class MultilineQuotedStringTests
    {
        [Fact]
        public static void TestMultilineDoubleQuotedReference()
        {
            var input = @"(
  ""long
string literal representing
the reference""

  'another
long string literal
as another reference'
)";
            var parser = new Parser();
            var result = parser.Parse(input);
```

```

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Null(link.Id);
            Assert.NotNull(link.Values);
            Assert.Equal(2, link.Values.Count);

            // The two references sit on separate lines inside the parentheses, so the
            // nested context turns each of them into its own link.
            Assert.Equal(@"long
string literal representing
the reference", link.Values![0].Values![0].Id);

            Assert.Equal(@"another
long string literal
as another reference", link.Values![1].Values![0].Id);
        }

        [Fact]
        public static void TestSimpleMultilineDoubleQuoted()
        {
            var input = @"(""line1
line2"")";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Null(link.Id);
            Assert.NotNull(link.Values);
            Assert.Single(link.Values);
            Assert.Equal("line1\nline2", link.Values[0].Id);
        }

        [Fact]
        public static void TestSimpleMultilineSingleQuoted()
        {
            var input = @"('line1
line2')";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Null(link.Id);
            Assert.NotNull(link.Values);
            Assert.Single(link.Values);
            Assert.Equal("line1\nline2", link.Values[0].Id);
        }

        [Fact]
        public static void TestMultilineQuotedAsId()
        {
            var input = @"(""multi
line
id"": value1 value2)";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.Single(result);

            var link = result[0];
            Assert.Equal("multi\nline\nid", link.Id);
            Assert.NotNull(link.Values);
            Assert.Equal(2, link.Values.Count);
        }
    }
}
```

1.42 ./Link.Foundation.Links.Notation.Tests/NestedIndentationTests.cs

```
using Xunit;

```

```
namespace Link.Foundation.Links.Notation.Tests
{
    /// <summary>
    /// Tests for indentation inside parentheses.
    ///
    /// See https://github.com/link-foundation/links-notation/issues/282
    ///
    /// Indentation is structural at the root, so it must be structural inside parentheses too:
    /// a parenthesized group opens a nested context that starts fresh at indentation level zero
    /// and follows exactly the root's rules.
    /// </summary>
    public class NestedIndentationTests
    {
        private static string FormatSource(string source) => new Parser().Parse(source).Format();

        private static void AssertFormat(string source, string expected) =>
    Assert.Equal(expected, FormatSource(source));

        [Fact]
        public void ParenthesesReproduceRootIndentation()
        {
            AssertFormat("a\n  b\nc\n  d", "(a)\n((a) (b))\n(c)\n((c) (d))");

            // The same lines inside parentheses keep the same structure, nested under
            // the link the group belongs to.
            AssertFormat("array (\n  a\n    b\n  c\n    d\n)", "(array ((a) ((a) (b)) (c) ((c)
    (d))))");
        }

        [Fact]
        public void ParenthesesKeepRecordBoundaries()
        {
            var source = "value (\n  id \"1\"\n  label \"one\"\n)";
            AssertFormat(source, "(value ((id 1) (label one)))");

            var links = new Parser().Parse(source);
            Assert.Single(links);

            var group = links[0].Values![1];
            Assert.Null(group.Id);
            Assert.Equal(2, group.Values!.Count);
            Assert.Equal("id", group.Values[0].Values![0].Id);
            Assert.Equal("1", group.Values[0].Values![1].Id);
            Assert.Equal("label", group.Values[1].Values![0].Id);
            Assert.Equal("one", group.Values[1].Values![1].Id);
        }

        [Fact]
        public void ParenthesesKeepSeveralRecordsSeparate()
        {
            AssertFormat(
                "value (\n  (id \"1\" label \"one\")\n  (id \"2\" label \"two\")\n)",
                "(value ((id 1 label one) (id 2 label two)))");
        }

        [Fact]
        public void ParenthesesNestDeeply()
        {
            AssertFormat("outer (\n  inner (\n    x 1\n    y 2\n  )\n  z 3\n)", "(outer ((inner
    ((x 1) (y 2))) (z 3)))");
        }

        [Fact]
        public void SingleLineParenthesesAreUnchanged()
        {
            AssertFormat("(a b c)", "(a b c)");
            AssertFormat("(1: 2 3)", "(1: 2 3)");
            AssertFormat("(a: b c)", "(a: b c)");
            AssertFormat("((a b))", "((a b))");
            AssertFormat("(a)", "(a)");
            AssertFormat("()", "()");
        }

        [Fact]
        public void ParenthesesWithIndentedIdSyntax()
        {
            AssertFormat("(\n  a:\n    b\n    c\n)", "(a: b c)");
        }
```

```

        [Fact]
        public void EmployeeRecordsKeepTheirFields()
        {
            var source = "empInfo\n  employees:\n    (\n      name (James Kirk)\n      age 40\n    )\n"
                + "    (\n      name (Jean-Luc Picard)\n      age 45\n    )";
            var expected = "(empInfo)\n((empInfo) (employees: ((name (James Kirk)) (age 40))"
                + " ((name (Jean-Luc Picard)) (age 45))))";
            AssertFormat(source, expected);
        }
    }
}
```

1.43 ./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs

```
using System;
using System.Linq;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class NestedParserTests
    {
        [Fact]
        public static void SignificantWhitespaceTest()
        {
            var source = @"
users
    user1
        id
            43
        name
            first
                John
            last
                Williams
        location
            New York
        age
            23
    user2
        id
            56
        name
            first
                Igor
            middle
                Petrovich
            last
                Ivanov
        location
            Moscow
        age
            20";
            var target = @"(users)
((users) (user1))
(((users) (user1)) (id))
((((users) (user1)) (id)) (43))
(((users) (user1)) (name))
((((users) (user1)) (name)) (first))
(((((users) (user1)) (name)) (first)) (John))
((((users) (user1)) (name)) (last))
(((((users) (user1)) (name)) (last)) (Williams))
(((users) (user1)) (location))
((((users) (user1)) (location)) (New York))
(((users) (user1)) (age))
((((users) (user1)) (age)) (23))
((users) (user2))
(((users) (user2)) (id))
((((users) (user2)) (id)) (56))
(((users) (user2)) (name))
((((users) (user2)) (name)) (first))
(((((users) (user2)) (name)) (first)) (Igor))
((((users) (user2)) (name)) (middle))
(((((users) (user2)) (name)) (middle)) (Petrovich))
((((users) (user2)) (name)) (last))
(((((users) (user2)) (name)) (last)) (Ivanov))
(((users) (user2)) (location))
((((users) (user2)) (location)) (Moscow))
(((users) (user2)) (age))
((((users) (user2)) (age)) (20))";
```

```
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void SimpleSignificantWhitespaceTest()
        {
            var source = @"a
    b
    c";
            var target = @"(a)
((a) (b))
((a) (c))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void TwoSpacesSizedWhitespaceTest()
        {
            var source = @"
users
  user1";
            var target = @"(users)
((users) (user1))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void ParseNestedStructureWithIndentationTest()
        {
            var source = @"parent
  child1
  child2";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            // Should create 3 links: (parent), (parent child1), (parent child2)
            Assert.Equal(3, links.Count);
        }

        [Fact]
        public static void IndentationConsistencyTest()
        {
            // Test that indentation must be consistent
            var input = @"parent
  child1
   child2"; // Inconsistent indentation
            var result = new Parser().Parse(input);
            // This should parse but child2 won't be a child of parent due to different
↪  indentation
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void IndentationBasedChildrenTest()
        {
            var input = @"parent
  child1
  child2
    grandchild";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotNull(result);
            // Expected flattened links:
            // (parent), (parent child1), (parent child2), ((parent child2) grandchild)
            Assert.Equal(4, result.Count);
        }

        [Fact]
        public static void ComplexIndentationTest()
        {
```

```
            var input = @"root
  level1a
    level2a
    level2b
  level1b
    level2c";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotNull(result);
            // Expected flattened links:
            // (root), (root level1a), ((root level1a) level2a), ((root level1a) level2b),
            // (root level1b), ((root level1b) level2c)
            Assert.Equal(6, result.Count);
        }

        [Fact]
        public static void NestedLinksTest()
        {
            var input = "(1: (2: (3: 3)))";
            var parser = new Parser();
            var parsed = parser.Parse(input);
            Assert.NotEmpty(parsed);

            // Validate regular formatting
            var output = parsed.Format();
            Assert.NotEmpty(output);

            // Validate that the structure is properly nested
            Assert.Single(parsed);
        }

        [Fact]
        public static void IndentationParserTest()
        {
            var input = "parent\n  child1\n  child2";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotEmpty(result);
            // Should have parent link
            var parentLink = result.FirstOrDefault(l => l.Id == "parent");
        }

        [Fact]
        public static void NestedIndentationParserTest()
        {
            var input = "parent\n  child\n    grandchild";
            var parser = new Parser();
            var result = parser.Parse(input);
            Assert.NotEmpty(result);
            // Should create nested structure with proper hierarchy
            Assert.True(result.Count >= 1);
        }

        [Fact]
        public static void ThreeLevelNestingRoundtripTest()
        {
            var input = "(1: (2: (3: 3)))";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate round-trip
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void DeepNestedStructureRoundtripTest()
        {
            var input = "(a: (b: (c: (d: d))))";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate round-trip
            var output = parsed.Format();
            Assert.Equal(input, output);
        }

        [Fact]
        public static void MultipleNestedLinksRoundtripTest()
```

```
        {
            var input = "(parent: (child1: value1) (child2: value2))";
            var parser = new Parser();
            var parsed = parser.Parse(input);

            // Validate round-trip
            var output = parsed.Format();
            Assert.Equal(input, output);
        }
    }
}                                           -
```

1.44 ./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class NestedSelfReferenceTests
    {
        [Fact]
        public static void NestedSelfReferencedObjectInPairValueTest()
        {
            // Test case from PARSER BUG.md
            // This should parse a dict with two pairs, where the second pair's value
            // is itself a self-referenced dict definition (obj 1: dict ...)
            var notation = "(obj 0: dict ((str bmFtZQ==) (str ZGljdDE=)) ((str b3RoZXI=) (obj 1:
↪  dict ((str bmFtZQ==) (str ZGljdDI=)) ((str b3RoZXI=) obj 0))))";

            var parser = new Parser();
            var links = parser.Parse(notation);

            // Should parse exactly one top-level link
            Assert.Single(links);

            var link = links[0];

            // Top-level link should have ID "obj 0"
            Assert.Equal("obj 0", link.Id);

            // Should have: type marker + 2 pairs = 3 values
            Assert.Equal(3, link.Values!.Count);

            // First value is the type marker "dict"
            Assert.Equal("dict", link.Values[0].Id);

            // Second and third values are the two pairs
            var pair1 = link.Values[1];
            var pair2 = link.Values[2];

            // Pair 1: ((str bmFtZQ==) (str ZGljdDE=))
            // This is a parenthesized expression containing two sub-expressions
            Assert.Null(pair1.Id);
            Assert.Equal(2, pair1.Values!.Count);

            // First element of pair1: (str bmFtZQ==)
            Assert.Null(pair1.Values[0].Id);
            Assert.Equal(2, pair1.Values![0].Values!.Count);
            Assert.Equal("str", pair1.Values![0].Values![0].Id);
            Assert.Equal("bmFtZQ==", pair1.Values![0].Values![1].Id);

            // Second element of pair1: (str ZGljdDE=)
            Assert.Null(pair1.Values[1].Id);
            Assert.Equal(2, pair1.Values![1].Values!.Count);
            Assert.Equal("str", pair1.Values![1].Values![0].Id);
            Assert.Equal("ZGljdDE=", pair1.Values![1].Values![1].Id);

            // Pair 2: ((str b3RoZXI=) (obj 1: dict ...))
            // This is a parenthesized expression containing two sub-expressions
            Assert.Null(pair2.Id);
            Assert.Equal(2, pair2.Values!.Count);

            // First element of pair2: (str b3RoZXI=)
            Assert.Null(pair2.Values[0].Id);
            Assert.Equal(2, pair2.Values![0].Values!.Count);
            Assert.Equal("str", pair2.Values![0].Values![0].Id);
            Assert.Equal("b3RoZXI=", pair2.Values![0].Values![1].Id);

```

```
            // Second element of pair2: (obj 1: dict ((str bmFtZQ==) (str ZGljdDI=)) ((str b3RoZXI=) obj 0))
            // THIS IS THE KEY TEST - obj 1 should have its ID preserved
            var obj1 = pair2.Values[1];
            Assert.Equal("obj 1", obj1.Id);
            Assert.NotNull(obj1.Values);
            Assert.Equal(3, obj1.Values.Count); // type marker + 2 pairs

            // obj 1's type marker
            Assert.Equal("dict", obj1.Values[0].Id);

            // obj 1's first pair: ((str bmFtZQ==) (str ZGljdDI=))
            var obj1Pair1 = obj1.Values[1];
            Assert.Equal(2, obj1Pair1.Values!.Count);
            Assert.Null(obj1Pair1.Values[0].Id);
            Assert.Equal(2, obj1Pair1.Values![0].Values!.Count);
            Assert.Equal("str", obj1Pair1.Values![0].Values![0].Id);
            Assert.Equal("bmFtZQ==", obj1Pair1.Values![0].Values![1].Id);
            Assert.Null(obj1Pair1.Values[1].Id);
            Assert.Equal(2, obj1Pair1.Values![1].Values!.Count);
            Assert.Equal("str", obj1Pair1.Values![1].Values![0].Id);
            Assert.Equal("ZGljdDI=", obj1Pair1.Values![1].Values![1].Id);

            // obj 1's second pair: ((str b3RoZXI=) obj 0) - reference back to obj 0
            var obj1Pair2 = obj1.Values[2];
            Assert.Equal(2, obj1Pair2.Values!.Count);
            Assert.Null(obj1Pair2.Values[0].Id);
            Assert.Equal(2, obj1Pair2.Values![0].Values!.Count);
            Assert.Equal("str", obj1Pair2.Values![0].Values![0].Id);
            Assert.Equal("b3RoZXI=", obj1Pair2.Values![0].Values![1].Id);
            Assert.Equal("obj 0", obj1Pair2.Values[1].Id);
            Assert.Null(obj1Pair2.Values[1].Values); // Just a reference, no nested values
        }

        [Fact]
        public static void SelfReferenceAsDirectChildWorksCorrectlyTest()
        {
            // This pattern should work (and did work before)
            var notation = "(obj 0: list (int 1) (int 2) (obj 1: list (int 3) (int 4) obj 0))";

            var parser = new Parser();
            var links = parser.Parse(notation);

            Assert.Single(links);
            Assert.Equal("obj 0", links[0].Id);
            Assert.Equal(4, links[0].Values!.Count); // list + 1 + 2 + obj 1

            // The fourth value should be obj 1 with a self-reference
            var obj1 = links[0].Values![3];
            Assert.Equal("obj 1", obj1.Id);
            Assert.Equal(4, obj1.Values!.Count); // list + 3 + 4 + obj 0
            Assert.Equal("obj 0", obj1.Values[3].Id); // Reference to obj 0
        }
    }
}
```

1.45 ./Link.Foundation.Links.Notation.Tests/NestingDepthTests.cs

```
using System;
using System.Linq;
using System.Threading;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    /// <summary>
    /// Reading nested groups must take time that grows with the size of the document,
    /// not with two to the power of its nesting depth
    /// (<see href="https://github.com/link-foundation/links-notation/issues/314">#314</see>).
    /// Before the fix, every level of nesting doubled the work: a document nested six
    /// levels deep took seconds. Each case runs on its own thread and fails when it has
    /// not finished within the budget, so a regression fails instead of hanging.
    /// </summary>
    public class NestingDepthTests
    {
        /// Generous enough for a debug build on a slow machine, and far below what the
        /// exponential reading took at the depths used here.
        private static readonly TimeSpan Budget = TimeSpan.FromSeconds(5);

        /// The depth used for the shapes that used to take exponential time.
```

```
        private const int Deep = 32;

        /// A depth that shows the reading stays linear well past the shallow cases.
        private const int VeryDeep = 256;

        private static string Closed(int depth) => new string('(', depth) + "a" + new string(')', depth);

        private static string ValueAfter(int depth) => new string('(', depth) + "a" + string.Concat(Enumerable.Repeat(") b", depth));

        private static string Unclosed(int depth) => new string('(', depth) + "a";

        /// Unclosed groups on lines that are each indented one space deeper.
        private static string IndentedUnclosed(int depth) =>
            string.Join("\n", Enumerable.Range(0, depth).Select(level => new string(' ', level) + "(a"));

        /// Formats the parsed document, or names the exception parsing raised.
        private static string ReadWithinBudget(string what, string source)
        {
            string? outcome = null;
            var reader = new Thread(() =>
            {
                try
                {
                    // The documents nest deeper than the default limit on purpose; only the
                    // time they take is measured here.
                    outcome = new Parser(true, int.MaxValue).Parse(source).Format();
                }
                catch (Exception error)
                {
                    outcome = error.GetType().Name;
                }
            }, 512 * 1024 * 1024)
            {
                // A reader that runs past the budget must not keep the test run alive.
                IsBackground = true,
            };
            reader.Start();
            Assert.True(reader.Join(Budget), $"{what} did not finish within {Budget}");
            return outcome!;
        }

        [Fact]
        public void ClosedGroupsReadInLinearTime()
        {
            var source = Closed(Deep);
            Assert.Equal(source, ReadWithinBudget("closed groups", source));
        }

        [Fact]
        public void ValuesAfterGroupsReadInLinearTime()
        {
            var source = ValueAfter(Deep);
            Assert.Equal($"({source})", ReadWithinBudget("values after groups", source));
        }

        [Fact]
        public void UnclosedGroupsFailInLinearTime()
        {
            Assert.Equal(nameof(ParseException), ReadWithinBudget("unclosed groups", Unclosed(Deep)));
        }

        [Fact]
        public void UnclosedGroupsOnIndentedLinesFailInLinearTime()
        {
            Assert.Equal(nameof(ParseException), ReadWithinBudget("unclosed indented groups", IndentedUnclosed(Deep)));
        }

        [Fact]
        public void ValuesAfterVeryDeepGroupsReadInLinearTime()
        {
            var source = ValueAfter(VeryDeep);
            Assert.Equal($"({source})", ReadWithinBudget("very deep values after groups", source));
```

```
        }

        [Fact]
        public void GroupFollowedByValuesKeepsItsStructure()
        {
            var cases = new[]
            {
                new[] { "(a) b", "((a) b)" },
                new[] { "(a) (b) c", "((a) (b) c)" },
                new[] { "((a) b) c", "(((a) b) c)" },
                new[] { "(a: b) c", "((a: b) c)" },
                new[] { "(a)\n(b) c", "(a)\n((b) c)" },
                new[] { "x\n  (a) b\n  (c)", "(x)\n((x) ((a) b))\n((x) (c))" },
            };

            var parser = new Parser();
            foreach (var testCase in cases)
            {
                Assert.Equal(testCase[1], parser.Parse(testCase[0]).Format());
            }
        }
    }
}
```

1.46 ./Link.Foundation.Links.Notation.Tests/NestingLimitTests.cs

```
using System;
using System.Linq;
using System.Threading;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    /// <summary>
    /// Links nested too deeply are refused with an error rather than recursed into until
    /// the stack overflows, which ends the process where no catch block sees it
    /// (https://github.com/link-foundation/links-notation/issues/315).
    /// </summary>
    /// <remarks>
    /// Every parenthesized group and every indentation level is one level, and the lines of
    /// a document start at level 0. The positions asserted here are the ones the Rust and
    /// JavaScript ports report for the same input.
    /// </remarks>
    public static class NestingLimitTests
    {
        private static string Parens(int depth) => new string('(', depth) + "a" + new
↪   string(')', depth);

        private static string Values(int depth) =>
            string.Concat(Enumerable.Repeat("(a ", depth)) + "b" + new string(')', depth);

        private static string Indentation(int depth) =>
            string.Concat(Enumerable.Range(0, depth + 1).Select(level => new string(' ', level)
↪   + "a\n"));

        private static ParseException TooDeep(string document, int maxDepth)
        {
            var error = Assert.Throws<ParseException>(() => new Parser(true,
↪   maxDepth).Parse(document));
            Assert.Equal(maxDepth, error.MaxDepth);
            return error;
        }

        private static bool Accepted(string document, int maxDepth) =>
            new Parser(true, maxDepth).Parse(document).Count > 0;

        [Fact]
        public static void DefaultLimitIsSharedByEveryImplementationTest()
        {
            Assert.Equal(64, Parser.DefaultMaxDepth);
            Assert.Equal(Parser.DefaultMaxDepth, new Parser().MaxDepth);
            Assert.Equal(Parser.DefaultMaxDepth, new Parser(comments: false).MaxDepth);
            Assert.Equal(3, new Parser { MaxDepth = 3 }.MaxDepth);
        }

        [Fact]
        public static void NegativeLimitIsRejectedTest()
        {
            Assert.Throws<ArgumentOutOfRangeException>(() => new Parser(true, -1));
            Assert.Throws<ArgumentOutOfRangeException>(() => new Parser { MaxDepth = -1 });
```

```
52          }
53
54          [Fact]
55          public static void ParenthesesUpToTheLimitAreAcceptedTest()
56          {
57              Assert.True(Accepted(Parens(3), 3));
58              Assert.True(Accepted(Values(3), 3));
59              Assert.NotEmpty(new Parser().Parse(Parens(Parser.DefaultMaxDepth)));
60              Assert.NotEmpty(new Parser().Parse(Values(Parser.DefaultMaxDepth)));
61          }
62
63          [Fact]
64          public static void EveryGroupAndEveryIndentationLevelIsOneLevelTest()
65          {
66              Assert.True(Accepted("(a)", 1));
67              Assert.True(Accepted("a b c", 0));
68              TooDeep("(a)", 0);
69              TooDeep("((a))", 1);
70              TooDeep("(a (b))", 1);
71              TooDeep("a\n  (b)\n", 1);
72          }
73
74          [Fact]
75          public static void ParenthesesPastTheLimitAreRefusedAtTheGroupThatIsTooDeepTest()
76          {
77              var error = TooDeep(Parens(4), 3);
78
79              Assert.Equal((1, 4, 3), (error.Line, error.Column, error.Offset));
80              Assert.Equal("line 1, column 4: nesting depth exceeds the maximum of 3",
    ↪ error.Summary);
81              Assert.Equal("1 | ((((a))))\n  |    ^", error.Snippet);
82              Assert.Equal(
83                  "Nesting too deep at line 1, column 4: nesting depth exceeds the maximum of 3\n"
    ↪ +
84                  "1 | ((((a))))\n" +
85                  "  |    ^",
86                  error.Message);
87          }
88
89          [Fact]
90          public static void GroupsInValuePositionCountLikeAnyOtherGroupTest()
91          {
92              var error = TooDeep(Values(4), 3);
93
94              Assert.Equal((1, 10), (error.Line, error.Column));
95          }
96
97          [Fact]
98          public static void IndentationUpToTheLimitIsAcceptedTest()
99          {
100             Assert.True(Accepted(Indentation(3), 3));
101             Assert.NotEmpty(new Parser().Parse(Indentation(Parser.DefaultMaxDepth)));
102         }
103
104         [Fact]
105         public static void IndentationPastTheLimitIsRefusedAtTheLineThatIsTooDeepTest()
106         {
107             var error = TooDeep(Indentation(4), 3);
108
109             Assert.Equal((5, 5), (error.Line, error.Column));
110             Assert.Equal("    a", error.LineText);
111         }
112
113         [Fact]
114         public static void GroupsAndIndentationAddUpTest()
115         {
116             // `(b)` on the line indented once is at level 2.
117             Assert.True(Accepted("a\n  (b)\n", 2));
118             var error = TooDeep("a\n  (b)\n", 1);
119
120             Assert.Equal((2, 3), (error.Line, error.Column));
121         }
122
123         [Fact]
124         public static void IndentationInsideAGroupCountsOnTopOfTheGroupTest()
125         {
126             // `b` is inside one group and indented once within it.
127             Assert.True(Accepted("(a\n  b)\n", 2));
128             var error = TooDeep("(a\n  b)\n", 1);
```

```
129
130             Assert.Equal((2, 3), (error.Line, error.Column));
131         }
132
133         [Fact]
134         public static void TrailingSpacesOnADeepLineAreNotADeeperLineTest()
135         {
136             Assert.True(Accepted("a\n  b\n    c   \n", 2));
137             Assert.True(Accepted("a\n  b\n    c\n      ", 2));
138         }
139
140         [Fact]
141         public static void SyntaxErrorIsNotMistakenForNestingThatIsTooDeepTest()
142         {
143             var error = Assert.Throws<ParseException>(() => new Parser().Parse("a: b: c"));
144
145             Assert.Null(error.MaxDepth);
146             Assert.StartsWith("Syntax error at ", error.Message);
147         }
148
149         [Fact]
150         public static void RefusesADocumentFarPastTheLimitWithoutOverflowingTheStackTest()
151         {
152             // Before the limit existed each of these overflowed the stack, which ends the
153             // process. The documents are parsed on a 1 MiB stack, smaller than any thread
154             // .NET starts by default.
155             string?[] messages = new string?[3];
156             Exception? unexpected = null;
157             var thread = new Thread(() =>
158             {
159                 try
160                 {
161                     var documents = new[] { Parens(100 000), Values(100 000), Indentation(2 000)
    ↪  };
162                     for (var index = 0; index < documents.Length; index++)
163                     {
164                         var error = Assert.Throws<ParseException>(() => new
    ↪  Parser().Parse(documents[index]));
165                         Assert.Equal(Parser.DefaultMaxDepth, error.MaxDepth);
166                         messages[index] = error.Message;
167                     }
168                 }
169                 catch (Exception error)
170                 {
171                     unexpected = error;
172                 }
173             }, 1 << 20);
174             thread.Start();
175             thread.Join();
176
177             Assert.Null(unexpected);
178             Assert.All(messages, message => Assert.StartsWith("Nesting too deep at ", message));
179         }
180
181         [Fact]
182         public static void StreamParserReportsWhereTheNestingIsTooDeepTest()
183         {
184             var stream = new StreamParser(new Parser(true, 1));
185             stream.Write("a\nb ((c))\n");
186
187             var error = Assert.Throws<StreamParseException>(() => stream.Finish());
188
189             Assert.Equal(1, error.ParseError.MaxDepth);
190             Assert.Equal((2, 4, 5), (error.Line, error.Column, error.Offset));
191         }
192     }
193 }
```

1.47 ./Link.Foundation.Links.Notation.Tests/ParseErrorPositionTests.cs

```
1  using System;
2  using Xunit;
3
4  namespace Link.Foundation.Links.Notation.Tests
5  {
6      /// <summary>
7      /// A parse error has to say where the document stopped making sense.
8      /// </summary>
9      /// <remarks>
10     /// The positions asserted here are the ones the JavaScript and Rust ports report for
```

```
    /// the same input, so the implementations can be held to the same contract
    /// (https://github.com/link-foundation/links-notation/issues/302).
    /// </remarks>
    public static class ParseErrorPositionTests
    {
        private static ParseException SyntaxError(string document) =>
            Assert.Throws<ParseException>(() => new Parser().Parse(document));

        [Fact]
        public static void ReportsTheLineAndColumnOfTheDefectTest()
        {
            // The example from the issue: the defect is the second colon on line 2,
            // and the two lines after it are fine.
            var error = SyntaxError("ci gate x\nstage: rust: nextest\nnext stage\n  clippy");

            Assert.Equal(2, error.Line);
            Assert.Equal(12, error.Column);
            Assert.Equal(':', error.Found);
        }

        [Fact]
        public static void OffsetAgreesWithTheOtherImplementationsTest()
        {
            // JavaScript and Rust report offset 21, line 2, column 12 for this document.
            var error = SyntaxError("ci gate x\nstage: rust: nextest\n");

            Assert.Equal(21, error.Offset);
            Assert.Equal(2, error.Line);
            Assert.Equal(12, error.Column);
        }

        [Fact]
        public static void ReportsTheLineALateDefectIsOnTest()
        {
            var error = SyntaxError("a\nb\nc\nd\ne: f: g\nh\n");

            Assert.Equal(5, error.Line);
            Assert.Equal(5, error.Column);
            Assert.Equal("e: f: g", error.LineText);
        }

        [Fact]
        public static void ReportsTheEndOfTheDocumentWhenAGroupIsNeverClosedTest()
        {
            var error = SyntaxError("a (b\n");

            Assert.Equal(5, error.Offset);
            Assert.Equal(2, error.Line);
            Assert.Equal(1, error.Column);
            Assert.Null(error.Found);
            Assert.Contains("end of input", error.Message);
        }

        [Fact]
        public static void ReportsAnUnmatchedClosingParenthesisTest()
        {
            var error = SyntaxError("a b)\n");

            Assert.Equal(3, error.Offset);
            Assert.Equal(1, error.Line);
            Assert.Equal(4, error.Column);
            Assert.Equal(')', error.Found);
        }

        [Fact]
        public static void MessagePointsACaretAtTheOffendingCharacterTest()
        {
            var error = SyntaxError("ci gate x\nstage: rust: nextest\n");

            Assert.Equal("line 2, column 12: unexpected \":\"", error.Summary);
            Assert.Equal("2 | stage: rust: nextest\n  |            ^", error.Snippet);
            Assert.Equal(
                "Syntax error at line 2, column 12: unexpected \":\"\n2 | stage: rust: nextest\n  |            ^",
                error.Message);
        }

        [Fact]
        public static void MessageQuotesOneLineRatherThanTheRestOfTheDocumentTest()
```

```
        {
            var document = "ci gate x\nstage: rust: nextest\n"
                + string.Concat(System.Linq.Enumerable.Repeat("trailing line\n", 500));

            var error = SyntaxError(document);

            Assert.Contains("line 2, column 12", error.Message);
            Assert.DoesNotContain("trailing line", error.Message);
            Assert.True(error.Message.Length < 200, $"message is {error.Message.Length}
↪ characters");
        }

        [Fact]
        public static void MessageOfALongLineStaysAMessageTest()
        {
            var document = new string('a', 400) + ": " + new string('b', 400) + ": c";

            var error = SyntaxError(document);

            Assert.Equal(1, error.Line);
            Assert.Equal(803, error.Column);
            Assert.Contains("...", error.Message);
            Assert.True(error.Message.Length < 300, $"message is {error.Message.Length}
↪ characters");
        }

        [Fact]
        public static void KeepsCatchingCodeThatExpectsAFormatExceptionWorkingTest()
        {
            // The generated parser used to raise a bare FormatException; code that catches
            // it keeps working, and now reads a position off the exception it catches.
            var error = Assert.Throws<ParseException>(() => new Parser().Parse("a: b: c"));

            Assert.IsAssignableFrom<FormatException>(error);
            Assert.Equal(4, error.Offset);
        }

        [Fact]
        public static void DoesNotMentionTheGrammarInternalsTest()
        {
            var error = SyntaxError("a: b: c");

            Assert.DoesNotContain("Failed to parse", error.Message);
            Assert.DoesNotContain("document", error.Message);
        }

        [Fact]
        public static void ADocumentThatParsesReportsNothingTest()
        {
            var links = new Parser().Parse("a: b\n  c: d\n");

            Assert.Equal(2, links.Count);
        }
    }
}
```

1.48 ./Link.Foundation.Links.Notation.Tests/ReferenceFidelityTests.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using Link.Foundation.Links.Notation.Binary;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests;

public static class ReferenceFidelityTests
{
    private static IEnumerable<string> Fixtures() =>
        File.ReadLines(Path.Combine(AppContext.BaseDirectory, "Protocol",
↪ "reference-literals.txt"))
            .Where(line => !line.StartsWith('#'))
            .Select(line => line == "-" ? "" :
↪ Encoding.UTF8.GetString(Convert.FromHexString(line)));

    [Fact]
    public static void NativeReferenceFidelity()
    {
```

```
        foreach (var reference in Fixtures())
        {
            var original = new[] { new Link<string>(reference, new[] { new Link<string>(reference), new Link<string>("nested", new[] { new Link<string>(reference) }) }) };
            Assert.Equal(original, new Parser().Parse(original[0].ToString()));
            Assert.Equal(original, new Parser().Parse(original.Format(true)));
            Assert.Equal(original, new Parser().Parse(original.Format(new FormatConfig { MaxInlineRefs = 1, PreferInline = false })));
        }
    }

    [Fact(Timeout = 30000)]
    public static void ReferenceLiteralsAtEveryChunkSplit()
    {
        foreach (var reference in Fixtures())
        {
            Assert.Equal(reference, ReferenceLiteral.Decode(ReferenceLiteral.Encode(reference)));
            foreach (var literal in new[] { Link<string>.EscapeReference(reference), LinoFormat.FormatReference(reference), ReferenceLiteral.Encode(reference) })
            {
                var text = $"(root: {literal})\n({literal}: fixture)";
                var expected = new[] { new Link<string>("root", new[] { new Link<string>(reference) }), new Link<string>(reference, new[] { new Link<string>("fixture") }) };
                Assert.Equal(expected, new Parser().Parse(text));
                for (var split = 0; split <= text.Length; split++)
                {
                    TestContext.Current.CancellationToken.ThrowIfCancellationRequested();
                    var stream = new StreamParser();
                    stream.Write(text[..split]);
                    stream.Write(text[split..]);
                    Assert.Equal(expected, stream.Finish());
                }
            }
        }
    }

    [Fact]
    public static void BinaryReferenceFidelity()
    {
        var codec = new BinaryLinoCodec();
        foreach (var reference in Fixtures())
        {
            var original = new[] { new Link<string>(reference, new[] { new Link<string>(reference) }) };
            var decoded = codec.Decode(codec.Encode(original));
            Assert.Equal(original, decoded);
            Assert.Equal(original, LinoFormat.ParseDocument(LinoFormat.FormatDocument(decoded)));
        }
    }

    [Fact]
    public static void MalformedReferenceLiteralsAreRejected()
    {
        foreach (var literal in File.ReadLines(Path.Combine(AppContext.BaseDirectory, "Protocol", "invalid-reference-literals.txt")).Where(line => !line.StartsWith('#')))
        {
            Assert.Throws<FormatException>(() => ReferenceLiteral.Decode(literal));
            Assert.ThrowsAny<Exception>(() => new Parser().Parse(literal));
        }
        Assert.Equal("é", ReferenceLiteral.Decode("~1{C3A9}"));
        Assert.Throws<EncoderFallbackException>(() => ReferenceLiteral.Encode("\ud800"));
        Assert.Throws<EncoderFallbackException>(() => Link<string>.EscapeReference("\ud800"));
    }
}
```

1.49 ./Link.Foundation.Links.Notation.Tests/SharedTextParityTests.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.Json;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests;

public static class SharedTextParityTests
```

```
{
    private static object[] Tree(IEnumerable<Link<string>> links) => links
        .Select(link => (object)new { id = link.Id, values = Tree(link.Values ??
↪   Array.Empty<Link<string>>()) }).ToArray();

    [Fact(Timeout = 30000)]
    public static void SharedTextParity()
    {
        using var document =
↪   JsonDocument.Parse(File.ReadAllText(Path.Combine(AppContext.BaseDirectory, "Protocol",
↪   "text-fixtures.json")));
        Assert.NotEmpty(document.RootElement.EnumerateArray());
        foreach (var fixture in document.RootElement.EnumerateArray())
        {
            TestContext.Current.CancellationToken.ThrowIfCancellationRequested();
            var input = fixture.GetProperty("input").GetString()!;
            var formatted = fixture.GetProperty("format").GetString()!;
            var expected = fixture.GetProperty("expected");
            var parsed = new Parser().Parse(input);
            Assert.True(JsonElement.DeepEquals(expected,
↪   JsonSerializer.SerializeToElement(Tree(parsed))));
            Assert.Equal(formatted, string.Concat(parsed.Select(link => link.ToString())));
            Assert.True(JsonElement.DeepEquals(expected,
↪   JsonSerializer.SerializeToElement(Tree(new Parser().Parse(formatted)))));
            var stream = new StreamParser();
            foreach (var character in input) stream.Write(character.ToString());
            Assert.True(JsonElement.DeepEquals(expected,
↪   JsonSerializer.SerializeToElement(Tree(stream.Finish()))));
        }
    }
}
```

1.50 ./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs

```
using System;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class SingleLineParserTests
    {
        [Fact]
        public static void SingleLinkTest()
        {
            var source = @"(address: source target)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void TripletSingleLinkTest()
        {
            var source = @"(papa has car)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void BugTest1()
        {
            var source = @"(ignore conan-center-index repository)";
            var links = new Parser().Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void QuotedReferencesTest()
        {
            var source = @"(a: 'b' ""c"")";
            var target = @"(a: b c)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
```

```
        }

        [Fact]
        public static void QuotedReferencesWithSpacesTest()
        {
            var source = @"('a a': 'b b' ""c c"")";
            var target = @"('a a': 'b b' 'c c')";
            var parser = new Parser();
            var links = parser.Parse(source);
            var formattedLinks = links.Format();
            Assert.Equal(target, formattedLinks);
        }

        [Fact]
        public static void ParseSimpleReferenceTest()
        {
            var source = "test";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            // Simple reference creates a singlet link with null Id and one value
            Assert.Null(links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Single(links[0].Values!);
            Assert.Equal("test", links[0].Values?[0].Id);
            Assert.Null(links[0].Values?[0].Values);
        }

        [Fact]
        public static void ParseReferenceWithColonAndValuesTest()
        {
            var source = "parent: child1 child2";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            Assert.Equal("parent", links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Equal(2, links[0].Values!.Count);
            Assert.Equal("child1", links[0].Values![0].Id);
            Assert.Equal("child2", links[0].Values![1].Id);
        }

        [Fact]
        public static void ParseMultilineLinkTest()
        {
            var source = "(parent: child1 child2)";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            Assert.Equal("parent", links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Equal(2, links[0].Values!.Count);
        }

        [Fact]
        public static void ParseValuesOnlyTest()
        {
            var source = ": value1 value2";
            var parser = new Parser();
            // Standalone ':' is now forbidden and should throw an exception
            Assert.Throws<ParseException>(() => parser.Parse(source));
        }

        [Fact]
        public static void SingletLinkTest()
        {
            // Test singlet link
            var input = "(singlet)";
            var result = new Parser().Parse(input);
            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values!);
            Assert.Equal("singlet", result[0].Values?[0].Id);
            Assert.Null(result[0].Values?[0].Values);
```

```
        }

        [Fact]
        public static void ValueLinkTest()
        {
            // Test value link
            var input = "(value1 value2 value3)";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void QuotedReferencesWithSpecialCharsTest()
        {
            // Test quoted references
            var input = @"(""id with spaces"": ""value with spaces"")";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void SingleQuotedReferencesTest()
        {
            // Test single-quoted references
            var input = "('id': 'value')";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void ParseQuotedReferencesValuesOnlyTest()
        {
            var source = "\"has space\" 'has:colon'";
            var parser = new Parser();
            var links = parser.Parse(source);
            Assert.NotNull(links);
            Assert.Single(links);
            Assert.Null(links[0].Id);
            Assert.NotNull(links[0].Values);
            Assert.Equal(2, links[0].Values!.Count);
            Assert.Equal("has space", links[0].Values![0].Id);
            Assert.Equal("has:colon", links[0].Values![1].Id);
            var formatted = links.Format();
            Assert.Equal("('has space' 'has:colon')", formatted);
        }

        [Fact]
        public static void NestedLinksTest()
        {
            // Test nested links
            var input = "(outer: (inner: value))";
            var result = new Parser().Parse(input);
            Assert.NotEmpty(result);
        }

        [Fact]
        public static void HyphenatedIdentifiersTest()
        {
            // Test support for hyphenated identifiers like in BugTest1
            var source = @"(conan-center-index: repository info)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            Assert.Equal(source, target);
        }

        [Fact]
        public static void MultipleWordsInQuotesTest()
        {
            var source = @"(""New York"": city state)";
            var parser = new Parser();
            var links = parser.Parse(source);
            var target = links.Format();
            // Should preserve quotes for multi-word references
            Assert.Contains("New York", target);
        }

        [Fact]
```

```
202         public static void SpecialCharactersInQuotesTest()
203         {
204             var input = @"(""key:with:colons"": ""value(with)parens"")";
205             var result = new Parser().Parse(input);
206             Assert.NotEmpty(result);
207 
208             input = @"('key with spaces': 'value: with special chars')";
209             result = new Parser().Parse(input);
210             Assert.NotEmpty(result);
211         }
212 
213         [Fact]
214         public static void DeeplyNestedTest()
215         {
216             var input = "(a: (b: (c: (d: (e: value)))))";
217             var result = new Parser().Parse(input);
218             Assert.NotEmpty(result);
219         }
220 
221         [Fact]
222         public static void SingleLineWithIdTest()
223         {
224             // Test single-line link with id
225             var input = "id: value1 value2";
226             var result = new Parser().Parse(input);
227             Assert.NotEmpty(result);
228         }
229 
230         [Fact]
231         public static void SingleLineWithoutIdTest()
232         {
233             // Test link without id (single-line) - now forbidden
234             var input = ": value1 value2";
235             Assert.Throws<ParseException>(() => new Parser().Parse(input));
236         }
237 
238         [Fact]
239         public static void MultilineWithoutIdTest()
240         {
241             // Test link without id (multi-line) - now forbidden
242             var input = "(: value1 value2)";
243             Assert.Throws<ParseException>(() => new Parser().Parse(input));
244         }
245 
246         [Fact]
247         public static void SimpleRefTest()
248         {
249             var input = "simple ref";
250             var result = new Parser().Parse(input);
251             Assert.NotEmpty(result);
252         }
253 
254         [Fact]
255         public static void MultiLineLinkWithIdTest()
256         {
257             var input = "(id: value1 value2)";
258             var result = new Parser().Parse(input);
259             Assert.NotEmpty(result);
260         }
261 
262         [Fact]
263         public static void LinkWithoutIdMultiLineTest()
264         {
265             var input = "(: value1 value2)";
266             Assert.Throws<ParseException>(() => new Parser().Parse(input));
267         }
268 
269         [Fact]
270         public static void SimpleReferenceParserTest()
271         {
272             var input = "hello";
273             var result = new Parser().Parse(input);
274             Assert.Single(result);
275             Assert.Null(result[0].Id);
276             Assert.NotNull(result[0].Values);
277             Assert.Single(result[0].Values!);
278             Assert.Equal("hello", result[0].Values?[0].Id);
279         }
280 
```

```
281         [Fact]
282         public static void QuotedReferenceParserTest()
283         {
284             var input = "\"hello world\"";
285             var result = new Parser().Parse(input);
286             Assert.Single(result);
287             Assert.Null(result[0].Id);
288             Assert.NotNull(result[0].Values);
289             Assert.Single(result[0].Values!);
290             Assert.Equal("hello world", result[0].Values?[0].Id);
291         }
292 
293         [Fact]
294         public static void SingletLinkParserTest()
295         {
296             var input = "(singlet)";
297             var result = new Parser().Parse(input);
298             Assert.Single(result);
299             Assert.Null(result[0].Id);
300             Assert.NotNull(result[0].Values);
301             Assert.Single(result[0].Values!);
302             Assert.Equal("singlet", result[0].Values?[0].Id);
303             Assert.Null(result[0].Values?[0].Values);
304         }
305 
306         [Fact]
307         public static void ValueLinkParserTest()
308         {
309             var input = "(a b c)";
310             var result = new Parser().Parse(input);
311             Assert.Single(result);
312             Assert.Null(result[0].Id);
313             Assert.Equal(3, result[0].Values?.Count);
314         }
315 
316         [Fact]
317         public static void ParseQuotedReferencesTest()
318         {
319             var input = "(\"quoted id\": \"value with spaces\")";
320             var parser = new Parser();
321             var result = parser.Parse(input);
322 
323             Assert.NotEmpty(result);
324             var formatted = result.Format();
325             Assert.Contains("quoted id", formatted);
326             Assert.Contains("value with spaces", formatted);
327         }
328 
329         [Fact]
330         public static void LinkWithoutIdSingleLineTest()
331         {
332             // Test that standalone ':' is forbidden and should throw
333             var input = ": value1 value2";
334             var parser = new Parser();
335 
336             // C# parser forbids this syntax (like JS/Rust)
337             Assert.Throws<ParseException>(() => parser.Parse(input));
338         }
339 
340         [Fact]
341         public static void SingleLineLinkWithIdTest()
342         {
343             var input = "id: value1 value2";
344             var parser = new Parser();
345             var result = parser.Parse(input);
346 
347             Assert.NotEmpty(result);
348             Assert.NotNull(result[0].Id);
349             Assert.Equal("id", result[0].Id);
350         }
351 
352         [Fact]
353         public static void LinkWithIdTest()
354         {
355             var input = "(id: a b c)";
356             var parser = new Parser();
357             var result = parser.Parse(input);
358 
359             Assert.Single(result);
```

```
            Assert.Equal("id", result[0].Id);
            Assert.Equal(3, result[0].Values?.Count);
        }

        [Fact]
        public static void QuotedReferenceTest()
        {
            var input = "\"quoted value\"";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
            Assert.NotNull(result[0].Values);
            Assert.Single(result[0].Values!);
            Assert.Equal("quoted value", result[0].Values![0].Id);
        }

        [Fact]
        public static void SimpleReferenceTest()
        {
            var input = "simplereference";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.NotEmpty(result);
        }

        [Fact]
        public static void SingleLineLinkTest()
        {
            var input = "myid: value1 value2";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("myid", result[0].Id);
            Assert.Equal(2, result[0].Values?.Count);
        }

        [Fact]
        public static void ValuesOnlyInParenthesesTest()
        {
            var input = "(value1 value2)";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Null(result[0].Id);
            Assert.Equal(2, result[0].Values?.Count);
        }

        [Fact]
        public static void ParseValuesOnlyStandaloneColonTest()
        {
            // Test that standalone ':' is forbidden and should throw
            var input = ": value1 value2";
            var parser = new Parser();

            // C# parser forbids this syntax (like JS/Rust)
            Assert.Throws<ParseException>(() => parser.Parse(input));
        }

        [Fact]
        public static void QuotedReferencesWithSpacesInLinkTest()
        {
            var input = "(id: \"value with spaces\")";
            var parser = new Parser();
            var result = parser.Parse(input);

            Assert.Single(result);
            Assert.Equal("id", result[0].Id);
            Assert.Single(result[0].Values!);
            Assert.Equal("value with spaces", result[0].Values?[0].Id);
        }
    }
}
```

1.51 ./Link.Foundation.Links.Notation.Tests/StreamParserTests.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Xunit;

namespace Link.Foundation.Links.Notation.Tests
{
    public static class StreamParserTests
    {
        private const string Document =
            "first loves data\n" +
            "# streamed comment\n" +
            "profile:\n" +
            "  name Ada\n" +
            "  note \"line one\nline two\"\n" +
            "(nested:\n  child value)\n" +
            "last sees first";

        private static string[] Render(IEnumerable<Link<string>> links) =>
            links.Select(link => link.ToString()).ToArray();

        [Fact]
        public static void MatchesCanonicalParserOneSymbolAtATime()
        {
            var stream = new StreamParser();
            foreach (var character in Document)
            {
                stream.Write(character.ToString());
            }

            Assert.Equal(Render(new Parser().Parse(Document)), Render(stream.Finish()));
        }

        [Fact]
        public static void EmitsOnlyCompleteRecords()
        {
            var seen = new List<Link<string>>();
            var stream = new StreamParser();
            stream.LinkParsed += seen.Add;

            Assert.Empty(stream.Write("profile:\n  name Ada\n  note \"first\nsecond\"\n"));
            var emitted = stream.Write("n");

            Assert.Single(emitted);
            Assert.Equal(emitted, seen);
        }

        [Fact]
        public static void SupportsLineChunksFinalRecordAndPosition()
        {
            var stream = new StreamParser();
            foreach (var line in Document.SplitAfter('\n'))
            {
                stream.Write(line);
            }

            Assert.Equal(Render(new Parser().Parse(Document)), Render(stream.Finish()));
            Assert.Equal(Document.Length, stream.Position.Offset);
            Assert.Equal(0, stream.Position.Buffered);
        }

        [Fact]
        public static void SupportsDrainResetAndBoundedMemory()
        {
            var seen = new List<Link<string>>();
            var stream = new StreamParser { Collect = false, MaxBufferSize = 8 };
            stream.LinkParsed += seen.Add;
            for (var index = 0; index < 100; index++) stream.Write("a\n");

            Assert.Empty(stream.Drain());
            Assert.Single(stream.Finish());
            Assert.Equal(100, seen.Count);

            stream.Reset();
            Assert.Throws<ArgumentException>(() => { stream.Write("123456789"); });
        }

        [Fact]
```

```
        public static void ProvidesLazyAdapters()
        {
            var actual = StreamParser.ParseChunks(new[] { "one link\n", "two link" });

            Assert.Equal(new[] { "(one link)", "(two link)" }, Render(actual));
        }

        [Fact]
        public static void RejectsWritesAfterFinish()
        {
            var stream = new StreamParser();
            stream.Finish("one");

            Assert.Throws<InvalidOperationException>(() => stream.Write("two"));
        }

        [Fact]
        public static async Task ProvidesAnAsyncEnumerableAdapter()
        {
            static async IAsyncEnumerable<string> Chunks()
            {
                yield return "one link\n";
                await Task.Yield();
                yield return "two link";
            }

            var actual = new List<Link<string>>();
            await foreach (var link in StreamParser.ParseChunksAsync(
                Chunks(),
                cancellationToken: TestContext.Current.CancellationToken)) actual.Add(link);

            Assert.Equal(new[] { "(one link)", "(two link)" }, Render(actual));
        }
    }
}

internal static class StreamingTestStringExtensions
{
    public static IEnumerable<string> SplitAfter(this string value, char delimiter)
    {
        var start = 0;
        for (var index = 0; index < value.Length; index++)
        {
            if (value[index] != delimiter) continue;
            yield return value.Substring(start, index - start + 1);
            start = index + 1;
        }
        if (start < value.Length) yield return value.Substring(start);
    }
}
```

1.52 ./Link.Foundation.Links.Notation.Tests/TupleTests.cs

```
using System.Collections.Generic;
using Xunit;
using LinkType = Link.Foundation.Links.Notation.Link<string>;
using id = Link.Foundation.Links.Notation.id<string>;
using   = Link.Foundation.Links.Notation. <string>;

namespace Link.Foundation.Links.Notation.Tests
{
    public class TupleTests
    {
        [Fact]
        public void TupleToLinkTest()
        {
            var source = @"(papa (lovesMama: loves mama))
(son lovesMama)
(daughter lovesMama)
(all (love mama))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var targetFromString = links.Format();

            IList<LinkType> constructedLinks = new List<LinkType>()
            {
                ("papa", ( )("lovesMama", "loves", "mama")),
                ("son", "lovesMama"),
                ("daughter", "lovesMama"),
                ("all", ("love", "mama")),
```

```
            };
            var targetFromTuples = constructedLinks.Format();
            Assert.Equal(targetFromString, targetFromTuples);
        }

        [Fact]
        public void NamedTupleToLinkTest()
        {
            var source = @"(papa (lovesMama: loves mama))
(son lovesMama)
(daughter lovesMama)
(all (love mama))";
            var parser = new Parser();
            var links = parser.Parse(source);
            var targetFromString = links.Format();

            IList<LinkType> constructedLinks = new List<LinkType>()
            {
                ("papa", ((id)"lovesMama", "loves", "mama")),
                ("son", "lovesMama"),
                ("daughter", "lovesMama"),
                ("all", ("love", "mama")),
            };
            var targetFromTuples = constructedLinks.Format();
            Assert.Equal(targetFromString, targetFromTuples);
        }
    }
}
```

Index

./Link.Foundation.Links.Notation.Tests/ApiTests.cs, 52
./Link.Foundation.Links.Notation.Tests/BinaryLinksNotationTests.cs, 55
./Link.Foundation.Links.Notation.Tests/BinaryLinoCodecTests.cs, 61
./Link.Foundation.Links.Notation.Tests/CommentsTests.cs, 66
./Link.Foundation.Links.Notation.Tests/EdgeCaseParserTests.cs, 68
./Link.Foundation.Links.Notation.Tests/EmptyReferenceTests.cs, 71
./Link.Foundation.Links.Notation.Tests/FormatConfigTests.cs, 72
./Link.Foundation.Links.Notation.Tests/IndentationConsistencyTests.cs, 74
./Link.Foundation.Links.Notation.Tests/IndentedIdNestedValuesTests.cs, 76
./Link.Foundation.Links.Notation.Tests/IndentedIdSyntaxTests.cs, 76
./Link.Foundation.Links.Notation.Tests/LinkTests.cs, 79
./Link.Foundation.Links.Notation.Tests/LinksGroupTests.cs, 80
./Link.Foundation.Links.Notation.Tests/MixedIndentationModesTests.cs, 82
./Link.Foundation.Links.Notation.Tests/MultiQuoteParserTests.cs, 84
./Link.Foundation.Links.Notation.Tests/MultilineParserTests.cs, 89
./Link.Foundation.Links.Notation.Tests/MultilineQuotedStringTests.cs, 91
./Link.Foundation.Links.Notation.Tests/NestedIndentationTests.cs, 92
./Link.Foundation.Links.Notation.Tests/NestedParserTests.cs, 94
./Link.Foundation.Links.Notation.Tests/NestedSelfReferenceTests.cs, 97
./Link.Foundation.Links.Notation.Tests/NestingDepthTests.cs, 98
./Link.Foundation.Links.Notation.Tests/NestingLimitTests.cs, 100
./Link.Foundation.Links.Notation.Tests/ParseErrorPositionTests.cs, 102
./Link.Foundation.Links.Notation.Tests/ReferenceFidelityTests.cs, 104
./Link.Foundation.Links.Notation.Tests/SharedTextParityTests.cs, 105
./Link.Foundation.Links.Notation.Tests/SingleLineParserTests.cs, 106
./Link.Foundation.Links.Notation.Tests/StreamParserTests.cs, 111
./Link.Foundation.Links.Notation.Tests/TupleTests.cs, 113
./Link.Foundation.Links.Notation/Binary/ArityRange.cs, 1
./Link.Foundation.Links.Notation/Binary/BinaryLinoCodec.cs, 3
./Link.Foundation.Links.Notation/Binary/BinaryNotationException.cs, 3
./Link.Foundation.Links.Notation/Binary/LinksPacket.cs, 4
./Link.Foundation.Links.Notation/Binary/LinoFormat.cs, 12
./Link.Foundation.Links.Notation/Binary/LinoMapping.cs, 14
./Link.Foundation.Links.Notation/Binary/PacketReader.cs, 22
./Link.Foundation.Links.Notation/Binary/SectionPlanner.cs, 23
./Link.Foundation.Links.Notation/Comments.cs, 25
./Link.Foundation.Links.Notation/FormatConfig.cs, 26
./Link.Foundation.Links.Notation/FormatOptions.cs, 26
./Link.Foundation.Links.Notation/FurthestFailureTracer.cs, 27
./Link.Foundation.Links.Notation/ILinksGroupListExtensions.cs, 28
./Link.Foundation.Links.Notation/IListExtensions.cs, 30
./Link.Foundation.Links.Notation/Link.cs, 32
./Link.Foundation.Links.Notation/LinkFormatExtensions.cs, 35
./Link.Foundation.Links.Notation/LinksGroup.cs, 37
./Link.Foundation.Links.Notation/ParseException.cs, 40
./Link.Foundation.Links.Notation/Parser.cs, 42
./Link.Foundation.Links.Notation/Quotes.cs, 44
./Link.Foundation.Links.Notation/ReferenceLiteral.cs, 46
./Link.Foundation.Links.Notation/StreamParseException.cs, 47
./Link.Foundation.Links.Notation/StreamParser.cs, 47
./Link.Foundation.Links.Notation/ .cs, 51
./Link.Foundation.Links.Notation/id.cs, 52